book_to_slide_BY_sections_V12

July 18, 2025

1 Set up Paths

[23]:
```
# Cell 1: Setup and Configuration
import os
import re
import logging
import warnings
from docx import Document
import pdfplumber
import ollama
from tenacity import retry, stop_after_attempt, wait_exponential, RetryError
import json
import logging
# Setup Logger for this cell
logging.basicConfig(level=logging.INFO, format='%(asctime)s - %(levelname)s -␣
 ↪%(message)s')
logger = logging.getLogger(__name__)

# Extract Unit ID from the filename
# --- Helper Functions ---
def print_header(text: str, char: str = "="):
    """Prints a centered header to the console."""
    print("\n" + char * 80)
    print(text.center(80))
    print(char * 80)

def extract_uo_id_from_filename(filename: str) -> str:
    match = re.match(r'^[A-Z]+\d+', os.path.basename(filename))
    if match:
        return match.group(0)
    raise ValueError(f"Could not extract a valid Unit ID from filename:␣
 ↪'{filename}'")


# To define outputs folder
TEST_MODE = False
```

```
# --- 1. CORE SETTINGS ---
# Set this to True for EPUB, False for PDF. This controls the entire notebook's␣
 ↪flow.
PROCESS_EPUB = True # for EPUB
# PROCESS_EPUB = False # for PDF

# Control process for testing
EXTRACT_UO = True
CREATE_RAG_BOOK = True

#==================================INPUTS==================================


# --- 2. INPUT FILE NAMES ---
# The name of the Unit Outline file (e.g., DOCX, PDF)
UNIT_OUTLINE_FILENAME = "ICT312 Digital Forensic_Final.docx" # epub
# UNIT_OUTLINE_FILENAME = "ICT311 Applied Cryptography.docx" # pdf


# The names of the book files
EPUB_BOOK_FILENAME = "Bill Nelson, Amelia Phillips, Christopher Steuart - Guide␣
 ↪to Computer Forensics and Investigations_ Processing Digital␣
 ↪Evidence-Cengage Learning (2018).epub"
PDF_BOOK_FILENAME = "(Chapman & Hall_CRC Cryptography and Network Security␣
 ↪Series) Jonathan Katz, Yehuda Lindell - Introduction to Modern␣
 ↪Cryptography-CRC Press (2020).pdf"

# --- 3. DIRECTORY STRUCTURE ---
# Define the base path to your project to avoid hardcoding long paths everywhere
PROJECT_BASE_DIR = "/home/sebas_dev_linux/projects/course_generator"


CONFIG_DIR = os.path.join(PROJECT_BASE_DIR, "configs")
SETTINGS_DECK_PATH = os.path.join(CONFIG_DIR, "settings_deck.json")
#TEACHING_FLOWS_PATH = os.path.join(CONFIG_DIR, "teaching_flows.json") # ␣
 ↪future add

# to check the layauts
LAYOUT_MAPPING_PATH = os.path.join(CONFIG_DIR, "layout_mapping_test_Mod.json.
 ↪json") #  using the modified one

# New output path for the processed settings
PROCESSED_SETTINGS_PATH = os.path.join(CONFIG_DIR, "processed_settings.json")

# Define subdirectories relative to the base path
DATA_DIR_INPUT = os.path.join(PROJECT_BASE_DIR, "data")
```

```
INPUT_UO_DIR = os.path.join(DATA_DIR_INPUT, "UO")
INPUT_BOOKS_DIR = os.path.join(DATA_DIR_INPUT, "books")

#==================================OUTPUTS=================================

try:
    UNIT_ID = extract_uo_id_from_filename(UNIT_OUTLINE_FILENAME)
except ValueError as e:
    print(f"Error: {e}")
    UNIT_ID = "UNKNOWN_ID"




# Define outputs

if TEST_MODE:
    DATA_DIR_OUTPUT = os.path.join(PROJECT_BASE_DIR, "data_output/test_Output")
else:
    DATA_DIR_OUTPUT = os.path.join(PROJECT_BASE_DIR, "data_output")

os.makedirs(DATA_DIR_OUTPUT, exist_ok=True)
PARSE_DATA_DIR = os.path.join(PROJECT_BASE_DIR, DATA_DIR_OUTPUT, "1.␣
 ↪Parse_data")
os.makedirs(PARSE_DATA_DIR, exist_ok=True)

# Construct full paths for clarity

OUTPUT_PARSED_UO_DIR = os.path.join(PARSE_DATA_DIR, f"Parse_UO/{UNIT_ID}")
os.makedirs(OUTPUT_PARSED_UO_DIR, exist_ok=True)
OUTPUT_PARSED_TOC_DIR = os.path.join(PARSE_DATA_DIR, f"Parse_TOC_books/
 ↪{UNIT_ID}")
os.makedirs(OUTPUT_PARSED_TOC_DIR, exist_ok=True)
OUTPUT_DB_DIR = os.path.join(DATA_DIR_OUTPUT, f"0. DataBase_Chroma/{UNIT_ID}")
os.makedirs(OUTPUT_DB_DIR, exist_ok=True)


# to Save the individual FINAL plan to a file
PLAN_OUTPUT_DIR = os.path.join(DATA_DIR_OUTPUT, f"2. generated_plans/{UNIT_ID}")
os.makedirs(PLAN_OUTPUT_DIR, exist_ok=True)

#to Save the individual FINAL Content to a file
CONTENT_OUTPUT_DIR = os.path.join(DATA_DIR_OUTPUT, f"3. generated_content/
 ↪{UNIT_ID}")
os.makedirs(CONTENT_OUTPUT_DIR, exist_ok=True)
```

```
CONTENT_LLM_OUTPUT_DIR = os.path.join(DATA_DIR_OUTPUT, f"4.␣
 ↪generated_content_llm/{UNIT_ID}")
os.makedirs(CONTENT_LLM_OUTPUT_DIR, exist_ok=True)


SLIDE_TEMPLATE_PATH = "/home/sebas_dev_linux/projects/course_generator/data/
 ↪slide_style/slide_style_test_2.pptx"
FINAL_PRESENTATION_DIR = os.path.join(DATA_DIR_OUTPUT, f"5. final_presentations/
 ↪{UNIT_ID}")
os.makedirs(FINAL_PRESENTATION_DIR, exist_ok=True)


# --- 4. LLM & EMBEDDING CONFIGURATION ---
LLM_PROVIDER = "ollama"  # Can be "ollama", "openai", "gemini"
OLLAMA_HOST = "http://localhost:11434"
OLLAMA_MODEL = "qwen3:8b" # "qwen3:8b", #"mistral:latest"
EMBEDDING_MODEL_OLLAMA = "nomic-embed-text"
CHUNK_SIZE = 800
CHUNK_OVERLAP = 100

# --- 5. DYNAMICALLY GENERATED PATHS & IDs (DO NOT EDIT THIS SECTION) ---
# This section uses the settings above to create all the necessary variables␣
 ↪for later cells.


# Full path to the unit outline file
FULL_PATH_UNIT_OUTLINE = os.path.join(INPUT_UO_DIR, UNIT_OUTLINE_FILENAME)

# Determine which book and output paths to use based on the PROCESS_EPUB flag
if PROCESS_EPUB:
    BOOK_PATH = os.path.join(INPUT_BOOKS_DIR, EPUB_BOOK_FILENAME)
    PRE_EXTRACTED_TOC_JSON_PATH = os.path.join(OUTPUT_PARSED_TOC_DIR,␣
 ↪f"{UNIT_ID}_epub_table_of_contents.json")
else:
    BOOK_PATH = os.path.join(INPUT_BOOKS_DIR, PDF_BOOK_FILENAME)
    PRE_EXTRACTED_TOC_JSON_PATH = os.path.join(OUTPUT_PARSED_TOC_DIR,␣
 ↪f"{UNIT_ID}_pdf_table_of_contents.json")

# Define paths for the vector database
file_type_suffix = 'epub' if PROCESS_EPUB else 'pdf'
CHROMA_PERSIST_DIR = os.path.join(OUTPUT_DB_DIR,␣
 ↪f"{UNIT_ID}_chroma_db_toc_chunks_{file_type_suffix}")
CHROMA_COLLECTION_NAME = f"book_toc_chunks_{file_type_suffix}"
```

```python
# Define path for the parsed unit outline
PARSED_UO_JSON_PATH = os.path.join(OUTPUT_PARSED_UO_DIR, f"{os.path.
 ↪splitext(UNIT_OUTLINE_FILENAME)[0]}_parsed.json")

# --- Sanity Check Printout ---
print("--- CONFIGURATION SUMMARY ---")
print(f"Processing Mode: {'  EPUB' if PROCESS_EPUB else ' PDF'}")
print(f"  Unit ID: {UNIT_ID}")
print(f" Unit Outline Path: {FULL_PATH_UNIT_OUTLINE}")
print(f" Book Path: {BOOK_PATH}")
print(f"  Parsed UO Output Path: {PARSED_UO_JSON_PATH}")
print(f"  Parsed ToC Output Path: {PRE_EXTRACTED_TOC_JSON_PATH}")
print(f" Vector DB Path: {CHROMA_PERSIST_DIR}")
print(f" Vector DB Collection: {CHROMA_COLLECTION_NAME}")
print(f"  Plan Develop output: {PLAN_OUTPUT_DIR}")
print(f"  Plan Fetched output: {CONTENT_OUTPUT_DIR}")
print(f"  Plan Generated LLM output: {CONTENT_LLM_OUTPUT_DIR}")
print(f"  Final slides: {FINAL_PRESENTATION_DIR}")
print("--- SETUP COMPLETE ---")
```

```
--- CONFIGURATION SUMMARY ---
Processing Mode:   EPUB
  Unit ID: ICT312
 Unit Outline Path:
/home/sebas_dev_linux/projects/course_generator/data/UO/ICT312 Digital
Forensic_Final.docx
 Book Path: /home/sebas_dev_linux/projects/course_generator/data/books/Bill
Nelson, Amelia Phillips, Christopher Steuart - Guide to Computer Forensics and
Investigations_ Processing Digital Evidence-Cengage Learning (2018).epub
  Parsed UO Output Path:
/home/sebas_dev_linux/projects/course_generator/data_output/1.
Parse_data/Parse_UO/ICT312/ICT312 Digital Forensic_Final_parsed.json
  Parsed ToC Output Path:
/home/sebas_dev_linux/projects/course_generator/data_output/1.
Parse_data/Parse_TOC_books/ICT312/ICT312_epub_table_of_contents.json
 Vector DB Path: /home/sebas_dev_linux/projects/course_generator/data_output/0.
DataBase_Chroma/ICT312/ICT312_chroma_db_toc_chunks_epub
 Vector DB Collection: book_toc_chunks_epub
  Plan Develop output:
/home/sebas_dev_linux/projects/course_generator/data_output/2.
generated_plans/ICT312
  Plan Fetched output:
/home/sebas_dev_linux/projects/course_generator/data_output/3.
generated_content/ICT312
  Plan Generated LLM output:
/home/sebas_dev_linux/projects/course_generator/data_output/4.
generated_content_llm/ICT312
  Final slides: /home/sebas_dev_linux/projects/course_generator/data_output/5.
```

```
final_presentations/ICT312
--- SETUP COMPLETE ---
```

2 System Prompt

[19]:
```
UNIT_OUTLINE_SYSTEM_PROMPT_TEMPLATE = """
You are an expert academic assistant tasked with parsing a university unit␣
 ↪outline document and extracting key information into a structured JSON␣
 ↪format.

The input will be the raw text content of a unit outline. Your goal is to␣
 ↪identify and extract the following details and structure them precisely as␣
 ↪specified in the JSON schema below. Note: do not change any key name

**JSON Output Schema:**

```json
{{
  "unitInformation": {{
    "unitCode": "string | null",
    "unitName": "string | null",
    "creditPoints": "integer | null",
    "unitRationale": "string | null",
    "prerequisites": "string | null"
  }},
  "learningOutcomes": [
    "string"
  ],
  "assessments": [
    {{
      "taskName": "string",
      "description": "string",
      "dueWeek": "string | null",
      "weightingPercent": "integer | null",
      "learningOutcomesAssessed": "string | null"
    }}
  ],
  "weeklySchedule": [
    {{
      "week": "string",
      "contentTopic": "string",
      "requiredReading": "string | null"
    }}
  ],
  "requiredReadings": [
    "string"
```
```

```
  ],
  "recommendedReadings": [
    "string"
  ]
}}

Instructions for Extraction:
Unit Information: Locate Unit Code, Unit Name, Credit Points. Capture 'Unit␣
 ↪Overview / Rationale' as unitRationale. Identify prerequisites.
Learning Outcomes: Extract each learning outcome statement.
Assessments: Each task as an object. Capture full task name, description, Due␣
 ↪Week, Weighting % (number), and Learning Outcomes Assessed.
weeklySchedule: Each week as an object. Capture Week, contentTopic, and␣
 ↪requiredReading.
Required and Recommended Readings: List full text for each.
**Important Considerations for the LLM**:
Pay close attention to headings and table structures.
If information is missing, use null for string/integer fields, or an empty list␣
 ↪[] for array fields.
Do no change keys in the template given
Ensure the output is ONLY the JSON object, starting with {{{{ and ending with␣
 ↪}}}}. No explanations or conversational text before or after the JSON.
Now, parse the following unit outline text:
--- UNIT_OUTLINE_TEXT_START ---
{outline_text}
--- UNIT_OUTLINE_TEXT_END ---
"""
```

[20]:
```python
# Place this in a new cell after your imports, or within Cell 3 before the␣
 ↪functions.
# This code is based on the schema from your screenshot on page 4.

from pydantic import BaseModel, Field, ValidationError
from typing import List, Optional
import time

# Define Pydantic models that match your JSON schema
class UnitInformation(BaseModel):
    unitCode: Optional[str] = None
    unitName: Optional[str] = None
    creditPoints: Optional[int] = None
    unitRationale: Optional[str] = None
    prerequisites: Optional[str] = None

class Assessment(BaseModel):
    taskName: str
    description: str
```

```
    dueWeek: Optional[str] = None
    weightingPercent: Optional[int] = None
    learningOutcomesAssessed: Optional[str] = None

class WeeklyScheduleItem(BaseModel):
    week: str
    contentTopic: str
    requiredReading: Optional[str] = None

class ParsedUnitOutline(BaseModel):
    unitInformation: UnitInformation
    learningOutcomes: List[str]
    assessments: List[Assessment]
    weeklySchedule: List[WeeklyScheduleItem]
    requiredReadings: List[str]
    recommendedReadings: List[str]
```

3 Helper functions

[24]:
```
# Cell 10: Configuration and Scoping for Content Generation (Corrected)

import os
import json
import logging

# Setup Logger for this cell
logging.basicConfig(level=logging.INFO, format='%(asctime)s - %(levelname)s -␣
 ↪%(message)s')
logger = logging.getLogger(__name__)



# --- Global Test Overrides (for easy testing) ---
TEST_OVERRIDE_WEEKS = None
TEST_OVERRIDE_FLOW_ID = None
TEST_OVERRIDE_SESSIONS_PER_WEEK = None
TEST_OVERRIDE_DISTRIBUTION_STRATEGY = None


def process_and_load_configurations():
    """
    PHASE 1: Loads configurations, calculates a PRELIMINARY time-based slide␣
 ↪budget,
    and saves the result as 'processed_settings.json' for the Planning Agent.
    """
    print_header("Phase 1: Configuration and Scoping Process", char="-")
```

```
    # --- Load all input files ---
    logger.info("Loading all necessary configuration and data files...")
    try:
        os.makedirs(CONFIG_DIR, exist_ok=True)
        with open(PARSED_UO_JSON_PATH, 'r', encoding='utf-8') as f:␣
↪unit_outline = json.load(f)
        with open(PRE_EXTRACTED_TOC_JSON_PATH, 'r', encoding='utf-8') as f:␣
↪book_toc = json.load(f)
        with open(SETTINGS_DECK_PATH, 'r', encoding='utf-8') as f:␣
↪settings_deck = json.load(f)
        # with open(TEACHING_FLOWS_PATH, 'r', encoding='utf-8') as f:␣
↪teaching_flows = json.load(f)
        logger.info("All files loaded successfully.")
    except FileNotFoundError as e:
        logger.error(f"FATAL: A required configuration file was not found: {e}")
        return None

    # --- Pre-process and Refine Settings ---
    logger.info("Pre-processing settings_deck for definitive plan...")
    processed_settings = json.loads(json.dumps(settings_deck))

    unit_info = unit_outline.get("unitInformation", {})
    processed_settings['course_id'] = unit_info.get("unitCode",␣
↪"UNKNOWN_COURSE")
    processed_settings['unit_name'] = unit_info.get("unitName", "Unknown Unit␣
↪Name")

    # --- Apply test overrides IF they are not None ---
    logger.info("Applying overrides if specified...")
    # This block now correctly sets the teaching_flow_id based on the␣
↪interactive flag.
    if TEST_OVERRIDE_FLOW_ID is not None:
        processed_settings['teaching_flow_id'] = TEST_OVERRIDE_FLOW_ID
        logger.info(f"OVERRIDE: teaching_flow_id set to␣
↪'{TEST_OVERRIDE_FLOW_ID}'")
    else:
        # If no override, use the 'interactive' boolean from the file as the␣
↪source of truth.
        is_interactive = processed_settings.get('interactive', False)
        if is_interactive:
            processed_settings['teaching_flow_id'] = 'apply_topic_interactive'
        else:
            processed_settings['teaching_flow_id'] = 'standard_lecture'
        logger.info(f"Loaded from settings: 'interactive' is {is_interactive}.␣
↪Set teaching_flow_id to '{processed_settings['teaching_flow_id']}'.")
```

```
    # The 'interactive' flag is now always consistent with the teaching_flow_id.
    processed_settings['interactive'] = "interactive" in␣
↪processed_settings['teaching_flow_id'].lower()

    if TEST_OVERRIDE_SESSIONS_PER_WEEK is not None:
        processed_settings['week_session_setup']['sessions_per_week'] =␣
↪TEST_OVERRIDE_SESSIONS_PER_WEEK
        logger.info(f"OVERRIDE: sessions_per_week set to␣
↪{TEST_OVERRIDE_SESSIONS_PER_WEEK}")

    if TEST_OVERRIDE_DISTRIBUTION_STRATEGY is not None:
        processed_settings['week_session_setup']['distribution_strategy'] =␣
↪TEST_OVERRIDE_DISTRIBUTION_STRATEGY
        logger.info(f"OVERRIDE: distribution_strategy set to␣
↪'{TEST_OVERRIDE_DISTRIBUTION_STRATEGY}'")

    if TEST_OVERRIDE_WEEKS is not None:
        processed_settings['generation_scope']['weeks'] = TEST_OVERRIDE_WEEKS
        logger.info(f"OVERRIDE: generation_scope weeks set to␣
↪{TEST_OVERRIDE_WEEKS}")




    # --- DYNAMIC SLIDE BUDGET CALCULATION (Phase 1) ---
    logger.info("Calculating preliminary slide budget based on session time...")

    params = processed_settings.get('parameters_slides', {})
    SLIDES_PER_HOUR = params.get('slides_per_hour', 18)

    duration_hours = processed_settings['week_session_setup'].
↪get('session_time_duration_in_hour', 1.0)
    sessions_per_week = processed_settings['week_session_setup'].
↪get('sessions_per_week', 1)

    logger.info(f" Duration Hours {duration_hours}.")
    logger.info(f" Sessions per week {sessions_per_week}.")

    slides_content_per_session = int(duration_hours * SLIDES_PER_HOUR)
    target_total_slides = slides_content_per_session * sessions_per_week

    processed_settings['slide_count_strategy']['target_total_slides'] =␣
↪target_total_slides
    processed_settings['slide_count_strategy']['slides_content_per_session'] =␣
↪slides_content_per_session
```

```
    logger.info(f" Preliminary weekly content slide target calculated:␣
↪# {target_total_slides} slides.")

    # --- Resolve Generation Scope if not overridden ---
    if TEST_OVERRIDE_WEEKS is None and processed_settings.
↪get('generation_scope', {}).get('weeks') == "all":
        num_weeks = len(unit_outline.get('weeklySchedule', []))
        processed_settings['generation_scope']['weeks'] = list(range(1,␣
↪num_weeks + 1))

    # --- Save the processed settings to disk ---
    logger.info(f"Saving preliminary processed configuration to:␣
↪{PROCESSED_SETTINGS_PATH}")
    with open(PROCESSED_SETTINGS_PATH, 'w', encoding='utf-8') as f:
        json.dump(processed_settings, f, indent=2)
    logger.info("File saved successfully.")

    # --- Assemble master config for optional preview ---
    master_config = {
        "processed_settings": processed_settings,
        "unit_outline": unit_outline,
        "book_toc": book_toc,
        # "teaching_flows": teaching_flows
    }

    print_header("Phase 1 Configuration Complete", char="-")
    logger.info("Master configuration object is ready for the Planning Agent.")
    return master_config
```

3.0.1 Component: Definitive PowerPoint Layout Inspector

Component: Definitive PowerPoint Layout Inspector Primary Purpose The inspect_and_generate_layout_config function serves as a critical pre-processing utility for the automated presentation generation system. Its primary purpose is to bridge the gap between a visual PowerPoint template and the programmatic logic of the content generation agents. It achieves this by performing a deep inspection of a given PowerPoint (.pptx) template file and auto-generating a detailed, structured, and human-readable JSON configuration file (layout_mapping.json). This configuration file acts as the "API documentation" for the presentation template, allowing both human users and a Large Language Model (LLM) to understand and utilize the available slide layouts effectively. Key Functions and GoOf course. Here is a formal description of the purpose and functionality of the "Definitive PowerPoint Layout Inspector" script. This description is suitable for project documentation, a README file, or for explaining its role to other developers.

Primary Purpose The `inspect_and_generate_layout_config` function serves as a critical pre-processing utility for the automated presentation generation system. Its primary purpose is to **bridge the gap between a visual PowerPoint template and the programmatic logic of**

the content generation agents.

It achieves this by performing a deep inspection of a given PowerPoint (`.pptx`) template file and auto-generating a detailed, structured, and human-readable JSON configuration file (`layout_mapping.json`). This configuration file acts as the "API documentation" for the presentation template, allowing both human users and a Large Language Model (LLM) to understand and utilize the available slide layouts effectively.

Key Functions and Goals

1. **Comprehensive Layout Discovery:**
   - The script guarantees that **every single slide layout** present in the PowerPoint template's Slide Master is detected and analyzed. This prevents a common problem where unused or unconventionally named layouts might be missed by simpler scripts.
2. **Detailed Placeholder Analysis:**
   - For each layout, the script extracts an exhaustive list of all its placeholders. For every placeholder, it records crucial metadata:
     - `type`: The functional role of the placeholder (e.g., `TITLE`, `BODY`, `OBJECT`, `TABLE`, `PICTURE`).
     - `name`: The unique name given to the placeholder in the PowerPoint interface (e.g., "Title 1", "Content Placeholder 2").
     - `idx`: The internal identification number of the placeholder.
     - `position and size`: The physical coordinates (`left`, `top`) and dimensions (`width`, `height`), converted to an intuitive unit (inches) for easy comprehension of the layout's visual structure.
3. **Intelligent Capability Summarization:**
   - The script's core innovation is its ability to generate a **machine-readable capabilities summary** for each layout. Instead of just listing raw data, it synthesizes the placeholder information into a concise description of what the layout is designed for. For example:
     - `{"title_support": "standard_title", "body_layout": "2_column"}`
     - `{"title_support": "centered_title_with_subtitle", "body_layout": "no_body"}`
     - `{"specific_content_types": ["TABLE", "CHART"]}`
   - This summary is specifically designed to be passed to an LLM as part of a prompt, enabling the LLM to make an informed, logical choice about the best layout for presenting a given piece of information.
4. **User-Friendly Configuration:**
   - While providing a detailed summary for the LLM, the script also generates a simplified `user_selections` section. This allows a human operator to easily map the system's semantic slide types (e.g., "Agenda", "Summary") to a specific layout index, providing a robust fallback and manual override capability.

How It Solves Critical Problems

- **Eliminates Ambiguity:** By capturing the name, index, and position of every placeholder, it solves the problem of layouts with multiple placeholders of the same type (e.g., two content boxes). The system can now programmatically target the "left column" vs. the "right column".
- **Decouples Logic from Design:** The presentation generation agent no longer needs hard-coded assumptions about the template's design. All the logic for choosing and populating

layouts is driven by the generated JSON file. This means the visual template can be updated or completely replaced without requiring changes to the core Python code.

- **Empowers the LLM:** It transforms a visual, unstructured design asset (the `.pptx` file) into a structured, well-defined set of "tools" (the layouts) that an LLM can reason about. This is the key to enabling more advanced tasks where the LLM doesn't just fill in content, but also makes decisions about the *visual structure* of the presentation.

In summary, the Definitive PowerPoint Layout Inspector is the foundational component that makes the entire presentation generation process intelligent, configurable, and robust. It translates the abstract design of a template into concrete, actionable data.

[25]:
```
# this process the layauts from the slides provide to use them to process the␣
 ↪plan (This require be change to the course of the system  )
# https://aistudio.google.com/prompts/18YaU5pG96eFMbM1l6yeG1v9j63PkLc5H

import logging
import os
import json
import logging
import random
from pptx import Presentation
from pptx.util import Inches
from pptx.enum.shapes import MSO_SHAPE_TYPE
from pptx.enum.shapes import PP_PLACEHOLDER
# Setup Logger for this cell
logging.basicConfig(level=logging.INFO, format='%(asctime)s - %(levelname)s -␣
 ↪%(message)s')
logger = logging.getLogger(__name__)
def print_header(text: str, char: str = "="):
    """Prints a centered header to the console."""
    print("\n" + char * 80)
    print(text.center(80))
    print(char * 80)

def generate_layout_capabilities(layout_name: str, placeholders: list) -> dict:

    essential_placeholders = [p for p in placeholders if p['type'] !=␣
 ↪'SLIDE_NUMBER']
    capabilities = {"title_support": "none", "body_layout": "none",␣
 ↪"specific_content_types": []}
    has_center_title = any(p['type'] == 'CENTER_TITLE' for p in␣
 ↪essential_placeholders)
    has_standard_title = any(p['type'] == 'TITLE' for p in␣
 ↪essential_placeholders)
    has_subtitle = any(p['type'] == 'SUBTITLE' for p in essential_placeholders)
    if has_center_title: capabilities["title_support"] = "centered_title"
    elif has_standard_title: capabilities["title_support"] = "standard_title"
```

```
    if has_subtitle: capabilities["title_support"] += "_with_subtitle"
    body_placeholders = [p for p in essential_placeholders if p['type'] not in␣
↪('TITLE', 'CENTER_TITLE', 'SUBTITLE')]
    if len(body_placeholders) == 0: capabilities["body_layout"] = "no_body"
    elif len(body_placeholders) == 1: capabilities["body_layout"] =␣
↪"single_column"
    elif len(body_placeholders) > 1:
        body_placeholders.sort(key=lambda p: p['left'])
        if len(body_placeholders) > 1 and body_placeholders[1]['left'] >␣
↪(body_placeholders[0]['left'] + body_placeholders[0]['width'] * 0.5):
            capabilities["body_layout"] = f"{len(body_placeholders)}_column"
        else: capabilities["body_layout"] = "stacked_sections"
    specific_types = {p['type'] for p in body_placeholders if p['type'] not in␣
↪('BODY', 'OBJECT')}
    if specific_types: capabilities["specific_content_types"] =␣
↪sorted(list(specific_types))
    return capabilities


def inspect_and_generate_layout_config(template_path: str, output_path: str):
    """
    Inspects a template, generates a machine-readable capabilities summary, and␣
↪creates
    a complete and definitive JSON configuration file for user and LLM use.
    """
    try:
        prs = Presentation(template_path)
    except Exception as e:
        logger.error(f"Could not open or parse the presentation template at␣
↪'{template_path}'. Error: {e}")
        return

    # --- 1. Analyze all Layouts ---

    available_layouts = []
    for i, layout in enumerate(prs.slide_layouts):
        placeholder_details = []
        for p in layout.placeholders:
            placeholder_details.append({
                "idx": p.placeholder_format.idx, "type": p.placeholder_format.
↪type.name,
                "name": p.name, "left": round(p.left.inches, 2), "top": round(p.
↪top.inches, 2),
                "width": round(p.width.inches, 2), "height": round(p.height.
↪inches, 2)
            })
```

```
        capabilities = generate_layout_capabilities(layout.name,␣
↪placeholder_details)
        layout_info = {"layout_index": i, "layout_name": layout.name,␣
↪"capabilities": capabilities, "placeholders": placeholder_details}
        available_layouts.append(layout_info)

    # --- 2. Create the Complete User-Facing Configuration Structure ---
    def find_default_by_capability(layouts, capability_key, capability_value,␣
↪fallback_index=1):
        for layout in layouts:
            if layout['capabilities'].get(capability_key) == capability_value:
                return layout['layout_index']
        # If no perfect match is found, return a safe fallback index
        return fallback_index if len(layouts) > fallback_index else 0

    # ** This map is now complete, explicitly including all required keys **
    user_selection_map = {
        "Title": { # title and subtitle
            "description": "Main title slide of the deck.",
            "selected_layout_index":␣
↪find_default_by_capability(available_layouts, "title_support",␣
↪"centered_title_with_subtitle")
        },
        "Agenda": { # title and object
            "description": "Agenda/Table of Contents slide.",
            "selected_layout_index":␣
↪find_default_by_capability(available_layouts, "body_layout", "single_column")
        },
        "Content": { # title and object
            "description": "Default slide for a standard topic with bullet␣
↪points.",
            "selected_layout_index":␣
↪find_default_by_capability(available_layouts, "body_layout", "single_column")
        },
        "Content_Two_Column": { # title and 2 objects
            "description": "Slide for side-by-side content, like comparisons.",
            "selected_layout_index":␣
↪find_default_by_capability(available_layouts, "body_layout", "2_column")
        },
        "Content_child": { ## # title, subtitle and object
            "description": "Default slide for a standard topic with bullet␣
↪points.",
            "selected_layout_index":␣
↪find_default_by_capability(available_layouts, "body_layout", "single_column")
        },
        "Content_Two_Column_child": { ## title, subtitle and 2 objects
```

```
            "description": "Slide for side-by-side content, like comparisons.",
            "selected_layout_index":␣
↪find_default_by_capability(available_layouts, "body_layout", "2_column")
        },
        "Application": {# title and  object
            "description": "Slide for interactive questions ('Let's Apply This!
↪').",
            "selected_layout_index":␣
↪find_default_by_capability(available_layouts, "body_layout", "single_column")
        },
        "Application_Two_Column ": { # title and 2 objects
            "description": "Slide for side-by-side content ('Let's Apply This!
↪').",
            "selected_layout_index":␣
↪find_default_by_capability(available_layouts, "body_layout", "2_column")
        },
        "Summary": { # title and object
            "description": "The final summary slide of the deck.",
            "selected_layout_index":␣
↪find_default_by_capability(available_layouts, "body_layout", "single_column")
        },
        "End": { # title
            "description": "The final 'Thank You / Questions?' slide.",
            "selected_layout_index":␣
↪find_default_by_capability(available_layouts, "title_support",␣
↪"centered_title", fallback_index=7)
        },
        "Divider": { # title and object
            "description": "Slide that divide sections general use base on the␣
↪agenda.",
            "selected_layout_index":␣
↪find_default_by_capability(available_layouts, "title_support",␣
↪"centered_title", fallback_index=7)
        }
    }

    config_to_save = {
        "//_INSTRUCTIONS": "This file describes the available slide layouts.␣
↪The 'available_layouts' section is for an LLM to read. The 'user_selections'␣
↪section is for you to edit. Please verify the 'selected_layout_index' for␣
↪each slide type.",
        "template_file": os.path.basename(template_path),
        "user_selections": user_selection_map,
        "available_layouts": available_layouts
    }
```

```
    # --- 3. Save the Configuration File ---
    try:
        with open(output_path, 'w', encoding='utf-8') as f:
            json.dump(config_to_save, f, indent=4)
        print_header("Configuration Generated Successfully", char="=")
        print(f"A new, human-readable configuration file has been saved to:
 ↪\n{output_path}")
        print("\nPlease open this file to review the classifications and edit␣
 ↪your layout selections.")
    except Exception as e:
        logger.error(f"Failed to write the layout configuration file to␣
 ↪'{output_path}'. Error: {e}")

# # --- Execution ---
# SLIDE_TEMPLATE_PATH = "/home/sebas_dev_linux/projects/course_generator/data/
 ↪slide_style/slide_style_test_2.pptx"
# LAYOUT_MAPPING_PATH = "/home/sebas_dev_linux/projects/course_generator/
 ↪configs/layout_mapping_test.json"
# # You run this function to generate the config file.
# # Make sure SLIDE_TEMPLATE_PATH and LAYOUT_MAPPING_PATH are defined.
# inspect_and_generate_layout_config(SLIDE_TEMPLATE_PATH, LAYOUT_MAPPING_PATH)
```

4 Extrac Unit outline details to process following steps - output raw json with UO details

[26]:
```
# Cell 3: Parse Unit Outline


# --- Helper Functions for Parsing ---
def extract_text_from_file(filepath: str) -> str:
    _, ext = os.path.splitext(filepath.lower())
    if ext == '.docx':
        doc = Document(filepath)
        full_text = [p.text for p in doc.paragraphs]
        for table in doc.tables:
            for row in table.rows:
                full_text.append(" | ".join(cell.text for cell in row.cells))
        return '\n'.join(full_text)
    elif ext == '.pdf':
        with pdfplumber.open(filepath) as pdf:
            return "\n".join(page.extract_text() for page in pdf.pages if page.
 ↪extract_text())
    else:
        raise TypeError(f"Unsupported file type: {ext}")
```

```python
def parse_llm_json_output(content: str) -> dict:
    try:
        match = re.search(r'\{.*\}', content, re.DOTALL)
        if not match: return None
        return json.loads(match.group(0))
    except (json.JSONDecodeError, TypeError):
        return None

@retry(stop=stop_after_attempt(3), wait=wait_exponential(min=2, max=10))
def call_ollama_with_retry(client, prompt):
    logger.info(f"Calling Ollama model '{OLLAMA_MODEL}'...")
    response = client.chat(
        model=OLLAMA_MODEL,
        messages=[{"role": "user", "content": prompt}],
        format="json",
        options={"temperature": 0.0}
    )
    if not response or 'message' not in response or not response['message'].
 ↪get('content'):
        raise ValueError("Ollama returned an empty or invalid response.")
    return response['message']['content']

# --- Main Orchestration Function for this Cell ---
def parse_and_save_outline_robust(
    input_filepath: str,
    output_filepath: str,
    prompt_template: str,
    max_retries: int = 3
):
    logger.info(f"Starting to robustly process Unit Outline: {input_filepath}")

    if not os.path.exists(input_filepath):
        logger.error(f"Input file not found: {input_filepath}")
        return

    try:
        outline_text = extract_text_from_file(input_filepath)
        if not outline_text.strip():
            logger.error("Extracted text is empty. Aborting.")
            return
    except Exception as e:
        logger.error(f"Failed to extract text from file: {e}", exc_info=True)
        return

    client = ollama.Client(host=OLLAMA_HOST)
    current_prompt = prompt_template.format(outline_text=outline_text)
```

```
    for attempt in range(max_retries):
        logger.info(f"Attempt {attempt + 1}/{max_retries} to parse outline.")

        try:
            # Call the LLM
            llm_output_str = call_ollama_with_retry(client, current_prompt)

            # Find the JSON blob in the response
            json_blob = parse_llm_json_output(llm_output_str) # Your existing␣
↪helper
            if not json_blob:
                raise ValueError("LLM did not return a parsable JSON object.")

            # *** THE KEY VALIDATION STEP ***
            # Try to parse the dictionary into your Pydantic model.
            # This will raise a `ValidationError` if keys are wrong, types are␣
↪wrong, or fields are missing.
            parsed_data = ParsedUnitOutline.model_validate(json_blob)

            # If successful, save the validated data and exit the loop
            logger.info("Successfully validated JSON structure against Pydantic␣
↪model.")
            os.makedirs(os.path.dirname(output_filepath), exist_ok=True)
            with open(output_filepath, 'w', encoding='utf-8') as f:
                # Use .model_dump_json() for clean, validated output
                f.write(parsed_data.model_dump_json(indent=2))

            logger.info(f"Successfully parsed and saved Unit Outline to:␣
↪{output_filepath}")
            return # Exit function on success

        except ValidationError as e:
            logger.warning(f"Validation failed on attempt {attempt + 1}. Error:␣
↪{e}")
            # Formulate a new prompt with the error message for self-correction
            error_feedback = (
                f"\n\nYour previous attempt failed. You MUST correct the␣
↪following errors:\n"
                f"{e}\n\n"
                f"Please regenerate the entire JSON object, ensuring it␣
↪strictly adheres to the schema "
                f"and corrects these specific errors. Do not change any key␣
↪names."
            )
            current_prompt = current_prompt + error_feedback # Append the error␣
↪to the prompt
```

```
        except Exception as e:
            # Catch other errors like network issues from call_ollama_with_retry
            logger.error(f"An unexpected error occurred on attempt {attempt +␣
 ↪1}: {e}", exc_info=True)
            # You might want to wait before retrying for non-validation errors
            time.sleep(5)

    logger.error(f"Failed to get valid structured data from the LLM after␣
 ↪{max_retries} attempts.")


# --- In your execution block, call the new function ---
# parse_and_save_outline(...) becomes:

if EXTRACT_UO:
    parse_and_save_outline_robust(
        input_filepath=FULL_PATH_UNIT_OUTLINE,
        output_filepath=PARSED_UO_JSON_PATH,
        prompt_template=UNIT_OUTLINE_SYSTEM_PROMPT_TEMPLATE
    )
```

```
2025-07-18 21:38:40,437 - INFO - Starting to robustly process Unit Outline:
/home/sebas_dev_linux/projects/course_generator/data/UO/ICT312 Digital
Forensic_Final.docx
2025-07-18 21:38:40,477 - INFO - Attempt 1/3 to parse outline.
2025-07-18 21:38:40,478 - INFO - Calling Ollama model 'qwen3:8b'…
2025-07-18 21:39:49,873 - INFO - HTTP Request: POST
http://localhost:11434/api/chat "HTTP/1.1 200 OK"
2025-07-18 21:39:49,874 - INFO - Successfully validated JSON structure against
Pydantic model.
2025-07-18 21:39:49,880 - INFO - Successfully parsed and saved Unit Outline to:
/home/sebas_dev_linux/projects/course_generator/data_output/1.
Parse_data/Parse_UO/ICT312/ICT312 Digital Forensic_Final_parsed.json
```

5 Extract TOC from epub or PDF

[27]:
```
# Cell 4: Extract Book Table of Contents (ToC) with Pre-assigned IDs & Links in␣
 ↪Order

from ebooklib import epub, ITEM_NAVIGATION
from bs4 import BeautifulSoup
import fitz  # PyMuPDF
import json
import os
from typing import List, Dict
```

```python
import urllib.parse # Needed to clean up links

# ============================================================================
# 1. HELPER FUNCTIONS (MODIFIED TO INCLUDE ID ASSIGNMENT AND LINK EXTRACTION)
# ============================================================================

def clean_epub_href(href: str) -> str:
    """Removes URL fragments and decodes URL-encoded characters."""
    if not href: return ""
    # Remove fragment identifier (e.g., '#section1')
    cleaned_href = href.split('#')[0]
    # Decode any URL-encoded characters (e.g., %20 -> space)
    return urllib.parse.unquote(cleaned_href)

# --- EPUB Extraction Logic ---
def parse_navpoint(navpoint: BeautifulSoup, counter: List[int], level: int = 0)␣
 ↪-> Dict:
    """Recursively parses EPUB 2 navPoints and assigns a toc_id and␣
 ↪link_filename."""
    title = navpoint.navLabel.text.strip()
    if not title: return None

    # --- MODIFICATION: Extract the linked filename ---
    content_tag = navpoint.find('content', recursive=False)
    link_filename = clean_epub_href(content_tag['src']) if content_tag else ""

    node = {
        "level": level,
        "toc_id": counter[0],
        "title": title,
        "link_filename": link_filename, # Add the cleaned link
        "children": []
    }
    counter[0] += 1

    for child_navpoint in navpoint.find_all('navPoint', recursive=False):
        child_node = parse_navpoint(child_navpoint, counter, level + 1)
        if child_node: node["children"].append(child_node)

    return node

def parse_li(li_element: BeautifulSoup, counter: List[int], level: int = 0) ->␣
 ↪Dict:
    """Recursively parses EPUB 3 <li> elements and assigns a toc_id and␣
 ↪link_filename."""
    a_tag = li_element.find('a', recursive=False)
    if a_tag:
```

```
        title = a_tag.get_text(strip=True)
        if not title: return None

        # --- MODIFICATION: Extract the linked filename ---
        link_filename = clean_epub_href(a_tag.get('href'))

        node = {
            "level": level,
            "toc_id": counter[0],
            "title": title,
            "link_filename": link_filename, # Add the cleaned link
            "children": []
        }
        counter[0] += 1

        nested_ol = li_element.find('ol', recursive=False)
        if nested_ol:
            for sub_li in nested_ol.find_all('li', recursive=False):
                child_node = parse_li(sub_li, counter, level + 1)
                if child_node: node["children"].append(child_node)
        return node
    return None

def extract_epub_toc(epub_path, output_json_path):
    print(f"Processing EPUB ToC for: {epub_path}")
    toc_data = []
    book = epub.read_epub(epub_path)
    id_counter = [0]

    for nav_item in book.get_items_of_type(ITEM_NAVIGATION):
        soup = BeautifulSoup(nav_item.get_content(), 'xml')
        # Logic to handle both EPUB 2 (NCX) and EPUB 3 (XHTML)
        if nav_item.get_name().endswith('.ncx'):
            print("INFO: Found EPUB 2 (NCX) Table of Contents. Parsing...")
            navmap = soup.find('navMap')
            if navmap:
                for navpoint in navmap.find_all('navPoint', recursive=False):
                    node = parse_navpoint(navpoint, id_counter, level=0)
                    if node: toc_data.append(node)
        else: # Assumes EPUB 3
            print("INFO: Found EPUB 3 (XHTML) Table of Contents. Parsing...")
            toc_nav = soup.select_one('nav[epub|type="toc"]')
            if toc_nav:
                top_ol = toc_nav.find('ol', recursive=False)
                if top_ol:
                    for li in top_ol.find_all('li', recursive=False):
                        node = parse_li(li, id_counter, level=0)
```

```
                        if node: toc_data.append(node)
        if toc_data: break

    if toc_data:
        os.makedirs(os.path.dirname(output_json_path), exist_ok=True)
        with open(output_json_path, 'w', encoding='utf-8') as f:
            json.dump(toc_data, f, indent=2, ensure_ascii=False)
        print(f"  Successfully wrote EPUB ToC with IDs and links to:␣
↪{output_json_path}")
    else:
        print("  WARNING: No ToC data extracted from EPUB.")

# --- PDF Extraction Logic (Unchanged) ---
def build_pdf_hierarchy_with_ids(toc_list: List) -> List[Dict]:
    root = []
    parent_stack = {-1: {"children": root}}
    id_counter = [0]
    for level, title, page in toc_list:
        normalized_level = level - 1
        node = {"level": normalized_level, "toc_id": id_counter[0], "title":␣
↪title.strip(), "page": page, "children": []}
        id_counter[0] += 1
        parent_node = parent_stack.get(normalized_level - 1)
        if parent_node: parent_node["children"].append(node)
        parent_stack[normalized_level] = node
    return root

def extract_pdf_toc(pdf_path, output_json_path):
    print(f"Processing PDF ToC for: {pdf_path}")
    try:
        doc = fitz.open(pdf_path)
        toc = doc.get_toc()
        hierarchical_toc = []
        if not toc: print("  WARNING: This PDF has no embedded bookmarks (ToC).
↪")
        else:
            print(f"INFO: Found {len(toc)} bookmark entries. Building hierarchy␣
↪and assigning IDs...")
            hierarchical_toc = build_pdf_hierarchy_with_ids(toc)
        os.makedirs(os.path.dirname(output_json_path), exist_ok=True)
        with open(output_json_path, 'w', encoding='utf-8') as f:
            json.dump(hierarchical_toc, f, indent=2, ensure_ascii=False)
        print(f"  Successfully wrote PDF ToC with assigned IDs to:␣
↪{output_json_path}")
    except Exception as e: print(f"An error occurred during PDF ToC extraction:␣
↪{e}")
```

```
# ============================================================================
# 2. EXECUTION BLOCK
# ============================================================================
if PROCESS_EPUB:
    extract_epub_toc(BOOK_PATH, PRE_EXTRACTED_TOC_JSON_PATH)
else:
    extract_pdf_toc(BOOK_PATH, PRE_EXTRACTED_TOC_JSON_PATH)
```

```
Processing EPUB ToC for:
/home/sebas_dev_linux/projects/course_generator/data/books/Bill Nelson, Amelia
Phillips, Christopher Steuart - Guide to Computer Forensics and Investigations_
Processing Digital Evidence-Cengage Learning (2018).epub
INFO: Found EPUB 2 (NCX) Table of Contents. Parsing…
 Successfully wrote EPUB ToC with IDs and links to:
/home/sebas_dev_linux/projects/course_generator/data_output/1.
Parse_data/Parse_TOC_books/ICT312/ICT312_epub_table_of_contents.json
```

6 Hirachical DB base on TOC

6.1 Process Book

[28]:
```
# Cell 5: Create Hierarchical Vector Database (with Sequential ToC ID and Chunk␣
 ↪ID)
# This cell processes the book, enriches it with hierarchical and sequential␣
 ↪metadata,
# chunks it, and creates the final vector database.

import os
import json
import shutil
import logging
from typing import List, Dict, Any, Tuple
from langchain_core.documents import Document
from langchain_community.document_loaders import PyPDFLoader,␣
 ↪UnstructuredEPubLoader
from langchain_ollama.embeddings import OllamaEmbeddings
from langchain_chroma import Chroma
from langchain.text_splitter import RecursiveCharacterTextSplitter

# Setup Logger for this cell
logging.basicConfig(level=logging.INFO, format='%(asctime)s - %(levelname)s -␣
 ↪%(message)s')
logger = logging.getLogger(__name__)

# --- Helper: Clean metadata values for ChromaDB ---
def clean_metadata_for_chroma(value: Any) -> Any:
```

```
    """Sanitizes metadata values to be compatible with ChromaDB."""
    if isinstance(value, list): return ", ".join(map(str, value))
    if isinstance(value, dict): return json.dumps(value)
    if isinstance(value, (str, int, float, bool)) or value is None: return value
    return str(value)

# --- Core Function to Process Book with Pre-extracted ToC ---
def process_book_with_extracted_toc(
    book_path: str,
    extracted_toc_json_path: str,
    chunk_size: int,
    chunk_overlap: int
) -> Tuple[List[Document], List[Dict[str, Any]]]:

    logger.info(f"Processing book '{os.path.basename(book_path)}' using ToC␣
 ↪from '{os.path.basename(extracted_toc_json_path)}'.")

    # 1. Load the pre-extracted hierarchical ToC
    try:
        with open(extracted_toc_json_path, 'r', encoding='utf-8') as f:
            hierarchical_toc = json.load(f)
        if not hierarchical_toc:
            logger.error(f"Pre-extracted ToC at '{extracted_toc_json_path}' is␣
 ↪empty or invalid.")
            return [], []
        logger.info(f"Successfully loaded pre-extracted ToC with␣
 ↪{len(hierarchical_toc)} top-level entries.")
    except Exception as e:
        logger.error(f"Error loading pre-extracted ToC JSON: {e}",␣
 ↪exc_info=True)
        return [], []

    # 2. Load all text elements/pages from the book
    all_raw_book_docs: List[Document] = []
    _, file_extension = os.path.splitext(book_path.lower())

    if file_extension == ".epub":
        loader = UnstructuredEPubLoader(book_path, mode="elements",␣
 ↪strategy="fast")
        try:
            all_raw_book_docs = loader.load()
            logger.info(f"Loaded {len(all_raw_book_docs)} text elements from␣
 ↪EPUB.")
        except Exception as e:
            logger.error(f"Error loading EPUB content: {e}", exc_info=True)
            return [], hierarchical_toc
```

```
    elif file_extension == ".pdf":
        loader = PyPDFLoader(book_path)
        try:
            all_raw_book_docs = loader.load()
            logger.info(f"Loaded {len(all_raw_book_docs)} pages from PDF.")
        except Exception as e:
            logger.error(f"Error loading PDF content: {e}", exc_info=True)
            return [], hierarchical_toc
    else:
        logger.error(f"Unsupported book file format: {file_extension}")
        return [], hierarchical_toc

    if not all_raw_book_docs:
        logger.error("No text elements/pages loaded from the book.")
        return [], hierarchical_toc

    # 3. Create enriched LangChain Documents by matching ToC to content
    final_documents_with_metadata: List[Document] = []

    # Flatten the ToC, AND add a unique sequential ID for sorting and␣
↪validation.
    flat_toc_entries: List[Dict[str, Any]] = []

    def _add_ids_and_flatten_recursive(nodes: List[Dict[str, Any]],␣
↪current_titles_path: List[str], counter: List[int]):
        """
        Recursively traverses ToC nodes to flatten them and assign a unique,␣
↪sequential toc_id.
        """
        for node in nodes:
            toc_id = counter[0]
            counter[0] += 1
            title = node.get("title", "").strip()
            if not title: continue
            new_titles_path = current_titles_path + [title]
            entry = {
                "titles_path": new_titles_path,
                "level": node.get("level"),
                "full_title_for_matching": title,
                "toc_id": toc_id
            }
            if "page" in node: entry["page"] = node["page"]
            flat_toc_entries.append(entry)
            if node.get("children"):
                _add_ids_and_flatten_recursive(node.get("children", []),␣
↪new_titles_path, counter)
```

```
    toc_id_counter = [0]
    _add_ids_and_flatten_recursive(hierarchical_toc, [], toc_id_counter)
    logger.info(f"Flattened ToC and assigned sequential IDs to␣
↪{len(flat_toc_entries)} entries.")

    # Logic for PDF metadata assignment
    if file_extension == ".pdf" and any("page" in entry for entry in␣
↪flat_toc_entries):
        logger.info("Assigning metadata to PDF pages based on ToC page numbers..
↪.")
        flat_toc_entries.sort(key=lambda x: x.get("page", -1) if x.get("page")␣
↪is not None else -1)
        for page_doc in all_raw_book_docs:
            page_num_0_indexed = page_doc.metadata.get("page", -1)
            page_num_1_indexed = page_num_0_indexed + 1
            assigned_metadata = {"source": os.path.basename(book_path),␣
↪"page_number": page_num_1_indexed}
            best_match_toc_entry = None
            for toc_entry in flat_toc_entries:
                toc_page = toc_entry.get("page")
                if toc_page is not None and toc_page <= page_num_1_indexed:
                    if best_match_toc_entry is None or toc_page >␣
↪best_match_toc_entry.get("page", -1):
                        best_match_toc_entry = toc_entry
                elif toc_page is not None and toc_page > page_num_1_indexed:
                    break
            if best_match_toc_entry:
                for i, title_in_path in␣
↪enumerate(best_match_toc_entry["titles_path"]):
                    assigned_metadata[f"level_{i+1}_title"] = title_in_path
                assigned_metadata['toc_id'] = best_match_toc_entry.get('toc_id')
            else:
                assigned_metadata["level_1_title"] = "Uncategorized PDF Page"
            cleaned_meta = {k: clean_metadata_for_chroma(v) for k, v in␣
↪assigned_metadata.items()}
            final_documents_with_metadata.append(Document(page_content=page_doc.
↪page_content, metadata=cleaned_meta))

    # Logic for EPUB metadata assignment
    elif file_extension == ".epub":
        logger.info("Assigning metadata to EPUB elements by matching ToC titles␣
↪in text...")
        toc_titles_for_search = [entry for entry in flat_toc_entries if entry.
↪get("full_title_for_matching")]
        current_hierarchy_metadata = {}
        for element_doc in all_raw_book_docs:
```

```
            element_text = element_doc.page_content.strip() if element_doc.
↪page_content else ""
            if not element_text: continue
            for toc_entry in toc_titles_for_search:
                if element_text == toc_entry["full_title_for_matching"]:
                    current_hierarchy_metadata = {"source": os.path.
↪basename(book_path)}
                    for i, title_in_path in enumerate(toc_entry["titles_path"]):
                        current_hierarchy_metadata[f"level_{i+1}_title"] =␣
↪title_in_path
                    current_hierarchy_metadata['toc_id'] = toc_entry.
↪get('toc_id')
                    if "page" in toc_entry:␣
↪current_hierarchy_metadata["epub_toc_page"] = toc_entry["page"]
                    break
            if not current_hierarchy_metadata:
                doc_metadata_to_assign = {"source": os.path.
↪basename(book_path), "level_1_title": "EPUB Preamble", "toc_id": -1}
            else:
                doc_metadata_to_assign = current_hierarchy_metadata.copy()
            cleaned_meta = {k: clean_metadata_for_chroma(v) for k, v in␣
↪doc_metadata_to_assign.items()}
            final_documents_with_metadata.
↪append(Document(page_content=element_text, metadata=cleaned_meta))

    else: # Fallback
        final_documents_with_metadata = all_raw_book_docs

    if not final_documents_with_metadata:
        logger.error("No documents were processed or enriched with hierarchical␣
↪metadata.")
        return [], hierarchical_toc

    logger.info(f"Total documents prepared for chunking:␣
↪{len(final_documents_with_metadata)}")

    text_splitter = RecursiveCharacterTextSplitter(
        chunk_size=chunk_size,
        chunk_overlap=chunk_overlap,
        length_function=len
    )
    final_chunks = text_splitter.split_documents(final_documents_with_metadata)
    logger.info(f"Split into {len(final_chunks)} final chunks, inheriting␣
↪hierarchical metadata.")
```

```
    # --- MODIFICATION START: Add a unique, sequential chunk_id to each chunk␣
 ↪---
    logger.info("Assigning sequential chunk_id to all final chunks...")
    for i, chunk in enumerate(final_chunks):
        chunk.metadata['chunk_id'] = i
    logger.info(f"Assigned chunk_ids from 0 to {len(final_chunks) - 1}.")
    # --- MODIFICATION END ---

    return final_chunks, hierarchical_toc

# --- Main Execution Block for this Cell ---
if CREATE_RAG_BOOK:
    if not os.path.exists(PRE_EXTRACTED_TOC_JSON_PATH):
        logger.error(f"CRITICAL: Pre-extracted ToC file not found at␣
 ↪'{PRE_EXTRACTED_TOC_JSON_PATH}'.")
        logger.error("Please run the 'Extract Book Table of Contents (ToC)'␣
 ↪cell (Cell 4) first.")
    else:
        final_chunks_for_db, toc_reloaded = process_book_with_extracted_toc(
            book_path=BOOK_PATH,
            extracted_toc_json_path=PRE_EXTRACTED_TOC_JSON_PATH,
            chunk_size=CHUNK_SIZE,
            chunk_overlap=CHUNK_OVERLAP
        )

        if final_chunks_for_db:
            if os.path.exists(CHROMA_PERSIST_DIR):
                logger.warning(f"Deleting existing ChromaDB directory:␣
 ↪{CHROMA_PERSIST_DIR}")
                shutil.rmtree(CHROMA_PERSIST_DIR)

            logger.info(f"Initializing embedding model␣
 ↪'{EMBEDDING_MODEL_OLLAMA}' and creating new vector database...")
            embedding_model = OllamaEmbeddings(model=EMBEDDING_MODEL_OLLAMA)

            vector_db = Chroma.from_documents(
                documents=final_chunks_for_db,
                embedding=embedding_model,
                persist_directory=CHROMA_PERSIST_DIR,
                collection_name=CHROMA_COLLECTION_NAME
            )

            reloaded_db = Chroma(persist_directory=CHROMA_PERSIST_DIR,␣
 ↪embedding_function=embedding_model, collection_name=CHROMA_COLLECTION_NAME)
            count = reloaded_db._collection.count()
```

```
            print("-" * 50)
            logger.info(f" Vector DB created successfully at:␣
↪{CHROMA_PERSIST_DIR}")
            logger.info(f" Collection '{CHROMA_COLLECTION_NAME}' contains␣
↪{count} documents.")
            print("-" * 50)
        else:
            logger.error(" Failed to generate chunks. Vector DB not created.")
```

```
2025-07-18 21:44:47,988 - INFO - Processing book 'Bill Nelson, Amelia Phillips,
Christopher Steuart - Guide to Computer Forensics and Investigations_ Processing
Digital Evidence-Cengage Learning (2018).epub' using ToC from
'ICT312_epub_table_of_contents.json'.
2025-07-18 21:44:47,989 - INFO - Successfully loaded pre-extracted ToC with 28
top-level entries.
2025-07-18 21:44:50,324 - INFO - Note: NumExpr detected 32 cores but
"NUMEXPR_MAX_THREADS" not set, so enforcing safe limit of 16.
2025-07-18 21:44:50,325 - INFO - NumExpr defaulting to 16 threads.
[WARNING] Could not load translations for en-US
  data file translations/en.yaml not found
2025-07-18 21:44:55,688 - WARNING - Could not load translations for en-US
  data file translations/en.yaml not found
[WARNING] The term Abstract has no translation defined.

2025-07-18 21:44:55,689 - WARNING - The term Abstract has no translation
defined.

2025-07-18 21:44:59,091 - INFO - Loaded 11815 text elements from EPUB.
2025-07-18 21:44:59,092 - INFO - Flattened ToC and assigned sequential IDs to
877 entries.
2025-07-18 21:44:59,092 - INFO - Assigning metadata to EPUB elements by matching
ToC titles in text…
2025-07-18 21:44:59,459 - INFO - Total documents prepared for chunking: 11483
2025-07-18 21:44:59,649 - INFO - Split into 11774 final chunks, inheriting
hierarchical metadata.
2025-07-18 21:44:59,649 - INFO - Assigning sequential chunk_id to all final
chunks…
2025-07-18 21:44:59,652 - INFO - Assigned chunk_ids from 0 to 11773.
2025-07-18 21:44:59,658 - INFO - Initializing embedding model 'nomic-embed-text'
and creating new vector database…
2025-07-18 21:44:59,722 - INFO - Anonymized telemetry enabled. See
https://docs.trychroma.com/telemetry for more information.
2025-07-18 21:46:17,066 - INFO - HTTP Request: POST
http://127.0.0.1:11434/api/embed "HTTP/1.1 200 OK"
2025-07-18 21:47:36,388 - INFO - HTTP Request: POST
http://127.0.0.1:11434/api/embed "HTTP/1.1 200 OK"
2025-07-18 21:47:52,347 - INFO - HTTP Request: POST
```

```
http://127.0.0.1:11434/api/embed "HTTP/1.1 200 OK"
2025-07-18 21:47:52,880 - INFO -   Vector DB created successfully at:
/home/sebas_dev_linux/projects/course_generator/data_output/0.
DataBase_Chroma/ICT312/ICT312_chroma_db_toc_chunks_epub
2025-07-18 21:47:52,880 - INFO -   Collection 'book_toc_chunks_epub' contains
11774 documents.

--------------------------------------------------
--------------------------------------------------
```

[29]:
```python
# Cell 5a: Inspecting EPUB Documents and Metadata BEFORE Chunking

import json
import os
import logging
from langchain_community.document_loaders import UnstructuredEPubLoader
from langchain_core.documents import Document

# --- Setup Logger for this inspection cell ---
logger = logging.getLogger(__name__)
logging.basicConfig(level=logging.INFO, format='%(asctime)s - %(levelname)s -␣
 ↪%(message)s')

def inspect_epub_preprocessing():
    """
    This function replicates the pre-chunking logic from Cell 5 for EPUB files
    to show the list of large documents with their assigned ToC metadata.
    """
    if not PROCESS_EPUB:
        print("This inspection cell is for EPUB processing. Please set␣
 ↪PROCESS_EPUB = True in Cell 1.")
        return

    print_header("EPUB Pre-Processing Inspection", char="~")

    # --- 1. Load the necessary data (replicating start of Cell 5) ---
    logger.info("Loading pre-extracted ToC and raw EPUB elements...")
    try:
        with open(PRE_EXTRACTED_TOC_JSON_PATH, 'r', encoding='utf-8') as f:
            hierarchical_toc = json.load(f)

        loader = UnstructuredEPubLoader(BOOK_PATH, mode="elements",␣
 ↪strategy="fast")
        all_raw_book_docs = loader.load()
        logger.info(f"Successfully loaded {len(all_raw_book_docs)} raw text␣
 ↪elements from the EPUB.")
    except Exception as e:
        logger.error(f"Failed to load necessary files: {e}")
```

```
        return

    # --- 2. Flatten the ToC (replicating logic from Cell 5) ---
    logger.info("Flattening the hierarchical ToC for matching...")
    flat_toc_entries = []
    def _add_ids_and_flatten_recursive(nodes, current_titles_path, counter):
        for node in nodes:
            toc_id = counter[0]
            counter[0] += 1
            title = node.get("title", "").strip()
            if not title: continue
            new_titles_path = current_titles_path + [title]
            entry = {
                "titles_path": new_titles_path,
                "level": node.get("level"),
                "full_title_for_matching": title,
                "toc_id": toc_id
            }
            flat_toc_entries.append(entry)
            if node.get("children"):
                _add_ids_and_flatten_recursive(node.get("children", []),␣
↪new_titles_path, counter)

    _add_ids_and_flatten_recursive(hierarchical_toc, [], [0])
    logger.info(f"Flattened ToC into {len(flat_toc_entries)} entries.")

    # --- 3. The Core Matching Logic for EPUB (the part you want to see) ---
    logger.info("Assigning metadata to EPUB elements by matching ToC titles...")

    final_documents_with_metadata = []
    toc_titles_for_search = [entry for entry in flat_toc_entries if entry.
↪get("full_title_for_matching")]
    current_hierarchy_metadata = {}

    for element_doc in all_raw_book_docs:
        element_text = element_doc.page_content.strip() if element_doc.
↪page_content else ""
        if not element_text:
            continue

        # Check if this element is a heading that matches a ToC entry
        is_heading = False
        for toc_entry in toc_titles_for_search:
            if element_text == toc_entry["full_title_for_matching"]:
                # It's a heading! Update the current context.
                current_hierarchy_metadata = {"source": os.path.
↪basename(BOOK_PATH)}
```

```
                for i, title_in_path in enumerate(toc_entry["titles_path"]):
                    current_hierarchy_metadata[f"level_{i+1}_title"] =␣
↪title_in_path
                current_hierarchy_metadata['toc_id'] = toc_entry.get('toc_id')
                is_heading = True
                break # Found the match, no need to search further

        # Assign metadata
        if not current_hierarchy_metadata:
            # Content before the first ToC entry (e.g., cover, title page)
            doc_metadata_to_assign = {"source": os.path.basename(BOOK_PATH),␣
↪"level_1_title": "EPUB Preamble", "toc_id": -1}
        else:
            doc_metadata_to_assign = current_hierarchy_metadata.copy()

        final_documents_with_metadata.
↪append(Document(page_content=element_text, metadata=doc_metadata_to_assign))

    logger.info(f"Processing complete. Generated␣
↪{len(final_documents_with_metadata)} documents with assigned metadata.")

    # --- 4. Print the result for inspection ---
    print_header("INSPECTION RESULTS: Documents Before Chunking", char="=")
    print(f"Total documents created: {len(final_documents_with_metadata)}\n")

    for i, doc in enumerate(final_documents_with_metadata[:100]): # Print first␣
↪30 to avoid flooding the output
        print(f"--- Document [{i+1}] ---")
        print(f"  Assigned Metadata: {doc.metadata}")
        print(f"  Content (Un-chunked Element):")
        print(f"  >> '{doc.page_content}'")
        print("-" * 25 + "\n")


# --- Execute the inspection ---
inspect_epub_preprocessing()
```

```
2025-07-18 21:47:52,897 - INFO - Loading pre-extracted ToC and raw EPUB
elements…


~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
                          EPUB Pre-Processing Inspection
~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~

[WARNING] Could not load translations for en-US
  data file translations/en.yaml not found
2025-07-18 21:47:57,926 - WARNING - Could not load translations for en-US
```

```
  data file translations/en.yaml not found
[WARNING] The term Abstract has no translation defined.

2025-07-18 21:47:57,927 - WARNING - The term Abstract has no translation
defined.

2025-07-18 21:48:01,551 - INFO - Successfully loaded 11815 raw text elements
from the EPUB.
2025-07-18 21:48:01,552 - INFO - Flattening the hierarchical ToC for matching…
2025-07-18 21:48:01,553 - INFO - Flattened ToC into 877 entries.
2025-07-18 21:48:01,553 - INFO - Assigning metadata to EPUB elements by matching
ToC titles…
2025-07-18 21:48:01,768 - INFO - Processing complete. Generated 11483 documents
with assigned metadata.


================================================================================
                  INSPECTION RESULTS: Documents Before Chunking
================================================================================
Total documents created: 11483

--- Document [1] ---
  Assigned Metadata: {'source': 'Bill Nelson, Amelia Phillips, Christopher
Steuart - Guide to Computer Forensics and Investigations_ Processing Digital
Evidence-Cengage Learning (2018).epub', 'level_1_title': 'EPUB Preamble',
'toc_id': -1}
  Content (Un-chunked Element):
  >> 'Guide to Computer Forensics and Investigations: Processing Digital
Evidence'
--------------------------

--- Document [2] ---
  Assigned Metadata: {'source': 'Bill Nelson, Amelia Phillips, Christopher
Steuart - Guide to Computer Forensics and Investigations_ Processing Digital
Evidence-Cengage Learning (2018).epub', 'level_1_title': 'EPUB Preamble',
'toc_id': -1}
  Content (Un-chunked Element):
  >> 'Copyright Statement'
--------------------------

--- Document [3] ---
  Assigned Metadata: {'source': 'Bill Nelson, Amelia Phillips, Christopher
Steuart - Guide to Computer Forensics and Investigations_ Processing Digital
Evidence-Cengage Learning (2018).epub', 'level_1_title': 'EPUB Preamble',
'toc_id': -1}
  Content (Un-chunked Element):
  >> 'Guide to Computer Forensics and Investigations: Processing Digital
Evidence'
```

```
-------------------------

--- Document [4] ---
  Assigned Metadata: {'source': 'Bill Nelson, Amelia Phillips, Christopher
Steuart - Guide to Computer Forensics and Investigations_ Processing Digital
Evidence-Cengage Learning (2018).epub', 'level_1_title': 'EPUB Preamble',
'toc_id': -1}
  Content (Un-chunked Element):
  >> 'COPYRIGHT © 2019, 2016 Cengage Learning, Inc.'
-------------------------

--- Document [5] ---
  Assigned Metadata: {'source': 'Bill Nelson, Amelia Phillips, Christopher
Steuart - Guide to Computer Forensics and Investigations_ Processing Digital
Evidence-Cengage Learning (2018).epub', 'level_1_title': 'EPUB Preamble',
'toc_id': -1}
  Content (Un-chunked Element):
  >> 'ALL RIGHTS RESERVED. No part of this work covered by the copyright herein
may be reproduced or distributed in any form or by any means, except as
permitted by U.S. copyright law, without the prior written permission of the
copyright owner.'
-------------------------

--- Document [6] ---
  Assigned Metadata: {'source': 'Bill Nelson, Amelia Phillips, Christopher
Steuart - Guide to Computer Forensics and Investigations_ Processing Digital
Evidence-Cengage Learning (2018).epub', 'level_1_title': 'EPUB Preamble',
'toc_id': -1}
  Content (Un-chunked Element):
  >> 'For product information and technology assistance, contact us at Cengage
Customer & Sales Support, 1-800-354-9706 or support.cengage.com.'
-------------------------

--- Document [7] ---
  Assigned Metadata: {'source': 'Bill Nelson, Amelia Phillips, Christopher
Steuart - Guide to Computer Forensics and Investigations_ Processing Digital
Evidence-Cengage Learning (2018).epub', 'level_1_title': 'EPUB Preamble',
'toc_id': -1}
  Content (Un-chunked Element):
  >> 'For permission to use material from this text or product, submit all
requests online at www.cengage.com/permissions.'
-------------------------

--- Document [8] ---
  Assigned Metadata: {'source': 'Bill Nelson, Amelia Phillips, Christopher
Steuart - Guide to Computer Forensics and Investigations_ Processing Digital
Evidence-Cengage Learning (2018).epub', 'level_1_title': 'EPUB Preamble',
'toc_id': -1}
```

```
  Content (Un-chunked Element):
  >> 'SOURCE FOR ILLUSTRATIONS: Copyright © Cengage.'
-------------------------

--- Document [9] ---
  Assigned Metadata: {'source': 'Bill Nelson, Amelia Phillips, Christopher
Steuart - Guide to Computer Forensics and Investigations_ Processing Digital
Evidence-Cengage Learning (2018).epub', 'level_1_title': 'EPUB Preamble',
'toc_id': -1}
  Content (Un-chunked Element):
  >> ', Microsoft® is a registered trademark of the Microsoft Corporation.'
-------------------------

--- Document [10] ---
  Assigned Metadata: {'source': 'Bill Nelson, Amelia Phillips, Christopher
Steuart - Guide to Computer Forensics and Investigations_ Processing Digital
Evidence-Cengage Learning (2018).epub', 'level_1_title': 'EPUB Preamble',
'toc_id': -1}
  Content (Un-chunked Element):
  >> 'Library of Congress Control Number: 2018936389'
-------------------------

--- Document [11] ---
  Assigned Metadata: {'source': 'Bill Nelson, Amelia Phillips, Christopher
Steuart - Guide to Computer Forensics and Investigations_ Processing Digital
Evidence-Cengage Learning (2018).epub', 'level_1_title': 'EPUB Preamble',
'toc_id': -1}
  Content (Un-chunked Element):
  >> 'ISBN: 978-1-337-56894-4'
-------------------------

--- Document [12] ---
  Assigned Metadata: {'source': 'Bill Nelson, Amelia Phillips, Christopher
Steuart - Guide to Computer Forensics and Investigations_ Processing Digital
Evidence-Cengage Learning (2018).epub', 'level_1_title': 'EPUB Preamble',
'toc_id': -1}
  Content (Un-chunked Element):
  >> 'Cengage'
-------------------------

--- Document [13] ---
  Assigned Metadata: {'source': 'Bill Nelson, Amelia Phillips, Christopher
Steuart - Guide to Computer Forensics and Investigations_ Processing Digital
Evidence-Cengage Learning (2018).epub', 'level_1_title': 'EPUB Preamble',
'toc_id': -1}
  Content (Un-chunked Element):
  >> '20 Channel Center Street'
-------------------------
```

```
--- Document [14] ---
  Assigned Metadata: {'source': 'Bill Nelson, Amelia Phillips, Christopher
Steuart - Guide to Computer Forensics and Investigations_ Processing Digital
Evidence-Cengage Learning (2018).epub', 'level_1_title': 'EPUB Preamble',
'toc_id': -1}
  Content (Un-chunked Element):
  >> 'Boston MA 02210'
-------------------------

--- Document [15] ---
  Assigned Metadata: {'source': 'Bill Nelson, Amelia Phillips, Christopher
Steuart - Guide to Computer Forensics and Investigations_ Processing Digital
Evidence-Cengage Learning (2018).epub', 'level_1_title': 'EPUB Preamble',
'toc_id': -1}
  Content (Un-chunked Element):
  >> 'USA'
-------------------------

--- Document [16] ---
  Assigned Metadata: {'source': 'Bill Nelson, Amelia Phillips, Christopher
Steuart - Guide to Computer Forensics and Investigations_ Processing Digital
Evidence-Cengage Learning (2018).epub', 'level_1_title': 'EPUB Preamble',
'toc_id': -1}
  Content (Un-chunked Element):
  >> 'Cengage is a leading provider of customized learning solutions with
employees residing in nearly 40 different countries and sales in more than 125
countries around the world. Find your local representative at www.cengage.com.'
-------------------------

--- Document [17] ---
  Assigned Metadata: {'source': 'Bill Nelson, Amelia Phillips, Christopher
Steuart - Guide to Computer Forensics and Investigations_ Processing Digital
Evidence-Cengage Learning (2018).epub', 'level_1_title': 'EPUB Preamble',
'toc_id': -1}
  Content (Un-chunked Element):
  >> 'Cengage products are represented in Canada by Nelson Education, Ltd.'
-------------------------

--- Document [18] ---
  Assigned Metadata: {'source': 'Bill Nelson, Amelia Phillips, Christopher
Steuart - Guide to Computer Forensics and Investigations_ Processing Digital
Evidence-Cengage Learning (2018).epub', 'level_1_title': 'EPUB Preamble',
'toc_id': -1}
  Content (Un-chunked Element):
  >> 'To learn more about Cengage platforms and services, visit
www.cengage.com.'
-------------------------
```

```
--- Document [19] ---
  Assigned Metadata: {'source': 'Bill Nelson, Amelia Phillips, Christopher
Steuart - Guide to Computer Forensics and Investigations_ Processing Digital
Evidence-Cengage Learning (2018).epub', 'level_1_title': 'EPUB Preamble',
'toc_id': -1}
  Content (Un-chunked Element):
  >> 'To register or access your online learning solution or purchase materials
for your course, visit www.cengagebrain.com.'
--------------------------

--- Document [20] ---
  Assigned Metadata: {'source': 'Bill Nelson, Amelia Phillips, Christopher
Steuart - Guide to Computer Forensics and Investigations_ Processing Digital
Evidence-Cengage Learning (2018).epub', 'level_1_title': 'EPUB Preamble',
'toc_id': -1}
  Content (Un-chunked Element):
  >> 'Notice to the Reader Publisher does not warrant or guarantee any of the
products described herein or perform any independent analysis in connection with
any of the product information contained herein. Publisher does not assume, and
expressly disclaims, any obligation to obtain and include information other than
that provided to it by the manufacturer. The reader is expressly warned to
consider and adopt all safety precautions that might be indicated by the
activities described herein and to avoid all potential hazards. By following the
instructions contained herein, the reader willingly assumes all risks in
connection with such instructions. The publisher makes no representations or
warranties of any kind, including but not limited to, the warranties of fitness
for particular purpose or merchantability, nor are any such representations
implied with respect to the material set forth herein, and the publisher takes
no responsibility with respect to such material. The publisher shall not be
liable for any special, consequential, or exemplary damages resulting, in whole
or part, from the readers' use of, or reliance upon, this material.'
--------------------------

--- Document [21] ---
  Assigned Metadata: {'source': 'Bill Nelson, Amelia Phillips, Christopher
Steuart - Guide to Computer Forensics and Investigations_ Processing Digital
Evidence-Cengage Learning (2018).epub', 'level_1_title': 'Preface', 'toc_id': 3}
  Content (Un-chunked Element):
  >> 'Preface'
--------------------------

--- Document [22] ---
  Assigned Metadata: {'source': 'Bill Nelson, Amelia Phillips, Christopher
Steuart - Guide to Computer Forensics and Investigations_ Processing Digital
Evidence-Cengage Learning (2018).epub', 'level_1_title': 'Preface', 'toc_id': 3}
  Content (Un-chunked Element):
  >> 'Guide to Computer Forensics and Investigations is now in its sixth
```

edition. As digital technology and cyberspace have evolved from their early roots as basic communication platforms into the hyper-connected world we live in today, so has the demand for people who have the knowledge and skills to investigate legal and technical issues involving computers and digital technology. My sincere compliments to the authors and publishing staff who have made this textbook such a remarkable resource for thousands of students and practitioners worldwide.'

--- Document [23] ---
  Assigned Metadata: {'source': 'Bill Nelson, Amelia Phillips, Christopher Steuart - Guide to Computer Forensics and Investigations_ Processing Digital Evidence-Cengage Learning (2018).epub', 'level_1_title': 'Preface', 'toc_id': 3}
  Content (Un-chunked Element):
  >> 'Computers, the Internet, and the world's digital ecosystem are all instrumental in how we conduct our daily lives. When the founding fathers of the modern computing era were designing the digital infrastructure as we know it today, security and temporal accountability issues were not at the top of their list of things to do. The technological advancement of these systems over the past 10 years has changed the way we learn, socialize, and conduct business. Finding digital data that can be used as evidence to incriminate or exonerate a suspect accused in a legal or administrative proceeding is not an easy task.'

--- Document [24] ---
  Assigned Metadata: {'source': 'Bill Nelson, Amelia Phillips, Christopher Steuart - Guide to Computer Forensics and Investigations_ Processing Digital Evidence-Cengage Learning (2018).epub', 'level_1_title': 'Preface', 'toc_id': 3}
  Content (Un-chunked Element):
  >> 'Cyberthreats have become pervasive in modern society. They range from simple computer viruses to complex ransomware and cyber extortion schemes. The ability to conduct sophisticated digital forensics investigations has become a requirement in both the government and commercial sectors. Currently, the organizations and agencies whose job it is to investigate both criminal and civil matters involving the use of rapidly developing digital technology often struggle to keep up with the ever-changing digital landscape. Additionally, finding trained and qualified people to conduct these types of inquiries has been challenging.'

--- Document [25] ---
  Assigned Metadata: {'source': 'Bill Nelson, Amelia Phillips, Christopher Steuart - Guide to Computer Forensics and Investigations_ Processing Digital Evidence-Cengage Learning (2018).epub', 'level_1_title': 'Preface', 'toc_id': 3}
  Content (Un-chunked Element):
  >> 'Today, an entire industry has evolved for the purpose of investigating events occurring in cyberspace to include incidents involving international and corporate espionage, massive data breaches, and even cyberterrorism. The

```
opportunities for employment in this field are expanding every day.
Professionals in this exciting field of endeavor are now in high demand and are
expected to have multiple skill sets in areas such as malware analysis, cloud
computing, social media, and mobile device forensics.'
-------------------------

--- Document [26] ---
  Assigned Metadata: {'source': 'Bill Nelson, Amelia Phillips, Christopher
Steuart - Guide to Computer Forensics and Investigations_ Processing Digital
Evidence-Cengage Learning (2018).epub', 'level_1_title': 'Preface', 'toc_id': 3}
  Content (Un-chunked Element):
  >> 'Guide to Computer Forensics and Investigations can now be found in both
academic and professional environments as a reliable source of current technical
information and practical exercises concerning investigations involving the
latest digital technologies. It's my belief that this book, combined with an
enthusiastic and knowledgeable facilitator, makes for a fascinating course of
instruction.'
-------------------------

--- Document [27] ---
  Assigned Metadata: {'source': 'Bill Nelson, Amelia Phillips, Christopher
Steuart - Guide to Computer Forensics and Investigations_ Processing Digital
Evidence-Cengage Learning (2018).epub', 'level_1_title': 'Preface', 'toc_id': 3}
  Content (Un-chunked Element):
  >> 'As I have stated to many of my students in the past, it's not just laptop
computers and servers that harbor the binary code of ones and zeros, but an
infinite array of digital devices. If one of these devices retains evidence of a
crime, it's up to newly trained and educated digital detectives to find the
evidence in a forensically sound manner. This book will assist both students and
practitioners in accomplishing this goal.'
-------------------------

--- Document [28] ---
  Assigned Metadata: {'source': 'Bill Nelson, Amelia Phillips, Christopher
Steuart - Guide to Computer Forensics and Investigations_ Processing Digital
Evidence-Cengage Learning (2018).epub', 'level_1_title': 'Preface', 'toc_id': 3}
  Content (Un-chunked Element):
  >> 'Respectfully,'
-------------------------

--- Document [29] ---
  Assigned Metadata: {'source': 'Bill Nelson, Amelia Phillips, Christopher
Steuart - Guide to Computer Forensics and Investigations_ Processing Digital
Evidence-Cengage Learning (2018).epub', 'level_1_title': 'Preface', 'toc_id': 3}
  Content (Un-chunked Element):
  >> 'John A. Sgromolo'
-------------------------
```

```
--- Document [30] ---
  Assigned Metadata: {'source': 'Bill Nelson, Amelia Phillips, Christopher
Steuart - Guide to Computer Forensics and Investigations_ Processing Digital
Evidence-Cengage Learning (2018).epub', 'level_1_title': 'Preface', 'toc_id': 3}
  Content (Un-chunked Element):
  >> 'As a Senior Special Agent, John was one of the founding members of the
NCIS Computer Crime Investigations Group. John left government service to run
his own company, Digital Forensics, Inc., and has taught hundreds of law
enforcement and corporate students nationwide in the art and science of digital
forensics investigations. Currently, he serves as a senior consultant for
Verizon's Global Security Services, where he helps manage the Threat Intel
Response Service.'
-------------------------

--- Document [31] ---
  Assigned Metadata: {'source': 'Bill Nelson, Amelia Phillips, Christopher
Steuart - Guide to Computer Forensics and Investigations_ Processing Digital
Evidence-Cengage Learning (2018).epub', 'level_1_title': 'Introduction',
'toc_id': 4}
  Content (Un-chunked Element):
  >> 'Introduction'
-------------------------

--- Document [32] ---
  Assigned Metadata: {'source': 'Bill Nelson, Amelia Phillips, Christopher
Steuart - Guide to Computer Forensics and Investigations_ Processing Digital
Evidence-Cengage Learning (2018).epub', 'level_1_title': 'Introduction',
'toc_id': 4}
  Content (Un-chunked Element):
  >> 'Computer forensics, now most commonly called "digital forensics," has been
a professional field for many years, but most well-established experts in the
field have been self-taught. The growth of the Internet and the worldwide
proliferation of computers have increased the need for digital investigations.
Computers can be used to commit crimes, and crimes can be recorded on computers,
including company policy violations, embezzlement, e-mail harassment, murder,
leaks of proprietary information, and even terrorism. Law enforcement, network
administrators, attorneys, and private investigators now rely on the skills of
professional digital forensics experts to investigate criminal and civil cases.'
-------------------------

--- Document [33] ---
  Assigned Metadata: {'source': 'Bill Nelson, Amelia Phillips, Christopher
Steuart - Guide to Computer Forensics and Investigations_ Processing Digital
Evidence-Cengage Learning (2018).epub', 'level_1_title': 'Introduction',
'toc_id': 4}
  Content (Un-chunked Element):
  >> 'This book is not intended to provide comprehensive training in digital
forensics. It does, however, give you a solid foundation by introducing digital
```

```
forensics to those who are new to the field. Other books on digital forensics
are targeted to experts; this book is intended for novices who have a thorough
grounding in computer and networking basics.'
-------------------------

--- Document [34] ---
  Assigned Metadata: {'source': 'Bill Nelson, Amelia Phillips, Christopher
Steuart - Guide to Computer Forensics and Investigations_ Processing Digital
Evidence-Cengage Learning (2018).epub', 'level_1_title': 'Introduction',
'toc_id': 4}
  Content (Un-chunked Element):
  >> 'The new generation of digital forensics experts needs more initial
training because operating systems, computer and mobile device hardware, and
forensics software tools are changing more quickly. This book covers current and
past operating systems and a range of hardware, from basic workstations and
high-end network servers to a wide array of mobile devices. Although this book
focuses on a few forensics software tools, it also reviews and discusses other
currently available tools.'
-------------------------

--- Document [35] ---
  Assigned Metadata: {'source': 'Bill Nelson, Amelia Phillips, Christopher
Steuart - Guide to Computer Forensics and Investigations_ Processing Digital
Evidence-Cengage Learning (2018).epub', 'level_1_title': 'Introduction',
'toc_id': 4}
  Content (Un-chunked Element):
  >> 'The purpose of this book is to guide you toward becoming a skilled digital
forensics investigator. A secondary goal is to help you pass related
certification exams. As the field of digital forensics and investigations
matures, keep in mind that certifications will change. You can find more
information on certifications in Chapter 2 and Appendix A.'
-------------------------

--- Document [36] ---
  Assigned Metadata: {'source': 'Bill Nelson, Amelia Phillips, Christopher
Steuart - Guide to Computer Forensics and Investigations_ Processing Digital
Evidence-Cengage Learning (2018).epub', 'level_1_title': 'Introduction',
'toc_id': 4}
  Content (Un-chunked Element):
  >> 'Intended Audience'
-------------------------

--- Document [37] ---
  Assigned Metadata: {'source': 'Bill Nelson, Amelia Phillips, Christopher
Steuart - Guide to Computer Forensics and Investigations_ Processing Digital
Evidence-Cengage Learning (2018).epub', 'level_1_title': 'Introduction',
'toc_id': 4}
  Content (Un-chunked Element):
```

>> 'Although this book can be used by people with a wide range of backgrounds, it's intended for those with A+ and Network+ certifications or the equivalent. A networking background is necessary so that you understand how computers operate in a networked environment and can work with a network administrator when needed. In addition, you must know how to use a computer from the command line and how to use common operating systems, including Windows, Linux, and macOS, and their related hardware.'

--- Document [38] ---
Assigned Metadata: {'source': 'Bill Nelson, Amelia Phillips, Christopher Steuart - Guide to Computer Forensics and Investigations_ Processing Digital Evidence-Cengage Learning (2018).epub', 'level_1_title': 'Introduction', 'toc_id': 4}
Content (Un-chunked Element):
>> 'This book can be used at any educational level, from technical high schools and community colleges to graduate students. Current professionals in the public and private sectors can also use this book. Each group will approach investigative problems from a different perspective, but all will benefit from the coverage.'

--- Document [39] ---
Assigned Metadata: {'source': 'Bill Nelson, Amelia Phillips, Christopher Steuart - Guide to Computer Forensics and Investigations_ Processing Digital Evidence-Cengage Learning (2018).epub', 'level_1_title': 'Introduction', 'toc_id': 4}
Content (Un-chunked Element):
>> 'What's New in This Edition'

--- Document [40] ---
Assigned Metadata: {'source': 'Bill Nelson, Amelia Phillips, Christopher Steuart - Guide to Computer Forensics and Investigations_ Processing Digital Evidence-Cengage Learning (2018).epub', 'level_1_title': 'Introduction', 'toc_id': 4}
Content (Un-chunked Element):
>> 'The chapter flow of this book is organized so that you're first exposed to what happens in a forensics lab and how to set one up before you get into the nuts and bolts. Coverage of several GUI tools has been added to give you a familiarity with some widely used software. In addition, Chapter 11 has additional coverage of social media forensics, Chapter 12 has been expanded to include more information on smartphones and tablets, and Chapter 13 on forensics procedures for information stored in the cloud has been updated. Corrections have been made to this edition based on feedback from users, and all software tools and Web sites have been updated to reflect what's current at the time of publication. Finally, a new digital lab manual is being offered in MindTap for Guide to Computer Forensics and Investigationsto go with the sixth edition

```
textbook.'
-------------------------

--- Document [41] ---
  Assigned Metadata: {'source': 'Bill Nelson, Amelia Phillips, Christopher
Steuart - Guide to Computer Forensics and Investigations_ Processing Digital
Evidence-Cengage Learning (2018).epub', 'level_1_title': 'Introduction',
'toc_id': 4}
  Content (Un-chunked Element):
  >> 'Chapter Descriptions'
-------------------------

--- Document [42] ---
  Assigned Metadata: {'source': 'Bill Nelson, Amelia Phillips, Christopher
Steuart - Guide to Computer Forensics and Investigations_ Processing Digital
Evidence-Cengage Learning (2018).epub', 'level_1_title': 'Introduction',
'toc_id': 4}
  Content (Un-chunked Element):
  >> 'Here is a summary of the topics covered in each chapter of this book:'
-------------------------

--- Document [43] ---
  Assigned Metadata: {'source': 'Bill Nelson, Amelia Phillips, Christopher
Steuart - Guide to Computer Forensics and Investigations_ Processing Digital
Evidence-Cengage Learning (2018).epub', 'level_1_title': 'Introduction',
'toc_id': 4}
  Content (Un-chunked Element):
  >> 'Chapter 1 , "Understanding the Digital Forensics Profession and
Investigations," introduces you to the history of digital forensics and explains
how the use of electronic evidence developed. It also reviews legal issues and
compares public and private sector cases. This chapter also explains how to take
a systematic approach to preparing a digital investigation, describes how to
conduct an investigation, and summarizes requirements for workstations and
software.'
-------------------------

--- Document [44] ---
  Assigned Metadata: {'source': 'Bill Nelson, Amelia Phillips, Christopher
Steuart - Guide to Computer Forensics and Investigations_ Processing Digital
Evidence-Cengage Learning (2018).epub', 'level_1_title': 'Introduction',
'toc_id': 4}
  Content (Un-chunked Element):
  >> 'Chapter 2 , "The Investigator's Office and Laboratory," outlines physical
requirements and equipment for digital forensics labs, from small private
investigators' labs to the regional FBI lab. It also covers certifications for
digital investigators and building a business case for a forensics lab.'
-------------------------
```

--- Document [45] ---
  Assigned Metadata: {'source': 'Bill Nelson, Amelia Phillips, Christopher Steuart - Guide to Computer Forensics and Investigations_ Processing Digital Evidence-Cengage Learning (2018).epub', 'level_1_title': 'Introduction', 'toc_id': 4}
  Content (Un-chunked Element):
  >> 'Chapter 3 , "Data Acquisition," explains how to prepare to acquire data from a suspect’s drive and discusses available Linux and GUI acquisition tools. This chapter also discusses acquiring data from RAID systems and gives you an overview of tools for remote acquisitions.'

--- Document [46] ---
  Assigned Metadata: {'source': 'Bill Nelson, Amelia Phillips, Christopher Steuart - Guide to Computer Forensics and Investigations_ Processing Digital Evidence-Cengage Learning (2018).epub', 'level_1_title': 'Introduction', 'toc_id': 4}
  Content (Un-chunked Element):
  >> 'Chapter 4 , "Processing Crime and Incident Scenes," explains search warrants and the nature of a typical digital forensics case. It discusses when to use outside professionals, how to assemble a team, and how to evaluate a case and explains the correct procedures for searching and seizing evidence. This chapter also introduces you to calculating hashes to verify data you collect.'

--- Document [47] ---
  Assigned Metadata: {'source': 'Bill Nelson, Amelia Phillips, Christopher Steuart - Guide to Computer Forensics and Investigations_ Processing Digital Evidence-Cengage Learning (2018).epub', 'level_1_title': 'Introduction', 'toc_id': 4}
  Content (Un-chunked Element):
  >> 'Chapter 5 , "Working with Windows and CLI Systems," discusses the most common operating systems. You learn what happens and what files are altered during computer startup and how file systems deal with deleted and slack space. In addition, this chapter covers some options for decrypting drives encrypted with whole disk encryption and explains the purpose of using virtual machines.'

--- Document [48] ---
  Assigned Metadata: {'source': 'Bill Nelson, Amelia Phillips, Christopher Steuart - Guide to Computer Forensics and Investigations_ Processing Digital Evidence-Cengage Learning (2018).epub', 'level_1_title': 'Introduction', 'toc_id': 4}
  Content (Un-chunked Element):
  >> 'Chapter 6 , "Current Digital Forensics Tools," explores current digital forensics software and hardware tools, including those that might not be readily available, and evaluates their strengths and weaknesses.'

```
--- Document [49] ---
  Assigned Metadata: {'source': 'Bill Nelson, Amelia Phillips, Christopher
Steuart - Guide to Computer Forensics and Investigations_ Processing Digital
Evidence-Cengage Learning (2018).epub', 'level_1_title': 'Introduction',
'toc_id': 4}
  Content (Un-chunked Element):
  >> 'Chapter 7 , "Linux and Macintosh File Systems," continues the operating
system discussion from Chapter 5 by examining Macintosh and Linux OSs and file
systems. It also gives you practice in using Linux forensics tools.'
-------------------------

--- Document [50] ---
  Assigned Metadata: {'source': 'Bill Nelson, Amelia Phillips, Christopher
Steuart - Guide to Computer Forensics and Investigations_ Processing Digital
Evidence-Cengage Learning (2018).epub', 'level_1_title': 'Introduction',
'toc_id': 4}
  Content (Un-chunked Element):
  >> 'Chapter 8 , "Recovering Graphics Files," explains how to recover graphics
files and examines data compression, carving data, reconstructing file
fragments, and steganography and copyright issues.'
-------------------------

--- Document [51] ---
  Assigned Metadata: {'source': 'Bill Nelson, Amelia Phillips, Christopher
Steuart - Guide to Computer Forensics and Investigations_ Processing Digital
Evidence-Cengage Learning (2018).epub', 'level_1_title': 'Introduction',
'toc_id': 4}
  Content (Un-chunked Element):
  >> 'Chapter 9 , "Digital Forensics Analysis and Validation," covers
determining what data to collect and analyze and refining investigation plans.
It also explains validation with hex editors and forensics software and data-
hiding techniques.'
-------------------------

--- Document [52] ---
  Assigned Metadata: {'source': 'Bill Nelson, Amelia Phillips, Christopher
Steuart - Guide to Computer Forensics and Investigations_ Processing Digital
Evidence-Cengage Learning (2018).epub', 'level_1_title': 'Introduction',
'toc_id': 4}
  Content (Un-chunked Element):
  >> 'Chapter 10 , "Virtual Machine Forensics, Live Acquisitions, and Network
Forensics," covers tools and methods for conducting forensic analysis of virtual
machines, performing live acquisitions, reviewing network logs for evidence, and
using network-monitoring tools to detect unauthorized access. It also examines
using Linux tools and the Honeynet Project's resources.'
-------------------------
```

```
--- Document [53] ---
  Assigned Metadata: {'source': 'Bill Nelson, Amelia Phillips, Christopher
Steuart - Guide to Computer Forensics and Investigations_ Processing Digital
Evidence-Cengage Learning (2018).epub', 'level_1_title': 'Introduction',
'toc_id': 4}
  Content (Un-chunked Element):
  >> 'Chapter 11 , "E-mail and Social Media Investigations," examines e-mail
crimes and violations and reviews some specialized e-mail and social media
forensics tools. It also explains how to approach investigating social media
communications and handling the challenges this content poses.'
-------------------------

--- Document [54] ---
  Assigned Metadata: {'source': 'Bill Nelson, Amelia Phillips, Christopher
Steuart - Guide to Computer Forensics and Investigations_ Processing Digital
Evidence-Cengage Learning (2018).epub', 'level_1_title': 'Introduction',
'toc_id': 4}
  Content (Un-chunked Element):
  >> 'Chapter 12 , "Mobile Device Forensics and The Internet of Anything,"
covers investigation techniques and acquisition procedures for smartphones,
other mobile devices, Internet of Anything devices, and sensors. You learn where
data might be stored or backed up and what tools are available for these
investigations.'
-------------------------

--- Document [55] ---
  Assigned Metadata: {'source': 'Bill Nelson, Amelia Phillips, Christopher
Steuart - Guide to Computer Forensics and Investigations_ Processing Digital
Evidence-Cengage Learning (2018).epub', 'level_1_title': 'Introduction',
'toc_id': 4}
  Content (Un-chunked Element):
  >> 'Chapter 13 , "Cloud Forensics," summarizes the legal and technical
challenges in conducting cloud forensics. It also describes how to acquire cloud
data and explains how remote acquisition tools can be used in cloud
investigations.'
-------------------------

--- Document [56] ---
  Assigned Metadata: {'source': 'Bill Nelson, Amelia Phillips, Christopher
Steuart - Guide to Computer Forensics and Investigations_ Processing Digital
Evidence-Cengage Learning (2018).epub', 'level_1_title': 'Introduction',
'toc_id': 4}
  Content (Un-chunked Element):
  >> 'Chapter 14 , "Report Writing for High-Tech Investigations," discusses the
importance of report writing in digital forensics examinations; offers
guidelines on report content, structure, and presentation; and explains how to
generate report findings with forensics software tools.'
-------------------------
```

```
--- Document [57] ---
  Assigned Metadata: {'source': 'Bill Nelson, Amelia Phillips, Christopher
Steuart - Guide to Computer Forensics and Investigations_ Processing Digital
Evidence-Cengage Learning (2018).epub', 'level_1_title': 'Introduction',
'toc_id': 4}
  Content (Un-chunked Element):
  >> 'Chapter 15 , "Expert Testimony in Digital Investigations," explores the
role of an expert witness or a fact witness, including developing a curriculum
vitae, understanding the trial process, and preparing forensics evidence for
testimony. It also offers guidelines for testifying in court and at depositions
and hearings.'
-------------------------

--- Document [58] ---
  Assigned Metadata: {'source': 'Bill Nelson, Amelia Phillips, Christopher
Steuart - Guide to Computer Forensics and Investigations_ Processing Digital
Evidence-Cengage Learning (2018).epub', 'level_1_title': 'Introduction',
'toc_id': 4}
  Content (Un-chunked Element):
  >> 'Chapter 16 , "Ethics for the Expert Witness," provides guidance in the
principles and practice of ethics for digital forensics investigators and
examines other professional organizations' codes of ethics.'
-------------------------

--- Document [59] ---
  Assigned Metadata: {'source': 'Bill Nelson, Amelia Phillips, Christopher
Steuart - Guide to Computer Forensics and Investigations_ Processing Digital
Evidence-Cengage Learning (2018).epub', 'level_1_title': 'Introduction',
'toc_id': 4}
  Content (Un-chunked Element):
  >> 'Appendix A , "Certification Test References," provides information on the
National Institute of Standards and Technology (NIST) testing processes for
validating digital forensics tools and covers digital forensics certifications
and training programs.'
-------------------------

--- Document [60] ---
  Assigned Metadata: {'source': 'Bill Nelson, Amelia Phillips, Christopher
Steuart - Guide to Computer Forensics and Investigations_ Processing Digital
Evidence-Cengage Learning (2018).epub', 'level_1_title': 'Introduction',
'toc_id': 4}
  Content (Un-chunked Element):
  >> 'Appendix B , "Digital Forensics References," lists recommended books,
journals, e-mail lists, and Web sites for additional information and further
study. It also covers the latest ISO 27000 standards that apply to digital
forensics.'
-------------------------
```

```
--- Document [61] ---
  Assigned Metadata: {'source': 'Bill Nelson, Amelia Phillips, Christopher
Steuart - Guide to Computer Forensics and Investigations_ Processing Digital
Evidence-Cengage Learning (2018).epub', 'level_1_title': 'Introduction',
'toc_id': 4}
  Content (Un-chunked Element):
  >> 'Appendix C , "Digital Forensics Lab Considerations," provides more
information on considerations for forensics labs, including certifications,
ergonomics, structural design, and communication and fire-suppression systems.
It also covers applicable ISO standards.'
-------------------------

--- Document [62] ---
  Assigned Metadata: {'source': 'Bill Nelson, Amelia Phillips, Christopher
Steuart - Guide to Computer Forensics and Investigations_ Processing Digital
Evidence-Cengage Learning (2018).epub', 'level_1_title': 'Introduction',
'toc_id': 4}
  Content (Un-chunked Element):
  >> 'Appendix D , "Legacy File System and Forensics Tools," reviews FAT file
system basics and Mac legacy file systems and explains using DOS forensics
tools, creating forensic boot media, and using scripts. It also has an overview
of the hexadecimal numbering system and how it's applied to digital
information.'
-------------------------

--- Document [63] ---
  Assigned Metadata: {'source': 'Bill Nelson, Amelia Phillips, Christopher
Steuart - Guide to Computer Forensics and Investigations_ Processing Digital
Evidence-Cengage Learning (2018).epub', 'level_1_title': 'Introduction',
'toc_id': 4}
  Content (Un-chunked Element):
  >> 'Features'
-------------------------

--- Document [64] ---
  Assigned Metadata: {'source': 'Bill Nelson, Amelia Phillips, Christopher
Steuart - Guide to Computer Forensics and Investigations_ Processing Digital
Evidence-Cengage Learning (2018).epub', 'level_1_title': 'Introduction',
'toc_id': 4}
  Content (Un-chunked Element):
  >> 'To help you fully understand digital forensics, this book includes many
features designed to enhance your learning experience:'
-------------------------

--- Document [65] ---
  Assigned Metadata: {'source': 'Bill Nelson, Amelia Phillips, Christopher
Steuart - Guide to Computer Forensics and Investigations_ Processing Digital
```

Evidence-Cengage Learning (2018).epub', 'level_1_title': 'Introduction',
'toc_id': 4}
  Content (Un-chunked Element):
  >> 'Chapter objectives-Each chapter begins with a detailed list of the
concepts to be mastered in that chapter. This list gives you a quick reference
to the chapter's contents and is a useful study aid.'

--- Document [66] ---
  Assigned Metadata: {'source': 'Bill Nelson, Amelia Phillips, Christopher
Steuart - Guide to Computer Forensics and Investigations_ Processing Digital
Evidence-Cengage Learning (2018).epub', 'level_1_title': 'Introduction',
'toc_id': 4}
  Content (Un-chunked Element):
  >> 'Figures and tables-Screenshots are used as guidelines for stepping through
commands and forensics tools. For tools not included with the book or that
aren't offered in free demo versions, figures have been added when possible to
illustrate the tool's interface. Tables are used throughout the book to present
information in an organized, easy-to-grasp manner.'

--- Document [67] ---
  Assigned Metadata: {'source': 'Bill Nelson, Amelia Phillips, Christopher
Steuart - Guide to Computer Forensics and Investigations_ Processing Digital
Evidence-Cengage Learning (2018).epub', 'level_1_title': 'Introduction',
'toc_id': 4}
  Content (Un-chunked Element):
  >> 'Chapter summaries-Each chapter's material is followed by a summary of the
concepts introduced in that chapter. These summaries are a helpful way to review
the ideas covered in each chapter.'

--- Document [68] ---
  Assigned Metadata: {'source': 'Bill Nelson, Amelia Phillips, Christopher
Steuart - Guide to Computer Forensics and Investigations_ Processing Digital
Evidence-Cengage Learning (2018).epub', 'level_1_title': 'Introduction',
'toc_id': 4}
  Content (Un-chunked Element):
  >> 'Key terms-Following the chapter summary, all new terms introduced in the
chapter with boldfaced text are gathered together in the Key Terms list. This
list encourages a more thorough understanding of the chapter's key concepts and
is a useful reference.'

--- Document [69] ---
  Assigned Metadata: {'source': 'Bill Nelson, Amelia Phillips, Christopher
Steuart - Guide to Computer Forensics and Investigations_ Processing Digital
Evidence-Cengage Learning (2018).epub', 'level_1_title': 'Introduction',

'toc_id': 4}
  Content (Un-chunked Element):
  >> 'Review questions-The end-of-chapter assessment begins with a set of review questions that reinforce the main concepts in each chapter. These questions help you evaluate and apply the material you have learned.'

--- Document [70] ---
  Assigned Metadata: {'source': 'Bill Nelson, Amelia Phillips, Christopher Steuart - Guide to Computer Forensics and Investigations_ Processing Digital Evidence-Cengage Learning (2018).epub', 'level_1_title': 'Introduction', 'toc_id': 4}
  Content (Un-chunked Element):
  >> 'Hands-on projects-Although understanding the theory behind digital technology is important, nothing can improve on real-world experience. To this end, each chapter offers several hands-on projects with software supplied as free downloads on the student companion site and in MindTap. You can explore a variety of ways to acquire and even hide evidence. For the conceptual chapters, research projects are supplied.'

--- Document [71] ---
  Assigned Metadata: {'source': 'Bill Nelson, Amelia Phillips, Christopher Steuart - Guide to Computer Forensics and Investigations_ Processing Digital Evidence-Cengage Learning (2018).epub', 'level_1_title': 'Introduction', 'toc_id': 4}
  Content (Un-chunked Element):
  >> 'Case projects-At the end of each chapter are case projects. To do these projects, you must draw on real-world common sense as well as your knowledge of the technical topics covered to that point in the book. Your goal for each project is to come up with answers to problems similar to those you'll face as a working digital forensics investigator.'

--- Document [72] ---
  Assigned Metadata: {'source': 'Bill Nelson, Amelia Phillips, Christopher Steuart - Guide to Computer Forensics and Investigations_ Processing Digital Evidence-Cengage Learning (2018).epub', 'level_1_title': 'Introduction', 'toc_id': 4}
  Content (Un-chunked Element):
  >> 'Software and student data files-Student data files are available for download from the student companion site and the MindTap for this book and are used for activities and projects in the chapters. Demo and freeware software used in this book can be downloaded from the Web sites specified in activities and projects or in "Digital Forensics Software" later in this introduction.'

--- Document [73] ---

```
  Assigned Metadata: {'source': 'Bill Nelson, Amelia Phillips, Christopher
Steuart - Guide to Computer Forensics and Investigations_ Processing Digital
Evidence-Cengage Learning (2018).epub', 'level_1_title': 'Introduction',
'toc_id': 4}
  Content (Un-chunked Element):
  >> 'Student companion site-To access the student companion site, go to
www.cengagebrain.com and search for the sixth edition by entering the title,
author's name, or ISBN. On the product page, click the Free Materials tab, and
then click Save to MyHome. Then you can sign in as a returning student or choose
to create a new account. After you've logged on, you can begin accessing your
free study tools.'
-------------------------

--- Document [74] ---
  Assigned Metadata: {'source': 'Bill Nelson, Amelia Phillips, Christopher
Steuart - Guide to Computer Forensics and Investigations_ Processing Digital
Evidence-Cengage Learning (2018).epub', 'level_1_title': 'Introduction',
'toc_id': 4}
  Content (Un-chunked Element):
  >> 'Text and Graphic Conventions'
-------------------------

--- Document [75] ---
  Assigned Metadata: {'source': 'Bill Nelson, Amelia Phillips, Christopher
Steuart - Guide to Computer Forensics and Investigations_ Processing Digital
Evidence-Cengage Learning (2018).epub', 'level_1_title': 'Introduction',
'toc_id': 4}
  Content (Un-chunked Element):
  >> 'When appropriate, additional information and exercises have been added to
this book to help you better understand the topic at hand. The following icons
used in this book alert you to additional materials:'
-------------------------

--- Document [76] ---
  Assigned Metadata: {'source': 'Bill Nelson, Amelia Phillips, Christopher
Steuart - Guide to Computer Forensics and Investigations_ Processing Digital
Evidence-Cengage Learning (2018).epub', 'level_1_title': 'Introduction',
'toc_id': 4}
  Content (Un-chunked Element):
  >> 'Note'
-------------------------

--- Document [77] ---
  Assigned Metadata: {'source': 'Bill Nelson, Amelia Phillips, Christopher
Steuart - Guide to Computer Forensics and Investigations_ Processing Digital
Evidence-Cengage Learning (2018).epub', 'level_1_title': 'Introduction',
'toc_id': 4}
  Content (Un-chunked Element):
```

```
  >> 'Note Box'
-------------------------

--- Document [78] ---
  Assigned Metadata: {'source': 'Bill Nelson, Amelia Phillips, Christopher
Steuart - Guide to Computer Forensics and Investigations_ Processing Digital
Evidence-Cengage Learning (2018).epub', 'level_1_title': 'Introduction',
'toc_id': 4}
  Content (Un-chunked Element):
  >> 'The Note icon draws your attention to additional helpful material related
to the subject being covered.'
-------------------------

--- Document [79] ---
  Assigned Metadata: {'source': 'Bill Nelson, Amelia Phillips, Christopher
Steuart - Guide to Computer Forensics and Investigations_ Processing Digital
Evidence-Cengage Learning (2018).epub', 'level_1_title': 'Introduction',
'toc_id': 4}
  Content (Un-chunked Element):
  >> 'Tip'
-------------------------

--- Document [80] ---
  Assigned Metadata: {'source': 'Bill Nelson, Amelia Phillips, Christopher
Steuart - Guide to Computer Forensics and Investigations_ Processing Digital
Evidence-Cengage Learning (2018).epub', 'level_1_title': 'Introduction',
'toc_id': 4}
  Content (Un-chunked Element):
  >> 'Tips based on the authors' experiences offer extra information about how
to attack a problem or what to do in real-world situations.'
-------------------------

--- Document [81] ---
  Assigned Metadata: {'source': 'Bill Nelson, Amelia Phillips, Christopher
Steuart - Guide to Computer Forensics and Investigations_ Processing Digital
Evidence-Cengage Learning (2018).epub', 'level_1_title': 'Introduction',
'toc_id': 4}
  Content (Un-chunked Element):
  >> 'Caution'
-------------------------

--- Document [82] ---
  Assigned Metadata: {'source': 'Bill Nelson, Amelia Phillips, Christopher
Steuart - Guide to Computer Forensics and Investigations_ Processing Digital
Evidence-Cengage Learning (2018).epub', 'level_1_title': 'Introduction',
'toc_id': 4}
  Content (Un-chunked Element):
  >> 'Caution Box'
```

```
------------------------

--- Document [83] ---
  Assigned Metadata: {'source': 'Bill Nelson, Amelia Phillips, Christopher
Steuart - Guide to Computer Forensics and Investigations_ Processing Digital
Evidence-Cengage Learning (2018).epub', 'level_1_title': 'Introduction',
'toc_id': 4}
  Content (Un-chunked Element):
  >> 'The Caution icon warns you about potential mistakes or problems and
explains how to avoid them.'
------------------------

--- Document [84] ---
  Assigned Metadata: {'source': 'Bill Nelson, Amelia Phillips, Christopher
Steuart - Guide to Computer Forensics and Investigations_ Processing Digital
Evidence-Cengage Learning (2018).epub', 'level_1_title': 'Chapter 1.
Understanding the Digital Forensics Profession and Investigations',
'level_2_title': 'Chapter Review', 'level_3_title': 'Hands-On Projects',
'toc_id': 53}
  Content (Un-chunked Element):
  >> 'Hands-On Projects'
------------------------

--- Document [85] ---
  Assigned Metadata: {'source': 'Bill Nelson, Amelia Phillips, Christopher
Steuart - Guide to Computer Forensics and Investigations_ Processing Digital
Evidence-Cengage Learning (2018).epub', 'level_1_title': 'Chapter 1.
Understanding the Digital Forensics Profession and Investigations',
'level_2_title': 'Chapter Review', 'level_3_title': 'Hands-On Projects',
'toc_id': 53}
  Content (Un-chunked Element):
  >> 'The hands-on icon indicates that the projects following it give you a
chance to practice using software tools and get hands-on experience.'
------------------------

--- Document [86] ---
  Assigned Metadata: {'source': 'Bill Nelson, Amelia Phillips, Christopher
Steuart - Guide to Computer Forensics and Investigations_ Processing Digital
Evidence-Cengage Learning (2018).epub', 'level_1_title': 'Chapter 1.
Understanding the Digital Forensics Profession and Investigations',
'level_2_title': 'Chapter Review', 'level_3_title': 'Case Projects', 'toc_id':
54}
  Content (Un-chunked Element):
  >> 'Case Projects'
------------------------

--- Document [87] ---
  Assigned Metadata: {'source': 'Bill Nelson, Amelia Phillips, Christopher
```

Steuart - Guide to Computer Forensics and Investigations_ Processing Digital Evidence-Cengage Learning (2018).epub', 'level_1_title': 'Chapter 1. Understanding the Digital Forensics Profession and Investigations', 'level_2_title': 'Chapter Review', 'level_3_title': 'Case Projects', 'toc_id': 54}
  Content (Un-chunked Element):
  >> 'This icon marks the start of projects that require you to apply common sense and knowledge to solving problems involving that chapter's concepts.'

--- Document [88] ---
  Assigned Metadata: {'source': 'Bill Nelson, Amelia Phillips, Christopher Steuart - Guide to Computer Forensics and Investigations_ Processing Digital Evidence-Cengage Learning (2018).epub', 'level_1_title': 'Chapter 1. Understanding the Digital Forensics Profession and Investigations', 'level_2_title': 'Chapter Review', 'level_3_title': 'Case Projects', 'toc_id': 54}
  Content (Un-chunked Element):
  >> 'Student Resources'

--- Document [89] ---
  Assigned Metadata: {'source': 'Bill Nelson, Amelia Phillips, Christopher Steuart - Guide to Computer Forensics and Investigations_ Processing Digital Evidence-Cengage Learning (2018).epub', 'level_1_title': 'Chapter 1. Understanding the Digital Forensics Profession and Investigations', 'level_2_title': 'Chapter Review', 'level_3_title': 'Case Projects', 'toc_id': 54}
  Content (Un-chunked Element):
  >> 'MindTap for Guide to Computer Forensics and Investigations helps you learn on your terms.'

--- Document [90] ---
  Assigned Metadata: {'source': 'Bill Nelson, Amelia Phillips, Christopher Steuart - Guide to Computer Forensics and Investigations_ Processing Digital Evidence-Cengage Learning (2018).epub', 'level_1_title': 'Chapter 1. Understanding the Digital Forensics Profession and Investigations', 'level_2_title': 'Chapter Review', 'level_3_title': 'Case Projects', 'toc_id': 54}
  Content (Un-chunked Element):
  >> 'Instant access in your pocket: Take advantage of the MindTap Mobile App to learn on your terms. Read or listen to textbooks and study with the aid of instructor notifications, flashcards, and practice quizzes.'

--- Document [91] ---
  Assigned Metadata: {'source': 'Bill Nelson, Amelia Phillips, Christopher

Steuart - Guide to Computer Forensics and Investigations_ Processing Digital Evidence-Cengage Learning (2018).epub', 'level_1_title': 'Chapter 1. Understanding the Digital Forensics Profession and Investigations', 'level_2_title': 'Chapter Review', 'level_3_title': 'Case Projects', 'toc_id': 54}
  Content (Un-chunked Element):
  >> 'MindTap helps you create your own potential. Gear up for ultimate success: Track your scores and stay motivated toward your goals. Whether you have more work to do or are ahead of the curve, you'll know where you need to focus your efforts. The MindTap Green Dot will charge your confidence along the way.'

--- Document [92] ---
  Assigned Metadata: {'source': 'Bill Nelson, Amelia Phillips, Christopher Steuart - Guide to Computer Forensics and Investigations_ Processing Digital Evidence-Cengage Learning (2018).epub', 'level_1_title': 'Chapter 1. Understanding the Digital Forensics Profession and Investigations', 'level_2_title': 'Chapter Review', 'level_3_title': 'Case Projects', 'toc_id': 54}
  Content (Un-chunked Element):
  >> 'MindTap helps you own your progress. Make your textbook yours; no one knows what works for you better than you. Highlight key text, add notes, and create custom flashcards. When it's time to study, everything you've flagged or noted can be gathered into a guide you can organize.'

--- Document [93] ---
  Assigned Metadata: {'source': 'Bill Nelson, Amelia Phillips, Christopher Steuart - Guide to Computer Forensics and Investigations_ Processing Digital Evidence-Cengage Learning (2018).epub', 'level_1_title': 'Chapter 1. Understanding the Digital Forensics Profession and Investigations', 'level_2_title': 'Chapter Review', 'level_3_title': 'Case Projects', 'toc_id': 54}
  Content (Un-chunked Element):
  >> 'Lab Manual'

--- Document [94] ---
  Assigned Metadata: {'source': 'Bill Nelson, Amelia Phillips, Christopher Steuart - Guide to Computer Forensics and Investigations_ Processing Digital Evidence-Cengage Learning (2018).epub', 'level_1_title': 'Chapter 1. Understanding the Digital Forensics Profession and Investigations', 'level_2_title': 'Chapter Review', 'level_3_title': 'Case Projects', 'toc_id': 54}
  Content (Un-chunked Element):
  >> 'A new digital lab manual is being offered in the MindTap for Guide to Computer Forensics and Investigations to give you additional hands-on experience.'

```
-------------------------

--- Document [95] ---
  Assigned Metadata: {'source': 'Bill Nelson, Amelia Phillips, Christopher
Steuart - Guide to Computer Forensics and Investigations_ Processing Digital
Evidence-Cengage Learning (2018).epub', 'level_1_title': 'Chapter 1.
Understanding the Digital Forensics Profession and Investigations',
'level_2_title': 'Chapter Review', 'level_3_title': 'Case Projects', 'toc_id':
54}
  Content (Un-chunked Element):
  >> 'Lab Requirements'
-------------------------

--- Document [96] ---
  Assigned Metadata: {'source': 'Bill Nelson, Amelia Phillips, Christopher
Steuart - Guide to Computer Forensics and Investigations_ Processing Digital
Evidence-Cengage Learning (2018).epub', 'level_1_title': 'Chapter 1.
Understanding the Digital Forensics Profession and Investigations',
'level_2_title': 'Chapter Review', 'level_3_title': 'Case Projects', 'toc_id':
54}
  Content (Un-chunked Element):
  >> 'The hands-on projects in this book help you apply what you have learned
about digital forensics techniques. The following sections list the minimum
requirements for completing all the projects in this book. In addition to the
items listed, you must be able to download and install demo versions of
software.'
-------------------------

--- Document [97] ---
  Assigned Metadata: {'source': 'Bill Nelson, Amelia Phillips, Christopher
Steuart - Guide to Computer Forensics and Investigations_ Processing Digital
Evidence-Cengage Learning (2018).epub', 'level_1_title': 'Chapter 1.
Understanding the Digital Forensics Profession and Investigations',
'level_2_title': 'Chapter Review', 'level_3_title': 'Case Projects', 'toc_id':
54}
  Content (Un-chunked Element):
  >> 'Note'
-------------------------

--- Document [98] ---
  Assigned Metadata: {'source': 'Bill Nelson, Amelia Phillips, Christopher
Steuart - Guide to Computer Forensics and Investigations_ Processing Digital
Evidence-Cengage Learning (2018).epub', 'level_1_title': 'Chapter 1.
Understanding the Digital Forensics Profession and Investigations',
'level_2_title': 'Chapter Review', 'level_3_title': 'Case Projects', 'toc_id':
54}
  Content (Un-chunked Element):
  >> 'Note Box'
```

```
-------------------------

--- Document [99] ---
  Assigned Metadata: {'source': 'Bill Nelson, Amelia Phillips, Christopher
Steuart - Guide to Computer Forensics and Investigations_ Processing Digital
Evidence-Cengage Learning (2018).epub', 'level_1_title': 'Chapter 1.
Understanding the Digital Forensics Profession and Investigations',
'level_2_title': 'Chapter Review', 'level_3_title': 'Case Projects', 'toc_id':
54}
  Content (Un-chunked Element):
  >> 'Magnet AXIOM has a 30-day trial for download. If you aren't purchasing its
academic license or are planning to do the labs at home, it's recommended that
you wait until Chapter 11 to begin using this tool so that your trial doesn't
expire before you finish the projects. In addition, wait until Chapter 11 before
doing Hands-On Project 6-5, installing Magnet AXIOM.'
-------------------------

--- Document [100] ---
  Assigned Metadata: {'source': 'Bill Nelson, Amelia Phillips, Christopher
Steuart - Guide to Computer Forensics and Investigations_ Processing Digital
Evidence-Cengage Learning (2018).epub', 'level_1_title': 'Chapter 1.
Understanding the Digital Forensics Profession and Investigations',
'level_2_title': 'Chapter Review', 'level_3_title': 'Case Projects', 'toc_id':
54}
  Content (Un-chunked Element):
  >> 'Note'
-------------------------
```

6.1.1 Full Database Health & Hierarchy Diagnostic Report

[30]:
```python
# Cell 5.1: Full Database Health & Hierarchy Diagnostic Report (V5 - with␣
↪Content Preview)

import os
import json
import logging
import random
from typing import List, Dict, Any

# You might need to install pandas if you haven't already
try:
    import pandas as pd
    pandas_available = True
except ImportError:
    pandas_available = False
```

```
try:
    from langchain_chroma import Chroma
    from langchain_ollama.embeddings import OllamaEmbeddings
    from langchain_core.documents import Document
    langchain_available = True
except ImportError:
    langchain_available = False

# Setup Logger
logging.basicConfig(level=logging.INFO, format='%(asctime)s - %(levelname)s -␣
 ↪%(message)s')
logger = logging.getLogger(__name__)

# --- HELPER FUNCTIONS ---
def print_header(text: str, char: str = "="):
    """Prints a centered header to the console."""
    print("\n" + char * 80)
    print(text.center(80))
    print(char * 80)

def count_total_chunks(node: Dict) -> int:
    """Recursively counts all chunks in a node and its children."""
    total = node.get('_chunks', 0)
    for child_node in node.get('_children', {}).values():
        total += count_total_chunks(child_node)
    return total

def print_hierarchy_report(node: Dict, indent_level: int = 0):
    """
    Recursively prints the reconstructed hierarchy, sorting by sequential ToC␣
 ↪ID.
    """
    sorted_children = sorted(
        node.get('_children', {}).items(),
        key=lambda item: item[1].get('_toc_id', float('inf'))
    )

    for title, child_node in sorted_children:
        prefix = "  " * indent_level + "|-- "
        total_chunks_in_branch = count_total_chunks(child_node)
        direct_chunks = child_node.get('_chunks', 0)
        toc_id = child_node.get('_toc_id', 'N/A')
        print(f"{prefix}{title} [ID: {toc_id}] (Total Chuck in branch:␣
 ↪{total_chunks_in_branch}, Direct Chunk: {direct_chunks})")
        print_hierarchy_report(child_node, indent_level + 1)

def find_testable_sections(node: Dict, path: str, testable_list: List):
```

```
    """
    Recursively find sections with a decent number of "direct" chunks to test␣
 ↪sequence on.
    """
    if node.get('_chunks', 0) > 10 and not node.get('_children'):
        testable_list.append({
            "path": path,
            "toc_id": node.get('_toc_id'),
            "chunk_count": node.get('_chunks')
        })

    for title, child_node in node.get('_children', {}).items():
        new_path = f"{path} -> {title}" if path else title
        find_testable_sections(child_node, new_path, testable_list)


# --- MODIFIED TEST FUNCTION ---
def verify_chunk_sequence_and_content(vector_store: Chroma, hierarchy_tree:␣
 ↪Dict):
    """
    Selects a random ToC section, verifies chunk sequence, and displays the␣
 ↪reassembled content.
    """
    print_header("Chunk Sequence & Content Integrity Test", char="-")
    logger.info("Verifying chunk order and reassembling content for a random␣
 ↪ToC section.")

    # 1. Find a good section to test
    testable_sections = []
    find_testable_sections(hierarchy_tree, "", testable_sections)

    if not testable_sections:
        logger.warning("Could not find a suitable section with enough chunks to␣
 ↪test. Skipping content test.")
        return

    random_section = random.choice(testable_sections)
    test_toc_id = random_section['toc_id']
    section_title = random_section['path'].split(' -> ')[-1]

    logger.info(f"Selected random section for testing:␣
 ↪'{random_section['path']}' (toc_id: {test_toc_id})")

    # 2. Retrieve all documents (content + metadata) for that toc_id
    try:
        # Use .get() to retrieve full documents, not just similarity search
```

```
        retrieved_data = vector_store.get(
            where={"toc_id": test_toc_id},
            include=["metadatas", "documents"]
        )

        # Combine metadatas and documents into LangChain Document objects
        docs = [Document(page_content=doc, metadata=meta) for doc, meta in␣
↪zip(retrieved_data['documents'], retrieved_data['metadatas'])]

        logger.info(f"Retrieved {len(docs)} document chunks for toc_id␣
↪{test_toc_id}.")

        if len(docs) < 1:
            logger.warning("No chunks found in the selected section. Skipping.")
            return

        # 3. Sort the documents by chunk_id
        # Handle cases where chunk_id might be missing for robustness
        docs.sort(key=lambda d: d.metadata.get('chunk_id', -1))

        chunk_ids = [d.metadata.get('chunk_id') for d in docs]
        if None in chunk_ids:
            logger.error("TEST FAILED: Some retrieved chunks are missing a␣
↪'chunk_id'.")
            return

        # 4. Verify sequence
        is_sequential = all(chunk_ids[i] == chunk_ids[i-1] + 1 for i in␣
↪range(1, len(chunk_ids)))

        # 5. Reassemble and print content
        full_content = "\n".join([d.page_content for d in docs])

        print("\n" + "-"*25 + " CONTENT PREVIEW " + "-"*25)
        print(f"Title: {section_title} [toc_id: {test_toc_id}]")
        print(f"Chunk IDs: {chunk_ids}")
        print("-" * 70)
        print(full_content)
        print("-" * 23 + " END CONTENT PREVIEW " + "-"*23 + "\n")

        if is_sequential:
            logger.info("  TEST PASSED: Chunk IDs for the section are␣
↪sequential and content is reassembled.")
        else:
            logger.warning("TEST PASSED (with note): Chunk IDs are not␣
↪perfectly sequential but are in increasing order.")
```

```
            logger.warning("This is acceptable. Sorting by chunk_id␣
 ↪successfully restored narrative order.")

    except Exception as e:
        logger.error(f"TEST FAILED: An error occurred during chunk sequence␣
 ↪verification: {e}", exc_info=True)


# --- MAIN DIAGNOSTIC FUNCTION ---
def run_full_diagnostics():
    if not langchain_available:
        logger.error("LangChain components not installed. Skipping diagnostics.
 ↪")
        return
    if not pandas_available:
        logger.warning("Pandas not installed. Some reports may not be available.
 ↪")

    print_header("Full Database Health & Hierarchy Diagnostic Report")

    # 1. Connect to the Database
    logger.info("Connecting to the vector database...")
    if not os.path.exists(CHROMA_PERSIST_DIR):
        logger.error(f"FATAL: Chroma DB directory not found at␣
 ↪{CHROMA_PERSIST_DIR}.")
        return

    vector_store = Chroma(
        persist_directory=CHROMA_PERSIST_DIR,
        embedding_function=OllamaEmbeddings(model=EMBEDDING_MODEL_OLLAMA),
        collection_name=CHROMA_COLLECTION_NAME
    )
    logger.info("Successfully connected to the database.")

    # 2. Retrieve ALL Metadata
    total_docs = vector_store._collection.count()
    if total_docs == 0:
        logger.warning("Database is empty. No diagnostics to run.")
        return

    logger.info(f"Retrieving metadata for all {total_docs} chunks...")
    metadatas = vector_store.get(limit=total_docs,␣
 ↪include=["metadatas"])['metadatas']
    logger.info("Successfully retrieved all metadata.")

    # 3. Reconstruct the Hierarchy Tree
```

```
    logger.info("Reconstructing hierarchy from chunk metadata...")
    hierarchy_tree = {'_children': {}}
    chunks_without_id = 0

    for meta in metadatas:
        toc_id = meta.get('toc_id')
        if toc_id is None or toc_id == -1:
            chunks_without_id += 1
            node_title = meta.get('level_1_title', 'Orphaned Chunks')
            if node_title not in hierarchy_tree['_children']:
                hierarchy_tree['_children'][node_title] = {'_children': {},␣
↪'_chunks': 0, '_toc_id': float('inf')}
            hierarchy_tree['_children'][node_title]['_chunks'] += 1
            continue

        current_node = hierarchy_tree
        for level in range(1, 7):
            level_key = f'level_{level}_title'
            title = meta.get(level_key)
            if not title: break
            if title not in current_node['_children']:
                current_node['_children'][title] = {'_children': {}, '_chunks':␣
↪0, '_toc_id': float('inf')}
            current_node = current_node['_children'][title]

        current_node['_chunks'] += 1
        current_node['_toc_id'] = min(current_node['_toc_id'], toc_id)

    logger.info("Hierarchy reconstruction complete.")

    # 4. Print Hierarchy Report
    print_header("Reconstructed Hierarchy Report (Book Order)", char="-")
    print_hierarchy_report(hierarchy_tree)

    # 5. Run Chunk Sequence and Content Test
    verify_chunk_sequence_and_content(vector_store, hierarchy_tree)

    # 6. Final Summary
    print_header("Diagnostic Summary", char="-")
    print(f"Total Chunks in DB: {total_docs}")

    if chunks_without_id > 0:
        logger.warning(f"Found {chunks_without_id} chunks MISSING a valid␣
↪'toc_id'. Check 'Orphaned' sections.")
    else:
        logger.info("All chunks contain valid 'toc_id' metadata. Sequential␣
↪integrity is maintained.")
```

```
    print_header("Diagnostic Complete")

# --- Execute Diagnostics ---
if 'CHROMA_PERSIST_DIR' in locals() and langchain_available:
    run_full_diagnostics()
else:
    logger.error("Skipping diagnostics: Global variables not defined or␣
 ↪LangChain not available.")
```

```
2025-07-18 21:48:01,796 - INFO - Connecting to the vector database…
2025-07-18 21:48:01,810 - INFO - Successfully connected to the database.
2025-07-18 21:48:01,811 - INFO - Retrieving metadata for all 11774 chunks…


================================================================================
               Full Database Health & Hierarchy Diagnostic Report
================================================================================

2025-07-18 21:48:02,247 - INFO - Successfully retrieved all metadata.
2025-07-18 21:48:02,248 - INFO - Reconstructing hierarchy from chunk metadata…
2025-07-18 21:48:02,259 - INFO - Hierarchy reconstruction complete.
2025-07-18 21:48:02,262 - INFO - Verifying chunk order and reassembling content
for a random ToC section.
2025-07-18 21:48:02,262 - INFO - Selected random section for testing: 'Chapter
4. Processing Crime and Incident Scenes -> Processing Law Enforcement Crime
Scenes -> Understanding Concepts and Terms Used in Warrants' (toc_id: 149)
2025-07-18 21:48:02,266 - INFO - Retrieved 18 document chunks for toc_id 149.
2025-07-18 21:48:02,266 - INFO -   TEST PASSED: Chunk IDs for the section are
sequential and content is reassembled.
2025-07-18 21:48:02,267 - WARNING - Found 21 chunks MISSING a valid 'toc_id'.
Check 'Orphaned' sections.


--------------------------------------------------------------------------------
                   Reconstructed Hierarchy Report (Book Order)
--------------------------------------------------------------------------------
|-- Preface [ID: 3] (Total Chuck in branch: 10, Direct Chunk: 10)
|-- Introduction [ID: 4] (Total Chuck in branch: 73, Direct Chunk: 73)
|-- About the Authors [ID: 5] (Total Chuck in branch: 5, Direct Chunk: 5)
|-- Acknowledgments [ID: 6] (Total Chuck in branch: 20, Direct Chunk: 20)
|-- Chapter 1. Understanding the Digital Forensics Profession and Investigations
[ID: 7] (Total Chuck in branch: 4566, Direct Chunk: 23)
  |-- An Overview of Digital Forensics [ID: 9] (Total Chuck in branch: 60,
Direct Chunk: 18)
    |-- Digital Forensics and Other Related Disciplines [ID: 10] (Total Chuck in
branch: 18, Direct Chunk: 18)
    |-- A Brief History of Digital Forensics [ID: 11] (Total Chuck in branch:
13, Direct Chunk: 13)
```

|-- Understanding Case Law [ID: 12] (Total Chuck in branch: 3, Direct Chunk: 3)
|-- Developing Digital Forensics Resources [ID: 13] (Total Chuck in branch: 8, Direct Chunk: 8)
|-- Preparing for Digital Investigations [ID: 14] (Total Chuck in branch: 84, Direct Chunk: 5)
|-- Understanding Law Enforcement Agency Investigations [ID: 15] (Total Chuck in branch: 10, Direct Chunk: 10)
|-- Following Legal Processes [ID: 16] (Total Chuck in branch: 13, Direct Chunk: 13)
|-- Understanding Private-Sector Investigations [ID: 17] (Total Chuck in branch: 56, Direct Chunk: 3)
|-- Establishing Company Policies [ID: 18] (Total Chuck in branch: 5, Direct Chunk: 5)
|-- Displaying Warning Banners [ID: 19] (Total Chuck in branch: 19, Direct Chunk: 19)
|-- Designating an Authorized Requester [ID: 20] (Total Chuck in branch: 9, Direct Chunk: 9)
|-- Conducting Security Investigations [ID: 21] (Total Chuck in branch: 15, Direct Chunk: 15)
|-- Distinguishing Personal and Company Property [ID: 22] (Total Chuck in branch: 5, Direct Chunk: 5)
|-- Maintaining Professional Conduct [ID: 23] (Total Chuck in branch: 10, Direct Chunk: 10)
|-- Preparing a Digital Forensics Investigation [ID: 24] (Total Chuck in branch: 97, Direct Chunk: 4)
|-- An Overview of a Computer Crime [ID: 25] (Total Chuck in branch: 12, Direct Chunk: 12)
|-- An Overview of a Company Policy Violation [ID: 26] (Total Chuck in branch: 4, Direct Chunk: 4)
|-- Taking a Systematic Approach [ID: 27] (Total Chuck in branch: 77, Direct Chunk: 16)
|-- Assessing the Case [ID: 28] (Total Chuck in branch: 11, Direct Chunk: 11)
|-- Planning Your Investigation [ID: 29] (Total Chuck in branch: 41, Direct Chunk: 41)
|-- Securing Your Evidence [ID: 30] (Total Chuck in branch: 9, Direct Chunk: 9)
|-- Procedures for Private-Sector High-Tech Investigations [ID: 31] (Total Chuck in branch: 124, Direct Chunk: 2)
|-- Employee Termination Cases [ID: 32] (Total Chuck in branch: 2, Direct Chunk: 2)
|-- Internet Abuse Investigations [ID: 33] (Total Chuck in branch: 19, Direct Chunk: 19)
|-- E-mail Abuse Investigations [ID: 34] (Total Chuck in branch: 16, Direct Chunk: 16)
|-- Attorney-Client Privilege Investigations [ID: 35] (Total Chuck in branch: 33, Direct Chunk: 33)

|-- Industrial Espionage Investigations [ID: 36] (Total Chuck in branch: 52, Direct Chunk: 41)
|-- Interviews and Interrogations in High-Tech Investigations [ID: 37] (Total Chuck in branch: 11, Direct Chunk: 11)
|-- Understanding Data Recovery Workstations and Software [ID: 38] (Total Chuck in branch: 37, Direct Chunk: 18)
|-- Setting Up Your Workstation for Digital Forensics [ID: 39] (Total Chuck in branch: 19, Direct Chunk: 19)
|-- Conducting an Investigation [ID: 40] (Total Chuck in branch: 109, Direct Chunk: 8)
|-- Gathering the Evidence [ID: 41] (Total Chuck in branch: 14, Direct Chunk: 14)
|-- Understanding Bit-stream Copies [ID: 42] (Total Chuck in branch: 8, Direct Chunk: 6)
|-- Acquiring an Image of Evidence Media [ID: 43] (Total Chuck in branch: 2, Direct Chunk: 2)
|-- Analyzing Your Digital Evidence [ID: 44] (Total Chuck in branch: 48, Direct Chunk: 44)
|-- Some Additional Features of Autopsy [ID: 45] (Total Chuck in branch: 4, Direct Chunk: 4)
|-- Completing the Case [ID: 46] (Total Chuck in branch: 22, Direct Chunk: 12)
|-- Autopsy's Report Generator [ID: 47] (Total Chuck in branch: 10, Direct Chunk: 10)
|-- Critiquing the Case [ID: 48] (Total Chuck in branch: 9, Direct Chunk: 9)
|-- Chapter Review [ID: inf] (Total Chuck in branch: 4022, Direct Chunk: 0)
|-- Chapter Summary [ID: 50] (Total Chuck in branch: 211, Direct Chunk: 211)
|-- Key Terms [ID: 51] (Total Chuck in branch: 309, Direct Chunk: 309)
|-- Review Questions [ID: 52] (Total Chuck in branch: 1785, Direct Chunk: 1785)
|-- Hands-On Projects [ID: 53] (Total Chuck in branch: 1527, Direct Chunk: 1527)
|-- Case Projects [ID: 54] (Total Chuck in branch: 190, Direct Chunk: 190)
|-- Chapter 2. The Investigator's Office and Laboratory [ID: 55] (Total Chuck in branch: 331, Direct Chunk: 22)
|-- Understanding Forensics Lab Accreditation Requirements [ID: 57] (Total Chuck in branch: 86, Direct Chunk: 7)
|-- Identifying Duties of the Lab Manager and Staff [ID: 58] (Total Chuck in branch: 7, Direct Chunk: 7)
|-- Lab Budget Planning [ID: 59] (Total Chuck in branch: 18, Direct Chunk: 18)
|-- Acquiring Certification and Training [ID: 60] (Total Chuck in branch: 54, Direct Chunk: 4)
|-- International Association of Computer Investigative Specialists [ID: 61] (Total Chuck in branch: 13, Direct Chunk: 13)
|-- ISC2 Certified Cyber Forensics Professional [ID: 62] (Total Chuck in branch: 2, Direct Chunk: 2)
|-- High Tech Crime Network [ID: 63] (Total Chuck in branch: 19, Direct

Chunk: 19)
      |-- EnCase Certified Examiner Certification [ID: 64] (Total Chuck in branch: 2, Direct Chunk: 2)
      |-- AccessData Certified Examiner [ID: 65] (Total Chuck in branch: 2, Direct Chunk: 2)
      |-- Other Training and Certifications [ID: 66] (Total Chuck in branch: 12, Direct Chunk: 12)
  |-- Determining the Physical Requirements for a Digital Forensics Lab [ID: 67] (Total Chuck in branch: 68, Direct Chunk: 3)
    |-- Identifying Lab Security Needs [ID: 68] (Total Chuck in branch: 9, Direct Chunk: 9)
    |-- Conducting High-Risk Investigations [ID: 69] (Total Chuck in branch: 7, Direct Chunk: 7)
    |-- Using Evidence Containers [ID: 70] (Total Chuck in branch: 24, Direct Chunk: 24)
    |-- Overseeing Facility Maintenance [ID: 71] (Total Chuck in branch: 4, Direct Chunk: 4)
    |-- Considering Physical Security Needs [ID: 72] (Total Chuck in branch: 6, Direct Chunk: 6)
    |-- Auditing a Digital Forensics Lab [ID: 73] (Total Chuck in branch: 8, Direct Chunk: 8)
    |-- Determining Floor Plans for Digital Forensics Labs [ID: 74] (Total Chuck in branch: 7, Direct Chunk: 7)
  |-- Selecting a Basic Forensic Workstation [ID: 75] (Total Chuck in branch: 51, Direct Chunk: 2)
    |-- Selecting Workstations for a Lab [ID: 76] (Total Chuck in branch: 12, Direct Chunk: 12)
    |-- Selecting Workstations for Private-Sector Labs [ID: 77] (Total Chuck in branch: 4, Direct Chunk: 4)
    |-- Stocking Hardware Peripherals [ID: 78] (Total Chuck in branch: 14, Direct Chunk: 14)
    |-- Maintaining Operating Systems and Software Inventories [ID: 79] (Total Chuck in branch: 9, Direct Chunk: 9)
    |-- Using a Disaster Recovery Plan [ID: 80] (Total Chuck in branch: 7, Direct Chunk: 7)
    |-- Planning for Equipment Upgrades [ID: 81] (Total Chuck in branch: 3, Direct Chunk: 3)
  |-- Building a Business Case for Developing a Forensics Lab [ID: 82] (Total Chuck in branch: 104, Direct Chunk: 11)
    |-- Preparing a Business Case for a Digital Forensics Lab [ID: 83] (Total Chuck in branch: 93, Direct Chunk: 2)
      |-- Justification [ID: 84] (Total Chuck in branch: 8, Direct Chunk: 8)
      |-- Budget Development [ID: 85] (Total Chuck in branch: 2, Direct Chunk: 2)
      |-- Facility Cost [ID: 86] (Total Chuck in branch: 15, Direct Chunk: 15)
      |-- Hardware Requirements [ID: 87] (Total Chuck in branch: 21, Direct Chunk: 21)
      |-- Software Requirements [ID: 88] (Total Chuck in branch: 23, Direct

Chunk: 23)
      |-- Miscellaneous Budget Needs [ID: 89] (Total Chuck in branch: 4, Direct Chunk: 4)
      |-- Approval and Acquisition [ID: 90] (Total Chuck in branch: 4, Direct Chunk: 4)
      |-- Implementation [ID: 91] (Total Chuck in branch: 2, Direct Chunk: 2)
      |-- Acceptance Testing [ID: 92] (Total Chuck in branch: 6, Direct Chunk: 6)
      |-- Correction for Acceptance [ID: 93] (Total Chuck in branch: 2, Direct Chunk: 2)
      |-- Production [ID: 94] (Total Chuck in branch: 4, Direct Chunk: 4)
|-- Chapter 3. Data Acquisition [ID: 101] (Total Chuck in branch: 390, Direct Chunk: 28)
  |-- Understanding Storage Formats for Digital Evidence [ID: 103] (Total Chuck in branch: 31, Direct Chunk: 5)
    |-- Raw Format [ID: 104] (Total Chuck in branch: 4, Direct Chunk: 4)
    |-- Proprietary Formats [ID: 105] (Total Chuck in branch: 11, Direct Chunk: 11)
    |-- Advanced Forensic Format [ID: 106] (Total Chuck in branch: 11, Direct Chunk: 11)
  |-- Determining the Best Acquisition Method [ID: 107] (Total Chuck in branch: 20, Direct Chunk: 20)
  |-- Contingency Planning for Image Acquisitions [ID: 108] (Total Chuck in branch: 10, Direct Chunk: 10)
  |-- Using Acquisition Tools [ID: 109] (Total Chuck in branch: 173, Direct Chunk: 5)
    |-- Mini-WinFE Boot CDs and USB Drives [ID: 110] (Total Chuck in branch: 9, Direct Chunk: 9)
    |-- Acquiring Data with a Linux Boot CD [ID: 111] (Total Chuck in branch: 113, Direct Chunk: 5)
      |-- Using Linux Live CD Distributions [ID: 112] (Total Chuck in branch: 17, Direct Chunk: 17)
      |-- Preparing a Target Drive for Acquisition in Linux [ID: 113] (Total Chuck in branch: 45, Direct Chunk: 45)
      |-- Acquiring Data with dd in Linux [ID: 114] (Total Chuck in branch: 32, Direct Chunk: 32)
      |-- Acquiring Data with dcfldd in Linux [ID: 115] (Total Chuck in branch: 14, Direct Chunk: 14)
    |-- Capturing an Image with AccessData FTK Imager Lite [ID: 116] (Total Chuck in branch: 46, Direct Chunk: 46)
  |-- Validating Data Acquisitions [ID: 117] (Total Chuck in branch: 32, Direct Chunk: 5)
    |-- Linux Validation Methods [ID: 118] (Total Chuck in branch: 21, Direct Chunk: 3)
      |-- Validating dd-Acquired Data [ID: 119] (Total Chuck in branch: 12, Direct Chunk: 12)
      |-- Validating dcfldd-Acquired Data [ID: 120] (Total Chuck in branch: 6, Direct Chunk: 6)

    |-- Getting a Detailed Description of the Location [ID: 154] (Total Chuck in branch: 7, Direct Chunk: 7)
    |-- Determining Who Is in Charge [ID: 155] (Total Chuck in branch: 2, Direct Chunk: 2)
    |-- Using Additional Technical Expertise [ID: 156] (Total Chuck in branch: 4, Direct Chunk: 4)
    |-- Determining the Tools You Need [ID: 157] (Total Chuck in branch: 11, Direct Chunk: 11)
    |-- Preparing the Investigation Team [ID: 158] (Total Chuck in branch: 3, Direct Chunk: 3)
  |-- Securing a Digital Incident or Crime Scene [ID: 159] (Total Chuck in branch: 9, Direct Chunk: 9)
  |-- Seizing Digital Evidence at the Scene [ID: 160] (Total Chuck in branch: 72, Direct Chunk: 4)
    |-- Preparing to Acquire Digital Evidence [ID: 161] (Total Chuck in branch: 8, Direct Chunk: 8)
    |-- Processing Incident or Crime Scenes [ID: 162] (Total Chuck in branch: 34, Direct Chunk: 34)
    |-- Processing Data Centers with RAID Systems [ID: 163] (Total Chuck in branch: 2, Direct Chunk: 2)
    |-- Using a Technical Advisor [ID: 164] (Total Chuck in branch: 9, Direct Chunk: 9)
    |-- Documenting Evidence in the Lab [ID: 165] (Total Chuck in branch: 4, Direct Chunk: 4)
    |-- Processing and Handling Digital Evidence [ID: 166] (Total Chuck in branch: 11, Direct Chunk: 11)
  |-- Storing Digital Evidence [ID: 167] (Total Chuck in branch: 18, Direct Chunk: 7)
    |-- Evidence Retention and Media Storage Needs [ID: 168] (Total Chuck in branch: 4, Direct Chunk: 4)
    |-- Documenting Evidence [ID: 169] (Total Chuck in branch: 7, Direct Chunk: 7)
  |-- Obtaining a Digital Hash [ID: 170] (Total Chuck in branch: 42, Direct Chunk: 42)
  |-- Reviewing a Case [ID: 171] (Total Chuck in branch: 79, Direct Chunk: 8)
    |-- Sample Civil Investigation [ID: 172] (Total Chuck in branch: 23, Direct Chunk: 23)
    |-- An Example of a Criminal Investigation [ID: 173] (Total Chuck in branch: 4, Direct Chunk: 4)
    |-- Reviewing Background Information for a Case [ID: 174] (Total Chuck in branch: 4, Direct Chunk: 4)
    |-- Planning the Investigation [ID: 175] (Total Chuck in branch: 7, Direct Chunk: 7)
    |-- Conducting the Investigation: Acquiring Evidence with OSForensics [ID: 176] (Total Chuck in branch: 33, Direct Chunk: 33)
|-- Chapter 5. Working with Windows and CLI Systems [ID: 183] (Total Chuck in branch: 471, Direct Chunk: 22)
  |-- Understanding File Systems [ID: 185] (Total Chuck in branch: 33, Direct

Chunk: 3)
    |-- Understanding the Boot Sequence [ID: 186] (Total Chuck in branch: 8, Direct Chunk: 8)
    |-- Understanding Disk Drives [ID: 187] (Total Chuck in branch: 14, Direct Chunk: 14)
    |-- Solid-State Storage Devices [ID: 188] (Total Chuck in branch: 8, Direct Chunk: 8)
  |-- Exploring Microsoft File Structures [ID: 189] (Total Chuck in branch: 71, Direct Chunk: 5)
    |-- Disk Partitions [ID: 190] (Total Chuck in branch: 39, Direct Chunk: 39)
    |-- Examining FAT Disks [ID: 191] (Total Chuck in branch: 27, Direct Chunk: 24)
      |-- Deleting FAT Files [ID: 192] (Total Chuck in branch: 3, Direct Chunk: 3)
  |-- Examining NTFS Disks [ID: 193] (Total Chuck in branch: 168, Direct Chunk: 14)
    |-- NTFS System Files [ID: 194] (Total Chuck in branch: 6, Direct Chunk: 6)
    |-- MFT and File Attributes [ID: 195] (Total Chuck in branch: 20, Direct Chunk: 20)
    |-- MFT Structures for File Data [ID: 196] (Total Chuck in branch: 69, Direct Chunk: 3)
      |-- MFT Header Fields [ID: 197] (Total Chuck in branch: 7, Direct Chunk: 7)
      |-- Attribute 0x10: Standard Information [ID: 198] (Total Chuck in branch: 8, Direct Chunk: 8)
      |-- Attribute 0x30: File Name [ID: 199] (Total Chuck in branch: 18, Direct Chunk: 18)
      |-- Attribute 0x40: Object_ID [ID: 200] (Total Chuck in branch: 10, Direct Chunk: 10)
      |-- Attribute 0x80: Data for a Resident File [ID: 201] (Total Chuck in branch: 8, Direct Chunk: 8)
      |-- Attribute 0x80: Data for a Nonresident File [ID: 202] (Total Chuck in branch: 6, Direct Chunk: 6)
      |-- Interpreting a Data Run [ID: 203] (Total Chuck in branch: 9, Direct Chunk: 9)
    |-- NTFS Alternate Data Streams [ID: 204] (Total Chuck in branch: 14, Direct Chunk: 14)
    |-- NTFS Compressed Files [ID: 205] (Total Chuck in branch: 3, Direct Chunk: 3)
    |-- NTFS Encrypting File System [ID: 206] (Total Chuck in branch: 5, Direct Chunk: 5)
    |-- EFS Recovery Key Agent [ID: 207] (Total Chuck in branch: 8, Direct Chunk: 8)
    |-- Deleting NTFS Files [ID: 208] (Total Chuck in branch: 20, Direct Chunk: 20)
    |-- Resilient File System [ID: 209] (Total Chuck in branch: 9, Direct Chunk: 9)
  |-- Understanding Whole Disk Encryption [ID: 210] (Total Chuck in branch: 26,

Direct Chunk: 11)
    |-- Examining Microsoft BitLocker [ID: 211] (Total Chuck in branch: 9, Direct Chunk: 9)
    |-- Examining Third-Party Disk Encryption Tools [ID: 212] (Total Chuck in branch: 6, Direct Chunk: 6)
  |-- Understanding the Windows Registry [ID: 213] (Total Chuck in branch: 56, Direct Chunk: 9)
    |-- Exploring the Organization of the Windows Registry [ID: 214] (Total Chuck in branch: 18, Direct Chunk: 18)
    |-- Examining the Windows Registry [ID: 215] (Total Chuck in branch: 29, Direct Chunk: 29)
  |-- Understanding Microsoft Startup Tasks [ID: 216] (Total Chuck in branch: 47, Direct Chunk: 3)
    |-- Startup in Windows 7, Windows 8, and Windows 10 [ID: 217] (Total Chuck in branch: 5, Direct Chunk: 5)
    |-- Startup in Windows NT and Later [ID: 218] (Total Chuck in branch: 39, Direct Chunk: 10)
      |-- Startup Files for Windows Vista [ID: 219] (Total Chuck in branch: 6, Direct Chunk: 6)
      |-- Startup Files for Windows XP [ID: 220] (Total Chuck in branch: 17, Direct Chunk: 17)
      |-- Windows XP System Files [ID: 221] (Total Chuck in branch: 4, Direct Chunk: 4)
      |-- Contamination Concerns with Windows XP [ID: 222] (Total Chuck in branch: 2, Direct Chunk: 2)
  |-- Understanding Virtual Machines [ID: 223] (Total Chuck in branch: 48, Direct Chunk: 10)
    |-- Creating a Virtual Machine [ID: 224] (Total Chuck in branch: 38, Direct Chunk: 38)
|-- Chapter 6. Current Digital Forensics Tools [ID: 231] (Total Chuck in branch: 315, Direct Chunk: 22)
  |-- Evaluating Digital Forensics Tool Needs [ID: 233] (Total Chuck in branch: 184, Direct Chunk: 10)
    |-- Types of Digital Forensics Tools [ID: 234] (Total Chuck in branch: 11, Direct Chunk: 4)
      |-- Hardware Forensics Tools [ID: 235] (Total Chuck in branch: 2, Direct Chunk: 2)
      |-- Software Forensics Tools [ID: 236] (Total Chuck in branch: 5, Direct Chunk: 5)
    |-- Tasks Performed by Digital Forensics Tools [ID: 237] (Total Chuck in branch: 153, Direct Chunk: 3)
      |-- Acquisition [ID: 238] (Total Chuck in branch: 22, Direct Chunk: 22)
      |-- Validation and Verification [ID: 239] (Total Chuck in branch: 14, Direct Chunk: 14)
      |-- Extraction [ID: 240] (Total Chuck in branch: 25, Direct Chunk: 25)
      |-- Reconstruction [ID: 241] (Total Chuck in branch: 66, Direct Chunk: 66)
      |-- Reporting [ID: 242] (Total Chuck in branch: 23, Direct Chunk: 23)
    |-- Tool Comparisons [ID: 243] (Total Chuck in branch: 6, Direct Chunk: 6)

|-- An Overview of Mac File Structures [ID: 277] (Total Chuck in branch: 23, Direct Chunk: 23)
|-- Forensics Procedures in Mac [ID: 278] (Total Chuck in branch: 29, Direct Chunk: 18)
|-- Acquisition Methods in macOS [ID: 279] (Total Chuck in branch: 11, Direct Chunk: 11)
|-- Using Linux Forensics Tools [ID: 280] (Total Chuck in branch: 68, Direct Chunk: 5)
|-- Installing Sleuth Kit and Autopsy [ID: 281] (Total Chuck in branch: 21, Direct Chunk: 21)
|-- Examining a Case with Sleuth Kit and Autopsy [ID: 282] (Total Chuck in branch: 42, Direct Chunk: 42)
|-- Chapter 8. Recovering Graphics Files [ID: 289] (Total Chuck in branch: 240, Direct Chunk: 19)
|-- Recognizing a Graphics File [ID: 291] (Total Chuck in branch: 54, Direct Chunk: 4)
|-- Understanding Bitmap and Raster Images [ID: 292] (Total Chuck in branch: 13, Direct Chunk: 13)
|-- Understanding Vector Graphics [ID: 293] (Total Chuck in branch: 2, Direct Chunk: 2)
|-- Understanding Metafile Graphics [ID: 294] (Total Chuck in branch: 2, Direct Chunk: 2)
|-- Understanding Graphics File Formats [ID: 295] (Total Chuck in branch: 14, Direct Chunk: 14)
|-- Understanding Digital Photograph File Formats [ID: 296] (Total Chuck in branch: 19, Direct Chunk: 2)
|-- Examining the Raw File Format [ID: 297] (Total Chuck in branch: 5, Direct Chunk: 5)
|-- Examining the Exchangeable Image File Format [ID: 298] (Total Chuck in branch: 12, Direct Chunk: 12)
|-- Understanding Data Compression [ID: 299] (Total Chuck in branch: 101, Direct Chunk: 2)
|-- Lossless and Lossy Compression [ID: 300] (Total Chuck in branch: 8, Direct Chunk: 8)
|-- Locating and Recovering Graphics Files [ID: 301] (Total Chuck in branch: 6, Direct Chunk: 6)
|-- Identifying Graphics File Fragments [ID: 302] (Total Chuck in branch: 3, Direct Chunk: 3)
|-- Repairing Damaged Headers [ID: 303] (Total Chuck in branch: 6, Direct Chunk: 6)
|-- Searching for and Carving Data from Unallocated Space [ID: 304] (Total Chuck in branch: 39, Direct Chunk: 9)
|-- Planning Your Examination [ID: 305] (Total Chuck in branch: 4, Direct Chunk: 4)
|-- Searching for and Recovering Digital Photograph Evidence [ID: 306] (Total Chuck in branch: 26, Direct Chunk: 26)
|-- Rebuilding File Headers [ID: 307] (Total Chuck in branch: 22, Direct Chunk: 22)

|-- Reconstructing File Fragments [ID: 308] (Total Chuck in branch: 15, Direct Chunk: 15)
|-- Identifying Unknown File Formats [ID: 309] (Total Chuck in branch: 47, Direct Chunk: 14)
|-- Analyzing Graphics File Headers [ID: 310] (Total Chuck in branch: 5, Direct Chunk: 5)
|-- Tools for Viewing Images [ID: 311] (Total Chuck in branch: 5, Direct Chunk: 5)
|-- Understanding Steganography in Graphics Files [ID: 312] (Total Chuck in branch: 16, Direct Chunk: 16)
|-- Using Steganalysis Tools [ID: 313] (Total Chuck in branch: 7, Direct Chunk: 7)
|-- Understanding Copyright Issues with Graphics [ID: 314] (Total Chuck in branch: 19, Direct Chunk: 19)
|-- Chapter 9. Digital Forensics Analysis and Validation [ID: 321] (Total Chuck in branch: 248, Direct Chunk: 16)
|-- Determining What Data to Collect and Analyze [ID: 323] (Total Chuck in branch: 99, Direct Chunk: 6)
|-- Approaching Digital Forensics Cases [ID: 324] (Total Chuck in branch: 35, Direct Chunk: 30)
|-- Refining and Modifying the Investigation Plan [ID: 325] (Total Chuck in branch: 5, Direct Chunk: 5)
|-- Using Autopsy to Validate Data [ID: 326] (Total Chuck in branch: 23, Direct Chunk: 8)
|-- Installing NSRL Hashes in Autopsy [ID: 327] (Total Chuck in branch: 15, Direct Chunk: 15)
|-- Collecting Hash Values in Autopsy [ID: 328] (Total Chuck in branch: 35, Direct Chunk: 35)
|-- Validating Forensic Data [ID: 329] (Total Chuck in branch: 51, Direct Chunk: 3)
|-- Validating with Hexadecimal Editors [ID: 330] (Total Chuck in branch: 31, Direct Chunk: 28)
|-- Using Hash Values to Discriminate Data [ID: 331] (Total Chuck in branch: 3, Direct Chunk: 3)
|-- Validating with Digital Forensics Tools [ID: 332] (Total Chuck in branch: 17, Direct Chunk: 17)
|-- Addressing Data-Hiding Techniques [ID: 333] (Total Chuck in branch: 82, Direct Chunk: 2)
|-- Hiding Files by Using the OS [ID: 334] (Total Chuck in branch: 3, Direct Chunk: 3)
|-- Hiding Partitions [ID: 335] (Total Chuck in branch: 8, Direct Chunk: 8)
|-- Marking Bad Clusters [ID: 336] (Total Chuck in branch: 6, Direct Chunk: 6)
|-- Bit-Shifting [ID: 337] (Total Chuck in branch: 34, Direct Chunk: 34)
|-- Understanding Steganalysis Methods [ID: 338] (Total Chuck in branch: 10, Direct Chunk: 10)
|-- Examining Encrypted Files [ID: 339] (Total Chuck in branch: 4, Direct Chunk: 4)

|-- Recovering Passwords [ID: 340] (Total Chuck in branch: 15, Direct Chunk: 15)
|-- Chapter 10. Virtual Machine Forensics, Live Acquisitions, and Network Forensics [ID: 347] (Total Chuck in branch: 287, Direct Chunk: 17)
  |-- An Overview of Virtual Machine Forensics [ID: 349] (Total Chuck in branch: 176, Direct Chunk: 7)
    |-- Type 2 Hypervisors [ID: 350] (Total Chuck in branch: 32, Direct Chunk: 6)
      |-- Parallels Desktop [ID: 351] (Total Chuck in branch: 2, Direct Chunk: 2)
      |-- KVM [ID: 352] (Total Chuck in branch: 2, Direct Chunk: 2)
      |-- Microsoft Hyper-V [ID: 353] (Total Chuck in branch: 2, Direct Chunk: 2)
      |-- VMware Workstation and Workstation Player [ID: 354] (Total Chuck in branch: 14, Direct Chunk: 14)
      |-- VirtualBox [ID: 355] (Total Chuck in branch: 6, Direct Chunk: 6)
    |-- Conducting an Investigation with Type 2 Hypervisors [ID: 356] (Total Chuck in branch: 103, Direct Chunk: 70)
      |-- Other VM Examination Methods [ID: 357] (Total Chuck in branch: 13, Direct Chunk: 13)
      |-- Using VMs as Forensics Tools [ID: 358] (Total Chuck in branch: 20, Direct Chunk: 20)
    |-- Working with Type 1 Hypervisors [ID: 359] (Total Chuck in branch: 34, Direct Chunk: 34)
  |-- Performing Live Acquisitions [ID: 360] (Total Chuck in branch: 18, Direct Chunk: 15)
    |-- Performing a Live Acquisition in Windows [ID: 361] (Total Chuck in branch: 3, Direct Chunk: 3)
  |-- Network Forensics Overview [ID: 362] (Total Chuck in branch: 76, Direct Chunk: 4)
    |-- The Need for Established Procedures [ID: 363] (Total Chuck in branch: 4, Direct Chunk: 4)
    |-- Securing a Network [ID: 364] (Total Chuck in branch: 13, Direct Chunk: 13)
    |-- Developing Procedures for Network Forensics [ID: 365] (Total Chuck in branch: 41, Direct Chunk: 8)
      |-- Reviewing Network Logs [ID: 366] (Total Chuck in branch: 11, Direct Chunk: 11)
      |-- Using Network Tools [ID: 367] (Total Chuck in branch: 4, Direct Chunk: 4)
      |-- Using Packet Analyzers [ID: 368] (Total Chuck in branch: 18, Direct Chunk: 18)
    |-- Investigating Virtual Networks [ID: 369] (Total Chuck in branch: 7, Direct Chunk: 7)
    |-- Examining the Honeynet Project [ID: 370] (Total Chuck in branch: 7, Direct Chunk: 7)
|-- Chapter 11. E-mail and Social Media Investigations [ID: 377] (Total Chuck in branch: 302, Direct Chunk: 20)

|-- Exploring the Role of E-mail in Investigations [ID: 379] (Total Chuck in branch: 11, Direct Chunk: 11)
|-- Exploring the Roles of the Client and Server in E-mail [ID: 380] (Total Chuck in branch: 10, Direct Chunk: 10)
|-- Investigating E-mail Crimes and Violations [ID: 381] (Total Chuck in branch: 101, Direct Chunk: 4)
|-- Understanding Forensic Linguistics [ID: 382] (Total Chuck in branch: 6, Direct Chunk: 6)
|-- Examining E-mail Messages [ID: 383] (Total Chuck in branch: 28, Direct Chunk: 8)
|-- Copying an E-mail Message [ID: 384] (Total Chuck in branch: 20, Direct Chunk: 20)
|-- Viewing E-mail Headers [ID: 385] (Total Chuck in branch: 33, Direct Chunk: 33)
|-- Examining E-mail Headers [ID: 386] (Total Chuck in branch: 14, Direct Chunk: 14)
|-- Examining Additional E-mail Files [ID: 387] (Total Chuck in branch: 6, Direct Chunk: 6)
|-- Tracing an E-mail Message [ID: 388] (Total Chuck in branch: 7, Direct Chunk: 7)
|-- Using Network E-mail Logs [ID: 389] (Total Chuck in branch: 3, Direct Chunk: 3)
|-- Understanding E-mail Servers [ID: 390] (Total Chuck in branch: 33, Direct Chunk: 13)
|-- Examining UNIX E-mail Server Logs [ID: 391] (Total Chuck in branch: 12, Direct Chunk: 12)
|-- Examining Microsoft E-mail Server Logs [ID: 392] (Total Chuck in branch: 8, Direct Chunk: 8)
|-- Using Specialized E-mail Forensics Tools [ID: 393] (Total Chuck in branch: 91, Direct Chunk: 22)
|-- Using Magnet AXIOM to Recover E-mail [ID: 394] (Total Chuck in branch: 14, Direct Chunk: 14)
|-- Using a Hex Editor to Carve E-mail Messages [ID: 395] (Total Chuck in branch: 41, Direct Chunk: 41)
|-- Recovering Outlook Files [ID: 396] (Total Chuck in branch: 7, Direct Chunk: 7)
|-- E-mail Case Studies [ID: 397] (Total Chuck in branch: 7, Direct Chunk: 7)
|-- Applying Digital Forensics Methods to Social Media Communications [ID: 398] (Total Chuck in branch: 36, Direct Chunk: 23)
|-- Forensics Tools for Social Media Investigations [ID: 399] (Total Chuck in branch: 13, Direct Chunk: 13)
|-- Chapter 12. Mobile Device Forensics and the Internet of Anything [ID: 406] (Total Chuck in branch: 169, Direct Chunk: 8)
|-- Understanding Mobile Device Forensics [ID: 408] (Total Chuck in branch: 65, Direct Chunk: 19)
|-- Mobile Phone Basics [ID: 409] (Total Chuck in branch: 24, Direct Chunk: 24)

|-- Inside Mobile Devices [ID: 410] (Total Chuck in branch: 22, Direct Chunk: 11)
|-- SIM Cards [ID: 411] (Total Chuck in branch: 11, Direct Chunk: 11)
|-- Understanding Acquisition Procedures for Mobile Devices [ID: 412] (Total Chuck in branch: 75, Direct Chunk: 30)
|-- Mobile Forensics Equipment [ID: 413] (Total Chuck in branch: 30, Direct Chunk: 6)
|-- SIM Card Readers [ID: 414] (Total Chuck in branch: 7, Direct Chunk: 7)
|-- Mobile Phone Forensics Tools and Methods [ID: 415] (Total Chuck in branch: 17, Direct Chunk: 17)
|-- Using Mobile Forensics Tools [ID: 416] (Total Chuck in branch: 15, Direct Chunk: 15)
|-- Understanding Forensics in the Internet of Anything [ID: 417] (Total Chuck in branch: 21, Direct Chunk: 21)
|-- Chapter 13. Cloud Forensics [ID: 424] (Total Chuck in branch: 290, Direct Chunk: 20)
|-- An Overview of Cloud Computing [ID: 426] (Total Chuck in branch: 42, Direct Chunk: 2)
|-- History of the Cloud [ID: 427] (Total Chuck in branch: 4, Direct Chunk: 4)
|-- Cloud Service Levels and Deployment Methods [ID: 428] (Total Chuck in branch: 13, Direct Chunk: 13)
|-- Cloud Vendors [ID: 429] (Total Chuck in branch: 15, Direct Chunk: 15)
|-- Basic Concepts of Cloud Forensics [ID: 430] (Total Chuck in branch: 8, Direct Chunk: 8)
|-- Legal Challenges in Cloud Forensics [ID: 431] (Total Chuck in branch: 56, Direct Chunk: 2)
|-- Service Level Agreements [ID: 432] (Total Chuck in branch: 25, Direct Chunk: 17)
|-- Policies, Standards, and Guidelines for CSPs [ID: 433] (Total Chuck in branch: 5, Direct Chunk: 5)
|-- CSP Processes and Procedures [ID: 434] (Total Chuck in branch: 3, Direct Chunk: 3)
|-- Jurisdiction Issues [ID: 435] (Total Chuck in branch: 6, Direct Chunk: 6)
|-- Accessing Evidence in the Cloud [ID: 436] (Total Chuck in branch: 23, Direct Chunk: 3)
|-- Search Warrants [ID: 437] (Total Chuck in branch: 10, Direct Chunk: 10)
|-- Subpoenas and Court Orders [ID: 438] (Total Chuck in branch: 10, Direct Chunk: 10)
|-- Technical Challenges in Cloud Forensics [ID: 439] (Total Chuck in branch: 33, Direct Chunk: 5)
|-- Architecture [ID: 440] (Total Chuck in branch: 12, Direct Chunk: 12)
|-- Analysis of Cloud Forensic Data [ID: 441] (Total Chuck in branch: 2, Direct Chunk: 2)
|-- Anti-Forensics [ID: 442] (Total Chuck in branch: 3, Direct Chunk: 3)
|-- Incident First Responders [ID: 443] (Total Chuck in branch: 5, Direct

Chunk: 5)
    |-- Role Management [ID: 444] (Total Chuck in branch: 2, Direct Chunk: 2)
    |-- Standards and Training [ID: 445] (Total Chuck in branch: 4, Direct Chunk: 4)
  |-- Acquisitions in the Cloud [ID: 446] (Total Chuck in branch: 21, Direct Chunk: 8)
    |-- Encryption in the Cloud [ID: 447] (Total Chuck in branch: 13, Direct Chunk: 13)
  |-- Conducting a Cloud Investigation [ID: 448] (Total Chuck in branch: 105, Direct Chunk: 2)
    |-- Investigating CSPs [ID: 449] (Total Chuck in branch: 8, Direct Chunk: 8)
    |-- Investigating Cloud Customers [ID: 450] (Total Chuck in branch: 3, Direct Chunk: 3)
    |-- Understanding Prefetch Files [ID: 451] (Total Chuck in branch: 3, Direct Chunk: 3)
    |-- Examining Stored Cloud Data on a PC [ID: 452] (Total Chuck in branch: 63, Direct Chunk: 5)
      |-- Dropbox [ID: 453] (Total Chuck in branch: 8, Direct Chunk: 8)
      |-- Google Drive [ID: 454] (Total Chuck in branch: 21, Direct Chunk: 21)
      |-- OneDrive [ID: 455] (Total Chuck in branch: 29, Direct Chunk: 29)
    |-- Windows Prefetch Artifacts [ID: 456] (Total Chuck in branch: 26, Direct Chunk: 26)
  |-- Tools for Cloud Forensics [ID: 457] (Total Chuck in branch: 13, Direct Chunk: 5)
    |-- Forensic Open-Stack Tools [ID: 458] (Total Chuck in branch: 4, Direct Chunk: 4)
    |-- F-Response for the Cloud [ID: 459] (Total Chuck in branch: 2, Direct Chunk: 2)
    |-- Magnet AXIOM Cloud [ID: 460] (Total Chuck in branch: 2, Direct Chunk: 2)
|-- Chapter 14. Report Writing for High-Tech Investigations [ID: 467] (Total Chuck in branch: 263, Direct Chunk: 16)
  |-- Understanding the Importance of Reports [ID: 469] (Total Chuck in branch: 31, Direct Chunk: 19)
    |-- Limiting a Report to Specifics [ID: 470] (Total Chuck in branch: 3, Direct Chunk: 3)
    |-- Types of Reports [ID: 471] (Total Chuck in branch: 9, Direct Chunk: 9)
  |-- Guidelines for Writing Reports [ID: 472] (Total Chuck in branch: 138, Direct Chunk: 19)
    |-- What to Include in Written Preliminary Reports [ID: 473] (Total Chuck in branch: 9, Direct Chunk: 9)
    |-- Report Structure [ID: 474] (Total Chuck in branch: 8, Direct Chunk: 8)
    |-- Writing Reports Clearly [ID: 475] (Total Chuck in branch: 17, Direct Chunk: 8)
      |-- Considering Writing Style [ID: 476] (Total Chuck in branch: 6, Direct Chunk: 6)
      |-- Including Signposts [ID: 477] (Total Chuck in branch: 3, Direct Chunk: 3)
    |-- Designing the Layout and Presentation of Reports [ID: 478] (Total Chuck

in branch: 85, Direct Chunk: 44)
      |-- Providing Supporting Material [ID: 479] (Total Chuck in branch: 3, Direct Chunk: 3)
      |-- Formatting Consistently [ID: 480] (Total Chuck in branch: 2, Direct Chunk: 2)
      |-- Explaining Examination and Data Collection Methods [ID: 481] (Total Chuck in branch: 3, Direct Chunk: 3)
      |-- Including Calculations [ID: 482] (Total Chuck in branch: 2, Direct Chunk: 2)
      |-- Providing for Uncertainty and Error Analysis [ID: 483] (Total Chuck in branch: 2, Direct Chunk: 2)
      |-- Explaining Results and Conclusions [ID: 484] (Total Chuck in branch: 3, Direct Chunk: 3)
      |-- Providing References [ID: 485] (Total Chuck in branch: 20, Direct Chunk: 20)
      |-- Including Appendixes [ID: 486] (Total Chuck in branch: 6, Direct Chunk: 6)
  |-- Generating Report Findings with Forensics Software Tools [ID: 487] (Total Chuck in branch: 78, Direct Chunk: 2)
    |-- Using Autopsy to Generate Reports [ID: 488] (Total Chuck in branch: 76, Direct Chunk: 76)
|-- Chapter 15. Expert Testimony in Digital Investigations [ID: 495] (Total Chuck in branch: 329, Direct Chunk: 14)
  |-- Preparing for Testimony [ID: 497] (Total Chuck in branch: 62, Direct Chunk: 15)
    |-- Documenting and Preparing Evidence [ID: 498] (Total Chuck in branch: 12, Direct Chunk: 12)
    |-- Reviewing Your Role as a Consulting Expert or an Expert Witness [ID: 499] (Total Chuck in branch: 9, Direct Chunk: 9)
    |-- Creating and Maintaining Your CV [ID: 500] (Total Chuck in branch: 8, Direct Chunk: 8)
    |-- Preparing Technical Definitions [ID: 501] (Total Chuck in branch: 12, Direct Chunk: 12)
    |-- Preparing to Deal with the News Media [ID: 502] (Total Chuck in branch: 6, Direct Chunk: 6)
  |-- Testifying in Court [ID: 503] (Total Chuck in branch: 155, Direct Chunk: 2)
    |-- Understanding the Trial Process [ID: 504] (Total Chuck in branch: 10, Direct Chunk: 10)
    |-- Providing Qualifications for Your Testimony [ID: 505] (Total Chuck in branch: 59, Direct Chunk: 59)
    |-- General Guidelines on Testifying [ID: 506] (Total Chuck in branch: 47, Direct Chunk: 27)
      |-- Using Graphics During Testimony [ID: 507] (Total Chuck in branch: 10, Direct Chunk: 10)
      |-- Avoiding Testimony Problems [ID: 508] (Total Chuck in branch: 7, Direct Chunk: 7)
      |-- Understanding Prosecutorial Misconduct [ID: 509] (Total Chuck in

branch: 3, Direct Chunk: 3)
    |-- Testifying During Direct Examination [ID: 510] (Total Chuck in branch: 12, Direct Chunk: 12)
    |-- Testifying During Cross-Examination [ID: 511] (Total Chuck in branch: 25, Direct Chunk: 25)
  |-- Preparing for a Deposition or Hearing [ID: 512] (Total Chuck in branch: 33, Direct Chunk: 6)
    |-- Guidelines for Testifying at Depositions [ID: 513] (Total Chuck in branch: 19, Direct Chunk: 10)
      |-- Recognizing Deposition Problems [ID: 514] (Total Chuck in branch: 9, Direct Chunk: 9)
    |-- Guidelines for Testifying at Hearings [ID: 515] (Total Chuck in branch: 8, Direct Chunk: 8)
  |-- Preparing Forensics Evidence for Testimony [ID: 516] (Total Chuck in branch: 65, Direct Chunk: 34)
    |-- Preparing a Defense of Your Evidence-Collection Methods [ID: 517] (Total Chuck in branch: 31, Direct Chunk: 31)
|-- Chapter 16. Ethics for the Expert Witness [ID: 524] (Total Chuck in branch: 310, Direct Chunk: 13)
  |-- Applying Ethics and Codes to Expert Witnesses [ID: 526] (Total Chuck in branch: 58, Direct Chunk: 11)
    |-- Forensics Examiners' Roles in Testifying [ID: 527] (Total Chuck in branch: 5, Direct Chunk: 5)
    |-- Considerations in Disqualification [ID: 528] (Total Chuck in branch: 22, Direct Chunk: 22)
    |-- Traps for Unwary Experts [ID: 529] (Total Chuck in branch: 16, Direct Chunk: 16)
    |-- Determining Admissibility of Evidence [ID: 530] (Total Chuck in branch: 4, Direct Chunk: 4)
  |-- Organizations with Codes of Ethics [ID: 531] (Total Chuck in branch: 26, Direct Chunk: 2)
    |-- International Society of Forensic Computer Examiners [ID: 532] (Total Chuck in branch: 11, Direct Chunk: 11)
    |-- International High Technology Crime Investigation Association [ID: 533] (Total Chuck in branch: 6, Direct Chunk: 6)
    |-- American Bar Association [ID: 535] (Total Chuck in branch: 5, Direct Chunk: 5)
    |-- American Psychological Association [ID: 536] (Total Chuck in branch: 2, Direct Chunk: 2)
  |-- Ethical Difficulties in Expert Testimony [ID: 537] (Total Chuck in branch: 19, Direct Chunk: 6)
    |-- Ethical Responsibilities Owed to You [ID: 538] (Total Chuck in branch: 9, Direct Chunk: 9)
    |-- Standard Forensics Tools and Tools You Create [ID: 539] (Total Chuck in branch: 4, Direct Chunk: 4)
  |-- An Ethics Exercise [ID: 540] (Total Chuck in branch: 194, Direct Chunk: 4)
    |-- Performing a Cursory Exam of a Forensic Image [ID: 541] (Total Chuck in branch: 18, Direct Chunk: 18)

|-- Performing a Detailed Exam of a Forensic Image [ID: 542] (Total Chuck in branch: 33, Direct Chunk: 33)
|-- Performing the Exam [ID: 543] (Total Chuck in branch: 76, Direct Chunk: 2)
|-- Preparing for an Examination [ID: 544] (Total Chuck in branch: 74, Direct Chunk: 74)
|-- Interpreting Attribute 0x80 Data Runs [ID: 545] (Total Chuck in branch: 44, Direct Chunk: 2)
|-- Finding Attribute 0x80 an MFT Record [ID: 546] (Total Chuck in branch: 21, Direct Chunk: 21)
|-- Configuring Data Interpreter Options in WinHex [ID: 547] (Total Chuck in branch: 6, Direct Chunk: 6)
|-- Calculating Data Runs [ID: 548] (Total Chuck in branch: 15, Direct Chunk: 15)
|-- Carving Data Run Clusters Manually [ID: 549] (Total Chuck in branch: 19, Direct Chunk: 19)
|-- Lab Manual for Guide to Computer Forensics and Investigations [ID: 556] (Total Chuck in branch: 2165, Direct Chunk: 1)
|-- Chapter 12. Mobile Device Forensics [ID: 792] (Total Chuck in branch: 13, Direct Chunk: 10)
|-- Lab 12.1. Examining Cell Phone Storage Devices [ID: 794] (Total Chuck in branch: 1, Direct Chunk: 1)
|-- Lab 12.2. Using FTK Imager Lite to View Text Messages, Phone Numbers, and Photos [ID: 799] (Total Chuck in branch: 1, Direct Chunk: 1)
|-- Lab 12.3. Using Autopsy to Search Cloud Backups of Mobile Devices [ID: 804] (Total Chuck in branch: 1, Direct Chunk: 1)
|-- Chapter 1. Understanding the Digital Forensics Profession and Investigations [ID: inf] (Total Chuck in branch: 1673, Direct Chunk: 0)
|-- Lab 1.1. Installing Autopsy for Windows [ID: 560] (Total Chuck in branch: 1670, Direct Chunk: 1)
|-- Objectives [ID: 561] (Total Chuck in branch: 702, Direct Chunk: 345)
|-- Materials Required [ID: 562] (Total Chuck in branch: 357, Direct Chunk: 357)
|-- Activity [ID: 563] (Total Chuck in branch: 967, Direct Chunk: 967)
|-- Lab 1.2. Downloading FTK Imager Lite [ID: 565] (Total Chuck in branch: 1, Direct Chunk: 1)
|-- Lab 1.3. Downloading WinHex [ID: 570] (Total Chuck in branch: 1, Direct Chunk: 1)
|-- Lab 1.4. Using Autopsy for Windows [ID: 575] (Total Chuck in branch: 1, Direct Chunk: 1)
|-- Chapter 2. The Investigator's Office and Laboratory [ID: inf] (Total Chuck in branch: 5, Direct Chunk: 0)
|-- Lab 2.1. Wiping a USB Drive Securely [ID: 582] (Total Chuck in branch: 1, Direct Chunk: 1)
|-- Lab 2.2. Using Directory Snoop to Image a USB Drive [ID: 587] (Total Chuck in branch: 1, Direct Chunk: 1)
|-- Lab 2.3. Converting a Raw Image to an .E01 Image [ID: 592] (Total Chuck in branch: 1, Direct Chunk: 1)

```
    |-- Lab 6.1. Using Autopsy 4.7.0 to Search an Image File [ID: 680] (Total
Chuck in branch: 41, Direct Chunk: 1)
      |-- Activity [ID: inf] (Total Chuck in branch: 40, Direct Chunk: 0)
        |-- Installing Autopsy 4.7.0 [ID: 684] (Total Chuck in branch: 12,
Direct Chunk: 12)
        |-- Searching E-mail in Autopsy 4.7.0 [ID: 685] (Total Chuck in branch:
28, Direct Chunk: 28)
    |-- Lab 6.2. Using OSForensics to Search an Image of a Hard Drive [ID: 687]
(Total Chuck in branch: 1, Direct Chunk: 1)
    |-- Lab 6.3. Examining a Corrupt Image File with FTK Imager Lite, Autopsy,
and WinHex [ID: 692] (Total Chuck in branch: 37, Direct Chunk: 1)
      |-- Activity [ID: inf] (Total Chuck in branch: 36, Direct Chunk: 0)
        |-- Testing an Image File in Autopsy 4.3.0 [ID: 696] (Total Chuck in
branch: 16, Direct Chunk: 16)
        |-- Examining Image Files in WinHex [ID: 697] (Total Chuck in branch:
20, Direct Chunk: 20)
  |-- Chapter 7. Linux and Macintosh File Systems [ID: inf] (Total Chuck in
branch: 3, Direct Chunk: 0)
    |-- Lab 7.1. Using Autopsy to Process a Mac OS X Image [ID: 701] (Total
Chuck in branch: 1, Direct Chunk: 1)
    |-- Lab 7.2. Using Autopsy to Process a Mac OS 9 Image [ID: 706] (Total
Chuck in branch: 1, Direct Chunk: 1)
    |-- Lab 7.3. Using Autopsy to Process a Linux Image [ID: 711] (Total Chuck
in branch: 1, Direct Chunk: 1)
  |-- Chapter 8. Recovering Graphics Files [ID: inf] (Total Chuck in branch: 3,
Direct Chunk: 0)
    |-- Lab 8.1. Using Autopsy to Analyze Multimedia Files [ID: 718] (Total
Chuck in branch: 1, Direct Chunk: 1)
    |-- Lab 8.2. Using OSForensics to Analyze Multimedia Files [ID: 723] (Total
Chuck in branch: 1, Direct Chunk: 1)
    |-- Lab 8.3. Using WinHex to Analyze Multimedia Files [ID: 728] (Total Chuck
in branch: 1, Direct Chunk: 1)
  |-- Chapter 9. Digital Forensics Analysis and Validation [ID: inf] (Total
Chuck in branch: 9, Direct Chunk: 0)
    |-- Lab 9.1. Using Autopsy to Search for Keywords in an Image [ID: 735]
(Total Chuck in branch: 1, Direct Chunk: 1)
    |-- Lab 9.2. Validating File Hash Values with FTK Imager Lite [ID: 740]
(Total Chuck in branch: 1, Direct Chunk: 1)
    |-- Lab 9.3. Validating File Hash Values with WinHex [ID: 745] (Total Chuck
in branch: 7, Direct Chunk: 1)
      |-- Objectives [ID: inf] (Total Chuck in branch: 6, Direct Chunk: 0)
        |-- Materials Required: [ID: 747] (Total Chuck in branch: 6, Direct
Chunk: 6)
  |-- Chapter 10. Virtual Machine Forensics, Live Acquisitions, and Network
Forensics [ID: inf] (Total Chuck in branch: 143, Direct Chunk: 0)
    |-- Lab 10.1. Analyzing a Forensic Image Hosting a Virtual Machine [ID: 752]
(Total Chuck in branch: 41, Direct Chunk: 1)
      |-- Activity [ID: inf] (Total Chuck in branch: 40, Direct Chunk: 0)
```

```
    |-- Lab 15.1. Conducting a Preliminary Investigation [ID: 845] (Total Chuck
in branch: 1, Direct Chunk: 1)
    |-- Lab 15.2. Investigating an Arsonist [ID: 850] (Total Chuck in branch: 1,
Direct Chunk: 1)
    |-- Lab 15.3. Recovering a Password from Password-Protected Files [ID: 855]
(Total Chuck in branch: 42, Direct Chunk: 1)
      |-- Activity [ID: inf] (Total Chuck in branch: 41, Direct Chunk: 0)
        |-- Verifying the Existence of a Warning Banner [ID: 859] (Total Chuck
in branch: 12, Direct Chunk: 12)
        |-- Recovering a Password from Password-Protected Files [ID: 860] (Total
Chuck in branch: 29, Direct Chunk: 29)
  |-- Chapter 16. Ethics for the Expert Witness [ID: inf] (Total Chuck in
branch: 73, Direct Chunk: 0)
    |-- Lab 16.1. Rebuilding an MFT Record from a Corrupt Image [ID: 864] (Total
Chuck in branch: 73, Direct Chunk: 1)
      |-- Activity [ID: inf] (Total Chuck in branch: 72, Direct Chunk: 0)
        |-- Creating a Duplicate Forensic Image [ID: 868] (Total Chuck in
branch: 14, Direct Chunk: 14)
        |-- Determining the Offset Byte Address of the Corrupt MFT Record [ID:
869] (Total Chuck in branch: 12, Direct Chunk: 12)
        |-- Copying the Corrected MFT Record [ID: 870] (Total Chuck in branch:
20, Direct Chunk: 20)
        |-- Extracting Additional Evidence [ID: 871] (Total Chuck in branch: 26,
Direct Chunk: 26)
|-- Appendix A. Certification Test References [ID: 873] (Total Chuck in branch:
56, Direct Chunk: 56)
|-- Appendix B. Digital Forensics References [ID: 874] (Total Chuck in branch:
109, Direct Chunk: 109)
|-- Appendix C. Digital Forensics Lab Considerations [ID: 875] (Total Chuck in
branch: 58, Direct Chunk: 58)
|-- Appendix D. Legacy File System and Forensics Tools [ID: 876] (Total Chuck in
branch: 59, Direct Chunk: 59)
|-- EPUB Preamble [ID: inf] (Total Chuck in branch: 21, Direct Chunk: 21)

================================================================================
                    Chunk Sequence & Content Integrity Test
================================================================================

------------------------- CONTENT PREVIEW -------------------------
Title: Understanding Concepts and Terms Used in Warrants [toc_id: 149]
Chunk IDs: [1967, 1968, 1969, 1970, 1971, 1972, 1973, 1974, 1975, 1976, 1977,
1978, 1979, 1980, 1981, 1982, 1983, 1984]
-------------------------------------------------------------------
Understanding Concepts and Terms Used in Warrants
You should be familiar with warrant terminology governing the type of evidence
that can be seized. Many digital investigations involve large amounts of data
you must sort through to find evidence; the Enron case, for example, involved
terabytes of information. Unrelated information (referred to as innocent
```

information Data that doesn’t contribute to evidence of a crime or violation.) is often included with the evidence you’re trying to recover. It might be personal records of innocent people or confidential business information, for example. When you find commingled evidence, judges often issue a limiting phrase Wording in a search warrant that limits the scope of a search for evidence. to the warrant, which allows the police to separate innocent information from evidence. The warrant
to the warrant, which allows the police to separate innocent information from evidence. The warrant must list which items can be seized.
When approaching or investigating a crime scene, you might find evidence related to the crime but not in the location the warrant specifies. You might also find evidence of another unrelated crime. In these situations, this evidence is subject to the plain view doctrine. The plain view doctrine When conducting a search and seizure, objects in plain view of a law enforcement officer, who has the right to be in position to have that view, are subject to seizure without a warrant and can be introduced as evidence. Applied to conducting searches of computers, the plain view doctrine’s limitations are less clear. states that objects falling in the direct sight of an officer who has the right to be in a location are subject to seizure without a warrant and can be introduced into evidence. For
be in a location are subject to seizure without a warrant and can be introduced into evidence. For the plain view doctrine to apply, three criteria must be met:
The officer is where he or she has a legal right to be.
Ordinary senses must not be enhanced by advanced technology in any way, such as with binoculars.
Any discovery must be by chance.
For the officer to seize the item, he or she must have probable cause to believe the item is evidence of a crime or is contraband. In addition, the police aren’t permitted to move objects to get a better view. In Arizona v. Hicks (480 U.S. 321, 1987), the officer was found to have acted unlawfully because he moved stereo equipment, without probable cause, to record the serial numbers. The plain view doctrine has also been expanded to include the subdoctrines of plain feel, plain smell, and plain hearing.
In Horton v. California (496 U.S. 128, 1990), the court eliminated the requirement that the discovery of evidence in plain view be inadvertent. Previously, "inadvertent discovery" was required, which led to difficulties in defining this term. The three-prong Horton test requires the following:
The officer must be lawfully present at the place where the evidence can be plainly viewed.
The officer must have a lawful right of access to the object.
The incriminating character of the object must be "immediately apparent."
Note
The plain view doctrine doesn’t extend to supporting a general exploratory search from one object to another unless something incriminating is found (Coolidge v. New Hampshire, 403 U.S. 443, 466, 1971).
However, the plain view doctrine’s applicability in the digital forensics world is being rejected. The U.S. Court of Appeals for the Ninth Circuit has directly addressed this doctrine and used it to give wide latitude to law enforcement

```
(United States v. Wong, 334 F.3d 831, 9th Cir., 2003). Other circuit courts have
been less willing to address applying the doctrine to computer searches. For
example, police investigating a case have a search warrant authorizing the
search of a computer for evidence related to illegal drug trafficking; during
the search, the examiner observes an .avi file, opens it, and sees that it's
child pornography. At that point, she must get an additional warrant or an
expansion of the existing warrant to continue the search for child pornography.
This approach is
an expansion of the existing warrant to continue the search for child
pornography. This approach is consistent with rulings in United States v. Carey
(172 F.3d 1268, 10th Cir., 1999) and United States v. Walser (275 F.3d 981, 10th
Cir., 2001). In a more recent case that went to the Ninth Circuit Court of
Appeals, the original search warrant was for 10 major league baseball players
suspected of steroid use (United States v. Comprehensive Drug Testing, 2010).
During the examination of files and e-mails, 200 more players were implicated.
Forensics investigators see many files when they're searching for evidence, so
in this case, their opinion was the data was in plain view. However, the
court disagreed. As with the example of discovering child pornography, a
separate warrant for all
court disagreed. As with the example of discovering child pornography, a
separate warrant for all other players should have been issued.
---------------------- END CONTENT PREVIEW ----------------------


--------------------------------------------------------------------------------
                              Diagnostic Summary
--------------------------------------------------------------------------------
Total Chunks in DB: 11774

================================================================================
                              Diagnostic Complete
================================================================================
```

[31]:
```python
# Cell 6: Verify Content Retrieval for a Specific toc_id with Reassembled Text

import os
import json
import logging
from langchain_chroma import Chroma
from langchain_ollama.embeddings import OllamaEmbeddings

# --- Logger Setup ---
logger = logging.getLogger(__name__)
logging.basicConfig(level=logging.INFO, format='%(asctime)s - %(levelname)s - %
 ↪(message)s')

def retrieve_and_print_chunks_for_toc_id(vector_store: Chroma, toc_id: int):
```

```
    """
    Retrieves all chunks for a specific toc_id, reconstructs the section title
    from hierarchical metadata, shows the reassembled text, and lists individual
    chunk details for verification.
    """
    try:
        # Use the 'get' method with a 'where' filter to find all chunks for the␣
↪toc_id
        results = vector_store.get(
            where={"toc_id": toc_id},
            include=["documents", "metadatas"]
        )

        if not results or not results.get('ids'):
            logger.warning(f"No chunks found in the database for toc_id =␣
↪{toc_id}")
            print("=" * 80)
            print(f"VERIFICATION FAILED: No content found for toc_id: {toc_id}")
            print("=" * 80)
            return

        documents = results['documents']
        metadatas = results['metadatas']

        # --- FIX START: Reconstruct the hierarchical section title from␣
↪metadata ---
        # We assume all chunks for the same toc_id share the same titles.
        # We will inspect the metadata of the first chunk to get the title.
        section_title = "Unknown or Uncategorized Section"
        if metadatas:
            first_meta = metadatas[0]

            # Find all 'level_X_title' keys in the metadata
            level_titles = []
            for key, value in first_meta.items():
                if key.startswith("level_") and key.endswith("_title"):
                    try:
                        # Extract the level number (e.g., 1 from␣
↪'level_1_title') for sorting
                        level_num = int(key.split('_')[1])
                        level_titles.append((level_num, value))
                    except (ValueError, IndexError):
                        # Ignore malformed keys, just in case
                        continue

            # Sort the titles by their level number (1, 2, 3...)
            level_titles.sort()
```

```
            # Join the sorted titles to create a breadcrumb-style title
            if level_titles:
                title_parts = [title for num, title in level_titles]
                section_title = " > ".join(title_parts)
        # --- FIX END ---

        # --- Print a clear header with the reconstructed section title ---
        print("=" * 80)
        print(f"VERIFYING SECTION: '{section_title}' (toc_id: {toc_id})")
        print("=" * 80)
        logger.info(f"Found {len(documents)} chunks in the database for this␣
↪section.")

        # Sort chunks by their chunk_id to ensure they are in the correct order␣
↪for reassembly
        sorted_items = sorted(zip(documents, metadatas), key=lambda item:␣
↪item[1].get('chunk_id', 0))

        # --- Reassemble and print the full text for the section ---
        all_chunk_texts = [item[0] for item in sorted_items]
        reassembled_text = "\n".join(all_chunk_texts)

        print("\n" + "#" * 28 + " Reassembled Text " + "#" * 28)
        print(reassembled_text)
        print("#" * 80)

        # --- Print individual chunk details for in-depth verification ---
        print("\n" + "-" * 24 + " Retrieved Chunk Details " + "-" * 25)
        for i, (doc, meta) in enumerate(sorted_items):
            print(f"\n[ Chunk {i+1} of {len(documents)} | chunk_id: {meta.
↪get('chunk_id', 'N/A')} ]")
            content_preview = doc.replace('\n', ' ').strip()
            print(f"  Content Preview: '{content_preview[:250]}...'")
            print(f"  Metadata: {json.dumps(meta, indent=2)}")

        print("\n" + "=" * 80)
        print(f"Verification complete for section '{section_title}'.")
        print("=" * 80)

    except Exception as e:
        logger.error(f"An error occurred during retrieval for toc_id {toc_id}:␣
↪{e}", exc_info=True)

# ==============================================================================
# EXECUTION BLOCK (No changes needed here)
# ==============================================================================
```

```
# --- IMPORTANT: Set the ID of the section you want to test here ---
# Example: ToC ID 10 might be "An Overview of Digital Forensics"
# Example: ToC ID 11 might be "Digital Forensics and Other Related Disciplines"
TOC_ID_TO_TEST = 9# Change this to an ID you know exists from your ToC


# Assume these variables are defined in a previous cell from your notebook
# CHROMA_PERSIST_DIR = "./chroma_db_with_metadata"
# EMBEDDING_MODEL_OLLAMA = "nomic-embed-text"
# CHROMA_COLLECTION_NAME = "forensics_handbook"

# Check if the database directory exists before attempting to connect
if 'CHROMA_PERSIST_DIR' in locals() and os.path.exists(CHROMA_PERSIST_DIR):
    logger.info(f"Connecting to the existing vector database at␣
 ↪'{CHROMA_PERSIST_DIR}'...")

    try:
        vector_store = Chroma(
            persist_directory=CHROMA_PERSIST_DIR,
            embedding_function=OllamaEmbeddings(model=EMBEDDING_MODEL_OLLAMA),
            collection_name=CHROMA_COLLECTION_NAME
        )

        # Run the verification function
        retrieve_and_print_chunks_for_toc_id(vector_store, TOC_ID_TO_TEST)

    except Exception as e:
        logger.error(f"Failed to initialize Chroma or run retrieval. Error:␣
 ↪{e}")
        logger.error("Please ensure your embedding model and collection names␣
 ↪are correct.")

else:
    logger.error("Database directory not found or 'CHROMA_PERSIST_DIR' variable␣
 ↪is not set.")
    logger.error("Please run the previous cell (Cell 5) to create the database␣
 ↪first.")
```

```
2025-07-18 21:48:02,283 - INFO - Connecting to the existing vector database at
'/home/sebas_dev_linux/projects/course_generator/data_output/0.
DataBase_Chroma/ICT312/ICT312_chroma_db_toc_chunks_epub'…
2025-07-18 21:48:02,297 - INFO - Found 18 chunks in the database for this
section.

================================================================================
VERIFYING SECTION: 'Chapter 1. Understanding the Digital Forensics Profession
and Investigations > An Overview of Digital Forensics' (toc_id: 9)
```

==========

############################ Reassembled Text ############################
An Overview of Digital Forensics
As the world has become more of a level playing field, with more people online who have access to the same information (Thomas L. Freidman, The World Is Flat, Farrar, Straus, and Giroux, 2005), the need to standardize digital forensics processes has become more urgent. The definition of digital forensics Applying investigative procedures for a legal purpose; involves the analysis of digital evidence as well as obtaining search warrants, maintaining a chain of custody, validating with mathematical hash functions, using validated tools, ensuring repeatability, reporting, and presenting evidence as an expert witness. has also evolved over the years from simply involving securing and analyzing digital information stored on a computer for use as evidence in civil, criminal, or administrative
digital information stored on a computer for use as evidence in civil, criminal, or administrative cases. The former director of the Defense Computer Forensics Laboratory, Ken Zatyko, wrote a treatise on the many specialties including computer forensics, network forensics, video forensics, and a host of others. He defined it as "[t]he application of computer science and investigative procedures for a legal purpose involving the analysis of digital evidence (information of probative value that is stored or transmitted in binary form) after proper search authority, chain of custody, validation with mathematics (hash function), use of validated tools, repeatability, reporting and possible expert presentation" ("Commentary: Defining Digital Forensics," Forensic Magazine, 2007).
The field of digital forensics can also encompass items such as research and incident response. With incident response, most organizations are concerned with protecting their assets and containing the situation, not necessarily prosecuting or finding the person responsible. Research in digital forensics also isn't concerned with prosecution or validity of evidence. This book is intended for digital forensics investigators and examiners at the civil, criminal, and administrative levels. Other facets of digital forensics are beyond the scope of this book. Keep in mind that depending on the jurisdiction and situation, forensic investigators and examiners might be the same or different personnel. In this book, the terms are used interchangeably.
Note
For a more in-depth discussion of what the term "digital forensics" means, see "Digital Forensic Evidence Examination" (Fred Cohen, www.fredcohen.net/Books/2013-DFE-Examination.pdf, 2012).
Many groups have tried to create digital forensics certifications that could be recognized worldwide but have failed in this attempt. However, they have created certifications for specific categories of practitioners, such as government investigators. With the ubiquitous access to mobile devices now, digital evidence is everywhere, so the need for a global standardized method is even more critical so that companies and governments can share and use digital evidence. In October 2012, an International Organization for Standardization (ISO) standard for digital forensics was ratified. This standard, ISO 27037

"Information technology - Security techniques - Guidelines for identification, collection, acquisition and preservation of digital evidence" (see www.iso.org/standard/44381.html),
acquisition and preservation of digital evidence" (see www.iso.org/standard/44381.html), defines the personnel and methods for acquiring and preserving digital evidence. To address the multinational cases that continue to emerge, agencies in every country should develop policies and procedures that meet this standard.
The Federal Rules of Evidence (FRE), signed into law in 1973, was created to ensure consistency in federal proceedings, but many states' rules map to the FRE, too. In another attempt to standardize procedures, the FBI Computer Analysis and Response Team (CART) was formed in 1984 to handle the increase in cases involving digital evidence. By the late 1990s, CART had teamed up with the Department of Defense Computer Forensics Laboratory (DCFL) for research and training. Much of the early curriculum in this field came from the DCFL. For more information on the FBI's cybercrime investigation services, see www.fbi.gov/investigate/cyber.
Files maintained on a computer are covered by different rules, depending on the nature of the documents. Many court cases in state and federal courts have developed and clarified how the rules apply to digital evidence. The Fourth Amendment The Fourth Amendment to the U.S. Constitution in the Bill of Rights dictates that the government and its agents must have probable cause for search and seizure. to the U.S. Constitution (and each state's constitution) protects everyone's right to be secure in their person, residence, and property from search and seizure. Continuing development of the jurisprudence of this amendment has played a role in determining whether the search for digital evidence has established a different precedent, so separate search warrants Legal documents that allow law
has established a different precedent, so separate search warrants Legal documents that allow law enforcement to search an office, a home, or other locale for evidence related to an alleged crime. might not be necessary. However, when preparing to search for evidence in a criminal case, many investigators still include the suspect's computer and its components in the search warrant to avoid later admissibility problems.
In an important case involving these issues, the Pennsylvania Supreme Court addressed expectations of privacy and whether evidence is admissible (see Commonwealth v. Copenhefer, 587 A.2d 1353, 526 Pa. 555 [1991]). Initial investigations by the FBI, state police, and local police resulted in discovering computer-generated notes and instructions-some of which had been deleted-that had been concealed in hiding places around Corry, Pennsylvania. The investigation also produced several possible suspects, including David Copenhefer, who owned a nearby bookstore and apparently had bad relationships with the victim and her husband. Examination of trash discarded from Copenhefer's store revealed drafts of the ransom note and directions. Subsequent search warrants resulted in seizure of evidence
of the ransom note and directions. Subsequent search warrants resulted in seizure of evidence against him. Copenhefer's computer contained several drafts and amendments of the text of phone calls to the victim and the victim's husband

the next day, the ransom note, the series of hidden notes, and a plan for the entire kidnapping scheme (Copenhefer, p. 559).
On direct appeal, the Pennsylvania Supreme Court concluded that the physical evidence, including the digital forensics evidence, was sufficient to support the bookstore owner's conviction. Copenhefer's argument was that "[E]ven though his computer was validly seized pursuant to a warrant, his attempted deletion of the documents in question created an expectation of privacy protected by the Fourth Amendment. Thus, he claims, under Katz v. United States, 389 U.S. 347, 357, 88 S.Ct. 507, 19 L.Ed.2d 576 (1967), and its progeny, Agent Johnson's retrieval of the documents, without first obtaining another search warrant, was unreasonable under the Fourth Amendment and the documents thus seized should have been suppressed" (Copenhefer, p. 561).
The Pennsylvania Supreme Court rejected this argument, stating, "A defendant's attempt to secrete evidence of a crime is not synonymous with a legally cognizable expectation of privacy. A mere hope for secrecy is not a legally protected expectation. If it were, search warrants would be required in a vast number of cases where warrants are clearly not necessary" (Copenhefer, p. 562).
Every U.S. jurisdiction has case law related to the admissibility of evidence recovered from computers and other digital devices. As you learn in this book, however, the laws on digital evidence vary between states as well as between provinces and countries.
Note
The U.S. Department of Justice offers a useful guide to search and seizure procedures for computers and computer evidence at https://pdfs.semanticscholar.org/aabb/8b0caf982a0aa211f932252af38d4c9376fb.pdf.
##

---------------------- Retrieved Chunk Details ------------------------

[Chunk 1 of 18 | chunk_id: 156]
  Content Preview: 'An Overview of Digital Forensics…'
  Metadata: {
  "toc_id": 9,
  "level_1_title": "Chapter 1. Understanding the Digital Forensics Profession and Investigations",
  "level_2_title": "An Overview of Digital Forensics",
  "chunk_id": 156,
  "source": "Bill Nelson, Amelia Phillips, Christopher Steuart - Guide to Computer Forensics and Investigations_ Processing Digital Evidence-Cengage Learning (2018).epub"
}

[Chunk 2 of 18 | chunk_id: 157]
  Content Preview: 'As the world has become more of a level playing field, with more people online who have access to the same information (Thomas L. Freidman, The World Is Flat, Farrar, Straus, and Giroux, 2005), the need to standardize digital forensics processes has …'
  Metadata: {

"chunk_id": 157,
"level_1_title": "Chapter 1. Understanding the Digital Forensics Profession and Investigations",
"source": "Bill Nelson, Amelia Phillips, Christopher Steuart - Guide to Computer Forensics and Investigations_ Processing Digital Evidence-Cengage Learning (2018).epub",
"level_2_title": "An Overview of Digital Forensics",
"toc_id": 9
}

[Chunk 3 of 18 | chunk_id: 158]
Content Preview: 'digital information stored on a computer for use as evidence in civil, criminal, or administrative cases. The former director of the Defense Computer Forensics Laboratory, Ken Zatyko, wrote a treatise on the many specialties including computer forens…'
Metadata: {
"chunk_id": 158,
"level_1_title": "Chapter 1. Understanding the Digital Forensics Profession and Investigations",
"level_2_title": "An Overview of Digital Forensics",
"toc_id": 9,
"source": "Bill Nelson, Amelia Phillips, Christopher Steuart - Guide to Computer Forensics and Investigations_ Processing Digital Evidence-Cengage Learning (2018).epub"
}

[Chunk 4 of 18 | chunk_id: 159]
Content Preview: 'The field of digital forensics can also encompass items such as research and incident response. With incident response, most organizations are concerned with protecting their assets and containing the situation, not necessarily prosecuting or finding…'
Metadata: {
"level_2_title": "An Overview of Digital Forensics",
"chunk_id": 159,
"toc_id": 9,
"source": "Bill Nelson, Amelia Phillips, Christopher Steuart - Guide to Computer Forensics and Investigations_ Processing Digital Evidence-Cengage Learning (2018).epub",
"level_1_title": "Chapter 1. Understanding the Digital Forensics Profession and Investigations"
}

[Chunk 5 of 18 | chunk_id: 160]
Content Preview: 'Note…'
Metadata: {
"toc_id": 9,
"level_1_title": "Chapter 1. Understanding the Digital Forensics Profession and Investigations",

"chunk_id": 160,
"source": "Bill Nelson, Amelia Phillips, Christopher Steuart - Guide to Computer Forensics and Investigations_ Processing Digital Evidence-Cengage Learning (2018).epub",
"level_2_title": "An Overview of Digital Forensics"
}

[Chunk 6 of 18 | chunk_id: 161]
Content Preview: 'For a more in-depth discussion of what the term "digital forensics" means, see "Digital Forensic Evidence Examination" (Fred Cohen, www.fredcohen.net/Books/2013-DFE-Examination.pdf, 2012)…'
Metadata: {
"level_2_title": "An Overview of Digital Forensics",
"chunk_id": 161,
"level_1_title": "Chapter 1. Understanding the Digital Forensics Profession and Investigations",
"toc_id": 9,
"source": "Bill Nelson, Amelia Phillips, Christopher Steuart - Guide to Computer Forensics and Investigations_ Processing Digital Evidence-Cengage Learning (2018).epub"
}

[Chunk 7 of 18 | chunk_id: 162]
Content Preview: 'Many groups have tried to create digital forensics certifications that could be recognized worldwide but have failed in this attempt. However, they have created certifications for specific categories of practitioners, such as government investigators…'
Metadata: {
"chunk_id": 162,
"toc_id": 9,
"level_1_title": "Chapter 1. Understanding the Digital Forensics Profession and Investigations",
"level_2_title": "An Overview of Digital Forensics",
"source": "Bill Nelson, Amelia Phillips, Christopher Steuart - Guide to Computer Forensics and Investigations_ Processing Digital Evidence-Cengage Learning (2018).epub"
}

[Chunk 8 of 18 | chunk_id: 163]
Content Preview: 'acquisition and preservation of digital evidence" (see www.iso.org/standard/44381.html), defines the personnel and methods for acquiring and preserving digital evidence. To address the multinational cases that continue to emerge, agencies in every co…'
Metadata: {
"chunk_id": 163,
"toc_id": 9,
"level_2_title": "An Overview of Digital Forensics",
"source": "Bill Nelson, Amelia Phillips, Christopher Steuart - Guide to

Computer Forensics and Investigations_ Processing Digital Evidence-Cengage Learning (2018).epub",
  "level_1_title": "Chapter 1. Understanding the Digital Forensics Profession and Investigations"
}

[Chunk 9 of 18 | chunk_id: 164]
  Content Preview: 'The Federal Rules of Evidence (FRE), signed into law in 1973, was created to ensure consistency in federal proceedings, but many states' rules map to the FRE, too. In another attempt to standardize procedures, the FBI Computer Analysis and Response T…'
  Metadata: {
  "source": "Bill Nelson, Amelia Phillips, Christopher Steuart - Guide to Computer Forensics and Investigations_ Processing Digital Evidence-Cengage Learning (2018).epub",
  "chunk_id": 164,
  "level_2_title": "An Overview of Digital Forensics",
  "toc_id": 9,
  "level_1_title": "Chapter 1. Understanding the Digital Forensics Profession and Investigations"
}

[Chunk 10 of 18 | chunk_id: 165]
  Content Preview: 'Files maintained on a computer are covered by different rules, depending on the nature of the documents. Many court cases in state and federal courts have developed and clarified how the rules apply to digital evidence. The Fourth Amendment The Fourt…'
  Metadata: {
  "level_2_title": "An Overview of Digital Forensics",
  "level_1_title": "Chapter 1. Understanding the Digital Forensics Profession and Investigations",
  "source": "Bill Nelson, Amelia Phillips, Christopher Steuart - Guide to Computer Forensics and Investigations_ Processing Digital Evidence-Cengage Learning (2018).epub",
  "chunk_id": 165,
  "toc_id": 9
}

[Chunk 11 of 18 | chunk_id: 166]
  Content Preview: 'has established a different precedent, so separate search warrants Legal documents that allow law enforcement to search an office, a home, or other locale for evidence related to an alleged crime. might not be necessary. However, when preparing to se…'
  Metadata: {
  "level_2_title": "An Overview of Digital Forensics",
  "toc_id": 9,
  "chunk_id": 166,
  "source": "Bill Nelson, Amelia Phillips, Christopher Steuart - Guide to

Computer Forensics and Investigations_ Processing Digital Evidence-Cengage Learning (2018).epub",
  "level_1_title": "Chapter 1. Understanding the Digital Forensics Profession and Investigations"
}

[Chunk 12 of 18 | chunk_id: 167]
  Content Preview: 'In an important case involving these issues, the Pennsylvania Supreme Court addressed expectations of privacy and whether evidence is admissible (see Commonwealth v. Copenhefer, 587 A.2d 1353, 526 Pa. 555 [1991]). Initial investigations by the FBI, s…'
  Metadata: {
  "toc_id": 9,
  "level_1_title": "Chapter 1. Understanding the Digital Forensics Profession and Investigations",
  "source": "Bill Nelson, Amelia Phillips, Christopher Steuart - Guide to Computer Forensics and Investigations_ Processing Digital Evidence-Cengage Learning (2018).epub",
  "chunk_id": 167,
  "level_2_title": "An Overview of Digital Forensics"
}

[Chunk 13 of 18 | chunk_id: 168]
  Content Preview: 'of the ransom note and directions. Subsequent search warrants resulted in seizure of evidence against him. Copenhefer’s computer contained several drafts and amendments of the text of phone calls to the victim and the victim’s husband the next day, t…'
  Metadata: {
  "level_2_title": "An Overview of Digital Forensics",
  "chunk_id": 168,
  "toc_id": 9,
  "level_1_title": "Chapter 1. Understanding the Digital Forensics Profession and Investigations",
  "source": "Bill Nelson, Amelia Phillips, Christopher Steuart - Guide to Computer Forensics and Investigations_ Processing Digital Evidence-Cengage Learning (2018).epub"
}

[Chunk 14 of 18 | chunk_id: 169]
  Content Preview: 'On direct appeal, the Pennsylvania Supreme Court concluded that the physical evidence, including the digital forensics evidence, was sufficient to support the bookstore owner’s conviction. Copenhefer’s argument was that "[E]ven though his computer wa…'
  Metadata: {
  "level_2_title": "An Overview of Digital Forensics",
  "source": "Bill Nelson, Amelia Phillips, Christopher Steuart - Guide to Computer Forensics and Investigations_ Processing Digital Evidence-Cengage Learning (2018).epub",

```
  "chunk_id": 169,
  "level_1_title": "Chapter 1. Understanding the Digital Forensics Profession
and Investigations",
  "toc_id": 9
}
```


[Chunk 15 of 18 | chunk_id: 170]
  Content Preview: 'The Pennsylvania Supreme Court rejected this argument, stating, "A defendant’s attempt to secrete evidence of a crime is not synonymous with a legally cognizable expectation of privacy. A mere hope for secrecy is not a legally protected expectation. …'

```
  Metadata: {
  "toc_id": 9,
  "level_2_title": "An Overview of Digital Forensics",
  "chunk_id": 170,
  "source": "Bill Nelson, Amelia Phillips, Christopher Steuart - Guide to
Computer Forensics and Investigations_ Processing Digital Evidence-Cengage
Learning (2018).epub",
  "level_1_title": "Chapter 1. Understanding the Digital Forensics Profession
and Investigations"
}
```


[Chunk 16 of 18 | chunk_id: 171]
  Content Preview: 'Every U.S. jurisdiction has case law related to the admissibility of evidence recovered from computers and other digital devices. As you learn in this book, however, the laws on digital evidence vary between states as well as between provinces and co…'

```
  Metadata: {
  "level_2_title": "An Overview of Digital Forensics",
  "chunk_id": 171,
  "level_1_title": "Chapter 1. Understanding the Digital Forensics Profession
and Investigations",
  "source": "Bill Nelson, Amelia Phillips, Christopher Steuart - Guide to
Computer Forensics and Investigations_ Processing Digital Evidence-Cengage
Learning (2018).epub",
  "toc_id": 9
}
```


[Chunk 17 of 18 | chunk_id: 172]
  Content Preview: 'Note…'

```
  Metadata: {
  "level_1_title": "Chapter 1. Understanding the Digital Forensics Profession
and Investigations",
  "chunk_id": 172,
  "level_2_title": "An Overview of Digital Forensics",
  "toc_id": 9,
  "source": "Bill Nelson, Amelia Phillips, Christopher Steuart - Guide to
Computer Forensics and Investigations_ Processing Digital Evidence-Cengage
```

```
Learning (2018).epub"
}

[ Chunk 18 of 18 | chunk_id: 173 ]
  Content Preview: 'The U.S. Department of Justice offers a useful guide to
search and seizure procedures for computers and computer evidence at https://pdf
s.semanticscholar.org/aabb/8b0caf982a0aa211f932252af38d4c9376fb.pdf…'
  Metadata: {
  "level_2_title": "An Overview of Digital Forensics",
  "source": "Bill Nelson, Amelia Phillips, Christopher Steuart - Guide to
Computer Forensics and Investigations_ Processing Digital Evidence-Cengage
Learning (2018).epub",
  "level_1_title": "Chapter 1. Understanding the Digital Forensics Profession
and Investigations",
  "toc_id": 9,
  "chunk_id": 173
}

================================================================================
Verification complete for section 'Chapter 1. Understanding the Digital
Forensics Profession and Investigations > An Overview of Digital Forensics'.
================================================================================
```

6.2 Test Data Base for content development

Require Description

[32]:
```python
# Cell 7: Verify Vector Database (Final Version with Rich Diagnostic Output)

import os
import json
import re
import random
import logging
from typing import List, Dict, Any, Tuple, Optional

# Third-party imports
try:
    from langchain_chroma import Chroma
    from langchain_ollama.embeddings import OllamaEmbeddings
    from langchain_core.documents import Document
    langchain_available = True
except ImportError:
    langchain_available = False

# Setup Logger for this cell
logging.basicConfig(level=logging.INFO, format='%(asctime)s - %(levelname)s -␣
 ↪%(message)s')
```

```python
logger = logging.getLogger(__name__)

# --- HELPER FUNCTIONS ---


def print_results(query_text: str, results: list, where_filter: Optional[Dict]␣
 ↪= None):
    """
    Richly prints query results, showing the query, filter, and retrieved␣
 ↪documents.
    """
    print("\n" + "-"*10 + " DIAGNOSTIC: RETRIEVAL RESULTS " + "-"*10)
    print(f"QUERY: '{query_text}'")
    if where_filter:
        print(f"FILTER: {json.dumps(where_filter, indent=2)}")

    if not results:
        print("--> No documents were retrieved for this query and filter.")
        print("-" * 55)
        return

    print(f"--> Found {len(results)} results. Displaying top {min(len(results),␣
 ↪3)}:")
    for i, doc in enumerate(results[:3]):
        print(f"\n[ RESULT {i+1} ]")
        content_preview = doc.page_content.replace('\n', ' ').strip()
        print(f"  Content : '{content_preview[:200]}...'")
        print(f"  Metadata: {json.dumps(doc.metadata, indent=2)}")
    print("-" * 55)


# --- HELPER FUNCTIONS FOR FINDING DATA (UNCHANGED) ---
def find_deep_entry(nodes: List[Dict], current_path: List[str] = []) ->␣
 ↪Optional[Tuple[Dict, List[str]]]:
    shuffled_nodes = random.sample(nodes, len(nodes))
    for node in shuffled_nodes:
        if node.get('level', 0) >= 2 and node.get('children'): return node,␣
 ↪current_path + [node['title']]
        if node.get('children'):
            path = current_path + [node['title']]
            deep_entry = find_deep_entry(node['children'], path)
            if deep_entry: return deep_entry
    return None

def find_chapter_title_by_number(toc_data: List[Dict], chap_num: int) ->␣
 ↪Optional[List[str]]:
```

```
    def search_nodes(nodes, num, current_path):
        for node in nodes:
            path = current_path + [node['title']]
            if re.match(rf"(Chapter\s)?{num}[.:\s]", node.get('title', ''), re.
 ↪IGNORECASE): return path
            if node.get('children'):
                found_path = search_nodes(node['children'], num, path)
                if found_path: return found_path
        return None
    return search_nodes(toc_data, chap_num, [])


# --- ENHANCED TEST CASES with DIAGNOSTIC OUTPUT ---

def basic_retrieval_test(db, outline):
    print_header("Test 1: Basic Retrieval", char="-")
    try:
        logger.info("Goal: Confirm the database is live and contains␣
 ↪thematically relevant content.")
        logger.info("Strategy: Perform a simple similarity search using the␣
 ↪course's 'unitName'.")
        query_text = outline.get("unitInformation", {}).get("unitName",␣
 ↪"introduction")

        logger.info(f"Action: Searching for query: '{query_text}'...")
        results = db.similarity_search(query_text, k=1)

        print_results(query_text, results) # <--- SHOW THE EVIDENCE

        logger.info("Verification: Check if at least one document was returned.
 ↪")
        assert len(results) > 0, "Basic retrieval query returned no results."

        logger.info("  Result: TEST 1 PASSED. The database is online and␣
 ↪responsive.")
        return True
    except Exception as e:
        logger.error(f"  Result: TEST 1 FAILED. Reason: {e}")
        return False

def deep_hierarchy_test(db, toc):
    print_header("Test 2: Deep Hierarchy Retrieval", char="-")
    try:
        logger.info("Goal: Verify that the multi-level hierarchical metadata␣
 ↪was ingested correctly.")
```

```
        logger.info("Strategy: Find a random, deeply nested sub-section and use␣
↪a precise filter to retrieve it.")
        deep_entry_result = find_deep_entry(toc)
        assert deep_entry_result, "Could not find a suitable deep entry (level␣
↪>= 2) to test."
        node, path = deep_entry_result
        query = node['title']

        logger.info(f"  - Selected random deep section: {' -> '.join(path)}")
        conditions = [{f"level_{i+1}_title": {"$eq": title}} for i, title in␣
↪enumerate(path)]
        w_filter = {"$and": conditions}

        logger.info("Action: Performing a similarity search with a highly␣
↪specific '$and' filter.")
        results = db.similarity_search(query, k=1, filter=w_filter)

        print_results(query, results, w_filter) # <--- SHOW THE EVIDENCE

        logger.info("Verification: Check if the precisely filtered query␣
↪returned any documents.")
        assert len(results) > 0, "Deeply filtered query returned no results."

        logger.info("  Result: TEST 2 PASSED. Hierarchical metadata is␣
↪structured correctly.")
        return True
    except Exception as e:
        logger.error(f"  Result: TEST 2 FAILED. Reason: {e}")
        return False

def advanced_alignment_test(db, outline, toc):
    print_header("Test 3: Advanced Unit Outline Alignment", char="-")
    try:
        logger.info("Goal: Ensure a weekly topic from the syllabus can be␣
↪mapped to the correct textbook chapter(s).")
        logger.info("Strategy: Pick a random week, find its chapter, and query␣
↪for the topic filtered by that chapter.")
        week_to_test = random.choice(outline['weeklySchedule'])
        logger.info(f"  - Selected random week: Week {week_to_test['week']} -␣
↪'{week_to_test['contentTopic']}'")

        reading = week_to_test.get('requiredReading', '')
        chap_nums_str = re.findall(r'\d+', reading)
        assert chap_nums_str, f"Could not find chapter numbers in required␣
↪reading: '{reading}'"
```

```
        logger.info(f"  - Extracted required chapter number(s):␣
↪{chap_nums_str}")

        chapter_paths = [find_chapter_title_by_number(toc, int(n)) for n in␣
↪chap_nums_str]
        chapter_paths = [path for path in chapter_paths if path is not None]
        assert chapter_paths, f"Could not map chapter numbers {chap_nums_str}␣
↪to a valid ToC path."

        level_1_titles = list(set([path[0] for path in chapter_paths]))
        logger.info(f"  - Mapped to top-level ToC entries: {level_1_titles}")

        or_filter = [{"level_1_title": {"$eq": title}} for title in␣
↪level_1_titles]
        w_filter = {"$or": or_filter} if len(or_filter) > 1 else or_filter[0]
        query = week_to_test['contentTopic']

        logger.info("Action: Searching for the weekly topic, filtered by the␣
↪mapped chapter(s).")
        results = db.similarity_search(query, k=5, filter=w_filter)

        print_results(query, results, w_filter) # <--- SHOW THE EVIDENCE

        logger.info("Verification: Check if at least one returned document is␣
↪from the correct chapter.")
        assert len(results) > 0, "Alignment query returned no results for the␣
↪correct section/chapter."

        logger.info("  Result: TEST 3 PASSED. The syllabus can be reliably␣
↪aligned with the textbook content.")
        return True
    except Exception as e:
        logger.error(f"  Result: TEST 3 FAILED. Reason: {e}")
        return False

def content_sequence_test(db, outline):
    print_header("Test 4: Content Sequence Verification", char="-")
    try:
        logger.info("Goal: Confirm that chunks for a topic can be re-ordered to␣
↪form a coherent narrative.")
        logger.info("Strategy: Retrieve several chunks for a random topic and␣
↪verify their 'chunk_id' is sequential.")
        topic_query = random.choice(outline['weeklySchedule'])['contentTopic']

        logger.info(f"Action: Performing similarity search for topic:␣
↪'{topic_query}' to get a set of chunks.")
```

```
        results = db.similarity_search(topic_query, k=10)

        print_results(topic_query, results) # <--- SHOW THE EVIDENCE

        docs_with_id = [doc for doc in results if 'chunk_id' in doc.metadata]
        assert len(docs_with_id) > 3, "Fewer than 4 retrieved chunks have a␣
↪'chunk_id' to test."

        chunk_ids = [doc.metadata['chunk_id'] for doc in docs_with_id]
        sorted_ids = sorted(chunk_ids)

        logger.info(f"  - Retrieved and sorted chunk IDs: {sorted_ids}")
        logger.info("Verification: Check if the sorted list of chunk_ids is␣
↪strictly increasing.")
        is_ordered = all(sorted_ids[i] >= sorted_ids[i-1] for i in range(1,␣
↪len(sorted_ids)))
        assert is_ordered, "The retrieved chunks' chunk_ids are not in␣
↪ascending order when sorted."

        logger.info("  Result: TEST 4 PASSED. Narrative order can be␣
↪reconstructed using 'chunk_id'.")
        return True
    except Exception as e:
        logger.error(f"  Result: TEST 4 FAILED. Reason: {e}")
        return False

# --- MAIN VERIFICATION EXECUTION ---
def run_verification():
    print_header("Database Verification Process")

    if not langchain_available:
        logger.error("LangChain libraries not found. Aborting tests.")
        return

    required_files = {
        "Chroma DB": CHROMA_PERSIST_DIR,
        "ToC JSON": PRE_EXTRACTED_TOC_JSON_PATH,
        "Parsed Outline": PARSED_UO_JSON_PATH
    }
    for name, path in required_files.items():
        if not os.path.exists(path):
            logger.error(f"Required '{name}' not found at '{path}'. Please run␣
↪previous cells.")
            return

    with open(PRE_EXTRACTED_TOC_JSON_PATH, 'r', encoding='utf-8') as f:
```

```
        toc_data = json.load(f)
    with open(PARSED_UO_JSON_PATH, 'r', encoding='utf-8') as f:
        unit_outline_data = json.load(f)

    logger.info("Connecting to DB and initializing components...")
    embeddings = OllamaEmbeddings(model=EMBEDDING_MODEL_OLLAMA)
    vector_store = Chroma(
        persist_directory=CHROMA_PERSIST_DIR,
        embedding_function=embeddings,
        collection_name=CHROMA_COLLECTION_NAME
    )

    results_summary = [
        basic_retrieval_test(vector_store, unit_outline_data),
        deep_hierarchy_test(vector_store, toc_data),
        advanced_alignment_test(vector_store, unit_outline_data, toc_data),
        content_sequence_test(vector_store, unit_outline_data)
    ]

    passed_count = sum(filter(None, results_summary))
    failed_count = len(results_summary) - passed_count

    print_header("Verification Summary")
    print(f"Total Tests Run: {len(results_summary)}")
    print(f"  Passed: {passed_count}")
    print(f"  Failed: {failed_count}")
    print_header("Verification Complete", char="=")

# --- Execute Verification ---
# Assumes global variables from Cell 1 are available in the notebook's scope
run_verification()
```

```
2025-07-18 21:48:02,320 - INFO - Connecting to DB and initializing components…
2025-07-18 21:48:02,331 - INFO - Goal: Confirm the database is live and contains
thematically relevant content.
2025-07-18 21:48:02,332 - INFO - Strategy: Perform a simple similarity search
using the course's 'unitName'.
2025-07-18 21:48:02,332 - INFO - Action: Searching for query: 'Digital
Forensic'…
2025-07-18 21:48:02,401 - INFO - HTTP Request: POST
http://127.0.0.1:11434/api/embed "HTTP/1.1 200 OK"
2025-07-18 21:48:02,405 - INFO - Verification: Check if at least one document
was returned.
2025-07-18 21:48:02,405 - INFO -   Result: TEST 1 PASSED. The database is online
and responsive.
2025-07-18 21:48:02,406 - INFO - Goal: Verify that the multi-level hierarchical
metadata was ingested correctly.
2025-07-18 21:48:02,406 - INFO - Strategy: Find a random, deeply nested sub-
```

```
section and use a precise filter to retrieve it.
2025-07-18 21:48:02,407 - INFO -   - Selected random deep section: Chapter 15.
Expert Testimony in Digital Investigations -> Preparing for a Deposition or
Hearing -> Guidelines for Testifying at Depositions
2025-07-18 21:48:02,407 - INFO - Action: Performing a similarity search with a
highly specific '$and' filter.
2025-07-18 21:48:02,462 - INFO - HTTP Request: POST
http://127.0.0.1:11434/api/embed "HTTP/1.1 200 OK"


================================================================================
                        Database Verification Process
================================================================================

--------------------------------------------------------------------------------
                            Test 1: Basic Retrieval
--------------------------------------------------------------------------------

---------- DIAGNOSTIC: RETRIEVAL RESULTS ----------
QUERY: 'Digital Forensic'
--> Found 1 results. Displaying top 1:

[ RESULT 1 ]
  Content : 'An Overview of Digital Forensics…'
  Metadata: {
  "toc_id": 9,
  "level_2_title": "An Overview of Digital Forensics",
  "chunk_id": 156,
  "level_1_title": "Chapter 1. Understanding the Digital Forensics Profession
and Investigations",
  "source": "Bill Nelson, Amelia Phillips, Christopher Steuart - Guide to
Computer Forensics and Investigations_ Processing Digital Evidence-Cengage
Learning (2018).epub"
}
---------------------------------------------------

--------------------------------------------------------------------------------
                        Test 2: Deep Hierarchy Retrieval
--------------------------------------------------------------------------------

2025-07-18 21:48:02,472 - INFO - Verification: Check if the precisely filtered
query returned any documents.
2025-07-18 21:48:02,473 - INFO -   Result: TEST 2 PASSED. Hierarchical metadata
is structured correctly.
2025-07-18 21:48:02,473 - INFO - Goal: Ensure a weekly topic from the syllabus
can be mapped to the correct textbook chapter(s).
2025-07-18 21:48:02,474 - INFO - Strategy: Pick a random week, find its chapter,
and query for the topic filtered by that chapter.
2025-07-18 21:48:02,474 - INFO -   - Selected random week: Week Week 4 -
```

```
'Processing Crime and Incident Scenes.'
2025-07-18 21:48:02,475 - INFO -   - Extracted required chapter number(s):
['2019', '978', '1', '337', '56894', '4', '4']
2025-07-18 21:48:02,480 - INFO -   - Mapped to top-level ToC entries: ['Chapter
1. Understanding the Digital Forensics Profession and Investigations', 'Chapter
4. Processing Crime and Incident Scenes']
2025-07-18 21:48:02,481 - INFO - Action: Searching for the weekly topic,
filtered by the mapped chapter(s).


---------- DIAGNOSTIC: RETRIEVAL RESULTS ----------
QUERY: 'Guidelines for Testifying at Depositions'
FILTER: {
  "$and": [
    {
      "level_1_title": {
        "$eq": "Chapter 15. Expert Testimony in Digital Investigations"
      }
    },
    {
      "level_2_title": {
        "$eq": "Preparing for a Deposition or Hearing"
      }
    },
    {
      "level_3_title": {
        "$eq": "Guidelines for Testifying at Depositions"
      }
    }
  ]
}
--> Found 1 results. Displaying top 1:

[ RESULT 1 ]
  Content : 'Guidelines for Testifying at Depositions…'
  Metadata: {
  "level_2_title": "Preparing for a Deposition or Hearing",
  "level_3_title": "Guidelines for Testifying at Depositions",
  "source": "Bill Nelson, Amelia Phillips, Christopher Steuart - Guide to
Computer Forensics and Investigations_ Processing Digital Evidence-Cengage
Learning (2018).epub",
  "level_1_title": "Chapter 15. Expert Testimony in Digital Investigations",
  "toc_id": 513,
  "chunk_id": 7313
}
------------------------------------------------------

--------------------------------------------------------------------------------
```

```
                    Test 3: Advanced Unit Outline Alignment
--------------------------------------------------------------------------------

2025-07-18 21:48:02,545 - INFO - HTTP Request: POST
http://127.0.0.1:11434/api/embed "HTTP/1.1 200 OK"
2025-07-18 21:48:02,567 - INFO - Verification: Check if at least one returned
document is from the correct chapter.
2025-07-18 21:48:02,567 - INFO -   Result: TEST 3 PASSED. The syllabus can be
reliably aligned with the textbook content.
2025-07-18 21:48:02,568 - INFO - Goal: Confirm that chunks for a topic can be
re-ordered to form a coherent narrative.
2025-07-18 21:48:02,569 - INFO - Strategy: Retrieve several chunks for a random
topic and verify their 'chunk_id' is sequential.
2025-07-18 21:48:02,570 - INFO - Action: Performing similarity search for topic:
'Working with Windows and CLI Systems.' to get a set of chunks.
2025-07-18 21:48:02,619 - INFO - HTTP Request: POST
http://127.0.0.1:11434/api/embed "HTTP/1.1 200 OK"
2025-07-18 21:48:02,624 - INFO -   - Retrieved and sorted chunk IDs: [48, 2438,
2450, 3057, 3275, 3288, 3382, 9201, 9303, 9345]
2025-07-18 21:48:02,625 - INFO - Verification: Check if the sorted list of
chunk_ids is strictly increasing.
2025-07-18 21:48:02,625 - INFO -   Result: TEST 4 PASSED. Narrative order can be
reconstructed using 'chunk_id'.


---------- DIAGNOSTIC: RETRIEVAL RESULTS ----------
QUERY: 'Processing Crime and Incident Scenes.'
FILTER: {
  "$or": [
    {
      "level_1_title": {
        "$eq": "Chapter 1. Understanding the Digital Forensics Profession and
Investigations"
      }
    },
    {
      "level_1_title": {
        "$eq": "Chapter 4. Processing Crime and Incident Scenes"
      }
    }
  ]
}
--> Found 5 results. Displaying top 3:

[ RESULT 1 ]
  Content : 'Processing Incident or Crime Scenes…'
  Metadata: {
  "source": "Bill Nelson, Amelia Phillips, Christopher Steuart - Guide to
Computer Forensics and Investigations_ Processing Digital Evidence-Cengage
```

```
Learning (2018).epub",
  "level_1_title": "Chapter 4. Processing Crime and Incident Scenes",
  "toc_id": 162,
  "level_3_title": "Processing Incident or Crime Scenes",
  "chunk_id": 2046,
  "level_2_title": "Seizing Digital Evidence at the Scene"
}

[ RESULT 2 ]
  Content : 'Chapter 4. Processing Crime and Incident Scenes…'
  Metadata: {
  "toc_id": 143,
  "level_1_title": "Chapter 4. Processing Crime and Incident Scenes",
  "chunk_id": 1844,
  "source": "Bill Nelson, Amelia Phillips, Christopher Steuart - Guide to
Computer Forensics and Investigations_ Processing Digital Evidence-Cengage
Learning (2018).epub"
}

[ RESULT 3 ]
  Content : 'Chapter 4. Processing Crime and Incident Scenes…'
  Metadata: {
  "source": "Bill Nelson, Amelia Phillips, Christopher Steuart - Guide to
Computer Forensics and Investigations_ Processing Digital Evidence-Cengage
Learning (2018).epub",
  "level_1_title": "Chapter 4. Processing Crime and Incident Scenes",
  "toc_id": 143,
  "chunk_id": 9102
}
--------------------------------------------------------

--------------------------------------------------------------------------------
                     Test 4: Content Sequence Verification
--------------------------------------------------------------------------------

---------- DIAGNOSTIC: RETRIEVAL RESULTS ----------
QUERY: 'Working with Windows and CLI Systems.'
--> Found 10 results. Displaying top 3:

[ RESULT 1 ]
  Content : 'Chapter 5. Working with Windows and CLI Systems…'
  Metadata: {
  "chunk_id": 2438,
  "toc_id": 183,
  "source": "Bill Nelson, Amelia Phillips, Christopher Steuart - Guide to
Computer Forensics and Investigations_ Processing Digital Evidence-Cengage
Learning (2018).epub",
  "level_1_title": "Chapter 5. Working with Windows and CLI Systems"
```

```
}

[ RESULT 2 ]
  Content : 'Chapter 5. Working with Windows and CLI Systems…'
  Metadata: {
  "source": "Bill Nelson, Amelia Phillips, Christopher Steuart - Guide to
Computer Forensics and Investigations_ Processing Digital Evidence-Cengage
Learning (2018).epub",
  "level_1_title": "Chapter 5. Working with Windows and CLI Systems",
  "chunk_id": 9345,
  "toc_id": 183
}

[ RESULT 3 ]
  Content : 'Chapter 5 , "Working with Windows and CLI Systems," discusses the
most common operating systems. You learn what happens and what files are altered
during computer startup and how file systems deal wit…'
  Metadata: {
  "source": "Bill Nelson, Amelia Phillips, Christopher Steuart - Guide to
Computer Forensics and Investigations_ Processing Digital Evidence-Cengage
Learning (2018).epub",
  "toc_id": 4,
  "chunk_id": 48,
  "level_1_title": "Introduction"
}
--------------------------------------------------

================================================================================
                              Verification Summary
================================================================================
Total Tests Run: 4
  Passed: 4
  Failed: 0

================================================================================
                             Verification Complete
================================================================================
```

7 Content Generation

7.1 Planning Agent

[38]:
```
# PlanningAgent
# Cell 8: The Data-Driven Planning Agent (Final Hierarchical Version)

import os
import json
```

```
import re
import math
import logging
from typing import List, Dict, Any, Optional, Tuple

# Setup Logger and LangChain components
logging.basicConfig(level=logging.INFO, format='%(asctime)s - %(levelname)s -␣
 ↪%(message)s')
logger = logging.getLogger(__name__)
try:
    from langchain_chroma import Chroma
    from langchain_ollama.embeddings import OllamaEmbeddings
    langchain_available = True
except ImportError:
    langchain_available = False

def print_header(text: str, char: str = "="):
    """Prints a centered header to the console."""
    print("\n" + char * 80)
    print(text.center(80))
    print(char * 80)

class PlanningAgent:
    """
    An agent that creates a hierarchical content plan, adaptively partitions␣
 ↪content
    into distinct lecture decks, and allocates presentation time.
    """
    def __init__(self, master_config: Dict, vector_store: Optional[Any] = None):
        self.config = master_config['processed_settings']
        self.unit_outline = master_config['unit_outline']
        self.book_toc = master_config['book_toc']
        self.flat_toc_with_ids = self._create_flat_toc_with_ids()
        self.vector_store = vector_store
        logger.info("Data-Driven PlanningAgent initialized successfully.")

    def _create_flat_toc_with_ids(self) -> List[Dict]:
        """Creates a flattened list of the ToC for easy metadata lookup."""
        flat_list = []
        def flatten_recursive(nodes, counter):
            for node in nodes:
                node_id = counter[0]; counter[0] += 1
                flat_list.append({'toc_id': node_id, 'title': node.get('title',␣
 ↪''), 'node': node})
                if node.get('children'):
                    flatten_recursive(node.get('children'), counter)
        flatten_recursive(self.book_toc, [0])
```

```
        return flat_list

    def _assign_sequence_ids_recursively(self, node: Dict, counter: list):
        """
        Recursively walks a content node, assigning a unique, sequential ID
        to the node, its children, and its interactive activity in a␣
↪depth-first order.
        """
        # 1. Assign sequence ID to the parent content node itself.
        node['seq_id'] = counter[0]
        counter[0] += 1

        # 2. Recurse through all children first.
        if 'children' in node and node['children']:
            for child in node['children']:
                self._assign_sequence_ids_recursively(child, counter)

        # 3. Assign an ID to the interactive activity LAST, so it appears after␣
↪the content and its children.
        if 'interactive_activity' in node:
            node['interactive_activity']['seq_id'] = counter[0]
            counter[0] += 1

    def _identify_relevant_chapters(self, weekly_schedule_item: Dict) ->␣
↪List[int]:
        """Extracts chapter numbers precisely from the 'requiredReading' string.
↪"""
        reading_str = weekly_schedule_item.get('requiredReading', '')
        match = re.search(r'Chapter(s)?', reading_str, re.IGNORECASE)
        if not match: return []
        search_area = reading_str[match.start():]
        chap_nums_str = re.findall(r'\d+', search_area)
        if chap_nums_str:
            return sorted(list(set(int(n) for n in chap_nums_str)))
        return []

    def _find_chapter_node(self, chapter_number: int) -> Optional[Dict]:
        """Finds the ToC node for a specific chapter number."""
        for item in self.flat_toc_with_ids:
            if re.match(rf"Chapter\s{chapter_number}(?:\D|$)", item['title']):
                return item['node']
        return None

    def _build_topic_plan_tree(self, toc_node: Dict) -> Dict:
        """
        Recursively builds a hierarchical plan tree from any ToC node,
        annotating it with direct and total branch chunk counts.
```

```
        """
        node_metadata = next((item for item in self.flat_toc_with_ids if␣
↪item['node'] is toc_node), None)
        if not node_metadata: return {}

        retrieved_docs = self.vector_store.get(where={'toc_id':␣
↪node_metadata['toc_id']})
        direct_chunk_count = len(retrieved_docs.get('ids', []))

        plan_node = {
            "title": node_metadata['title'],
            "toc_id": node_metadata['toc_id'],
            "chunk_count": direct_chunk_count,
            "total_chunks_in_branch": 0,
            "slides_allocated": 0,
            "children": []
        }

        child_branch_total = 0
        for child_node in toc_node.get('children', []):
            if any(ex in child_node.get('title', '').lower() for ex in␣
↪["review", "introduction", "summary", "key terms"]):
                continue
            child_plan_node = self._build_topic_plan_tree(child_node)
            if child_plan_node:
                plan_node['children'].append(child_plan_node)
                child_branch_total += child_plan_node.
↪get('total_chunks_in_branch', 0)

        plan_node['total_chunks_in_branch'] = direct_chunk_count +␣
↪child_branch_total
        return plan_node

    # In PlanningAgent Class...

    def _allocate_slides_to_tree(self, plan_tree: Dict, content_slides_budget:␣
↪int):
        """
        (FINAL, REORDERED FOR CLARITY) Performs a multi-pass process to␣
↪allocate content slides,
        add activities, sum totals, and reorders the keys in each node for␣
↪maximum readability.
        """
        if not plan_tree or content_slides_budget <= 0:
            return plan_tree
```

```
        # --- Pass 1: Allocate Content Slides ---
        def allocate_content_recursively(node, budget):
            node['budget_slides_content'] = round(budget)
            node['direct_slides_content'] = 0
            if not node.get('children'):
                node['direct_slides_content'] = round(budget)
                return
            total_branch_chunks = node.get('total_chunks_in_branch', 0)
            own_content_slides = 0
            if total_branch_chunks > 0:
                own_content_slides = round(budget * (node.get('chunk_count', 0)␣
↪/ total_branch_chunks))
            node['direct_slides_content'] = own_content_slides
            remaining_budget_for_children = budget - own_content_slides
            children_total_chunks = total_branch_chunks - node.
↪get('chunk_count', 0)
            if children_total_chunks <= 0: return
            for child in node.get('children', []):
                child_budget = remaining_budget_for_children * (child.
↪get('total_chunks_in_branch', 0) / children_total_chunks)
                allocate_content_recursively(child, child_budget)

        allocate_content_recursively(plan_tree, content_slides_budget)

        # --- Pass 2: Add Interactive Activities ---
        def add_interactive_nodes(node, depth, interactive_deep):
            if not node: return
            if self.config.get('interactive', False):
                if interactive_deep:
                    if depth == 2: node['interactive_activity'] = {"title":␣
↪f"{node.get('title')} (Deep-Dive Activity)", "toc_id": node.get('toc_id'),␣
↪"slides_allocated": 1}
                    if depth == 1: node['interactive_activity'] = {"title":␣
↪f"{node.get('title')} (General Activity)", "toc_id": node.get('toc_id'),␣
↪"slides_allocated": 1}
                else:
                    if depth == 1: node['interactive_activity'] = {"title":␣
↪f"{node.get('title')} (Interactive Activity)", "toc_id": node.get('toc_id'),␣
↪"slides_allocated": 1}
            for child in node.get('children', []):
                add_interactive_nodes(child, depth + 1, interactive_deep)

        add_interactive_nodes(plan_tree, 1, self.config.get('interactive_deep',␣
↪False))

        # --- Pass 3: Sum All Slides Up the Tree ---
```

```
        def sum_slides_upwards(node):
            total_slides = node.get('direct_slides_content', 0)
            total_slides += node.get('interactive_activity', {}).
↪get('slides_allocated', 0)
            if node.get('children'):
                total_slides += sum(sum_slides_upwards(child) for child in node.
↪get('children', []))
            node['total_slides_in_branch'] = total_slides
            return total_slides

        sum_slides_upwards(plan_tree)

        # --- NEW: Pass 4: Reorder keys for final clarity ---
        def reorder_keys_for_readability(node: Dict) -> Dict:
            if not node:
                return None

            # Define the desired order of keys
            key_order = [
                "title",
                "toc_id",
                "chunk_count",
                "total_chunks_in_branch",
                "budget_slides_content",
                "direct_slides_content",
                "total_slides_in_branch",
                "children",
                "interactive_activity"

            ]

            # Rebuild the dictionary in the specified order
            reordered_node = {key: node[key] for key in key_order if key in␣
↪node}

            # Recursively reorder children
            if 'children' in reordered_node:
                reordered_node['children'] =␣
↪[reorder_keys_for_readability(child) for child in reordered_node['children']]

            return reordered_node

        return reorder_keys_for_readability(plan_tree)

    def create_content_plan_for_week(self, week_number: int) -> Optional[Dict]:
        """Orchestrates the adaptive planning and partitioning process."""
        print_header(f"Planning Week {week_number}", char="*")
```

```
        weekly_schedule_item = self.unit_outline['weeklySchedule'][week_number␣
↪- 1]
        chapter_numbers = self._identify_relevant_chapters(weekly_schedule_item)
        if not chapter_numbers: return None

        num_decks = self.config['week_session_setup'].get('sessions_per_week',␣
↪1)

        # 1. Build a full plan tree for each chapter to get its weight.
        chapter_plan_trees = [self._build_topic_plan_tree(self.
↪_find_chapter_node(cn)) for cn in chapter_numbers if self.
↪_find_chapter_node(cn)]
        total_weekly_chunks = sum(tree.get('total_chunks_in_branch', 0) for␣
↪tree in chapter_plan_trees)

        # 2. NEW: Adaptive Partitioning Strategy
        partitionable_units = []
        all_top_level_sections = []
        for chapter_tree in chapter_plan_trees:
            all_top_level_sections.extend(chapter_tree.get('children', []))

        num_top_level_sections = len(all_top_level_sections)

        # Always prefer to split by top-level sections if there are enough to␣
↪distribute.
        if num_top_level_sections >= num_decks:
            logger.info(f"Partitioning strategy: Distributing␣
↪{num_top_level_sections} top-level sections across {num_decks} decks.")
            partitionable_units = all_top_level_sections
        else:
            # Fallback for rare cases where there are fewer topics than decks␣
↪(e.g., 1 chapter with 1 section, but 2 decks).
            logger.info(f"Partitioning strategy: Not enough top-level sections␣
↪({num_top_level_sections}) to fill all decks ({num_decks}). Distributing␣
↪whole chapters instead.")
            partitionable_units = chapter_plan_trees

        # 3. Partition the chosen units into decks using a bin-packing algorithm
        decks = [[] for _ in range(num_decks)]
        deck_weights = [0] * num_decks
        sorted_units = sorted(partitionable_units, key=lambda x: x.
↪get('toc_id', 0))

        for unit in sorted_units:
            lightest_deck_index = deck_weights.index(min(deck_weights))
```

```
            decks[lightest_deck_index].append(unit)
            deck_weights[lightest_deck_index] += unit.
↪get('total_chunks_in_branch', 0)

        # 4. Plan each deck
        content_slides_per_week = self.config['slide_count_strategy'].
↪get('target_total_slides', 25)
        final_deck_plans = []
        for i, deck_content_trees in enumerate(decks):
            deck_number = i + 1
            deck_chunk_weight = sum(tree.get('total_chunks_in_branch', 0) for␣
↪tree in deck_content_trees)
            deck_slide_budget = round((deck_chunk_weight / total_weekly_chunks)␣
↪* content_slides_per_week) if total_weekly_chunks > 0 else 0

            logger.info(f"--- Planning Deck {deck_number}/{num_decks} | Topics:␣
↪{[t['title'] for t in deck_content_trees]} | Weight: {deck_chunk_weight}␣
↪chunks | Slide Budget: {deck_slide_budget} ---")

            # The allocation function is recursive and works on any tree or␣
↪sub-tree
            planned_content = [self._allocate_slides_to_tree(tree,␣
↪round(deck_slide_budget * (tree.get('total_chunks_in_branch', 0) /␣
↪deck_chunk_weight))) if deck_chunk_weight > 0 else tree for tree in␣
↪deck_content_trees]

            final_deck_plans.append({
                "deck_number": deck_number,
                "deck_title": f"{self.config.get('unit_name', 'Course')} - Week␣
↪{week_number}, Lecture {deck_number}",
                "session_content": planned_content
            })

        return {
            "week": week_number,
            "overall_topic": weekly_schedule_item.get('contentTopic'),
            "deck_plans": final_deck_plans
        }


    def finalize_and_calculate_time_plan(self,draft_plan: Dict, config: Dict)␣
↪-> Dict:
        """
        Takes a draft plan and enriches it by:
        1. Calculating detailed slide counts and time allocations for every␣
↪node.
```

```
        2. Adding framework sections and wrapping content.
        3. Calculating and adding summaries for decks and the entire week.
        4. Assigning a final sequence ID (seq_id) for slide generation.
        5. Reordering all keys for maximum readability.
        """
        final_plan = json.loads(json.dumps(draft_plan))

        # --- Time Constants from Config ---
        params = config.get('parameters_slides', {})
        TIME_PER_CONTENT = params.get('time_per_content_slides_min', 3)
        TIME_PER_INTERACTIVE = params.get('time_per_interactive_slide_min', 5)
        TIME_FOR_FRAMEWORK_DECK = params.get('time_for_framework_slides_min', 6)
        FRAMEWORK_SLIDES_PER_DECK = 4


        # --- Recursive Helper Functions ---
        def _calculate_time_and_reorder(node: Dict):
            # 1. Recurse to the bottom first to perform a bottom-up calculation
            children_total_time = 0
            if 'children' in node and node['children']:
                for child in node['children']:
                    _calculate_time_and_reorder(child) # Recursive call
                    children_total_time += child.get('time_allocation_minutes',␣
↪{}).get('total_branch_time', 0)

            # 2. Calculate this node's direct time
            direct_content_time = node.get('direct_slides_content', 0) *␣
↪TIME_PER_CONTENT
            interactive_time = node.get('interactive_activity', {}).
↪get('slides_allocated', 0) * TIME_PER_INTERACTIVE

            # 3. Calculate this node's total branch time
            branch_total_time = direct_content_time + interactive_time +␣
↪children_total_time

            # 4. Create the time allocation object
            time_alloc = {
                "direct_content_time": direct_content_time,
                "direct_interactive_time": interactive_time,
                "total_branch_time": branch_total_time
            }
            node['time_allocation_minutes'] = time_alloc

            # 5. Reorder all keys for this node to ensure final clarity
            key_order = [
                "title",
                "toc_id",
```

```
                "chunk_count",
                "total_chunks_in_branch",
                "budget_slides_content",
                "direct_slides_content",
                "total_slides_in_branch",
                "time_allocation_minutes",
                "children",
                "interactive_activity"

            ]
            reordered_node = {key: node[key] for key in key_order if key in␣
↪node}

            # Clear the original node and update it with the reordered keys
            node.clear()
            node.update(reordered_node)

        # --- Main Processing Loop for Decks ---
        for deck in final_plan.get("deck_plans", []):
            session_content_blocks = deck.pop("session_content", [])

            # Perform the combined time calculation and reordering pass
            for block in session_content_blocks:
                _calculate_time_and_reorder(block)

            # Create Framework Sections
            week_number, deck_number = final_plan.get("week"), deck.
↪get("deck_number")
            title_section = {"section_type": "Title", "content": { "unit_name":␣
↪config.get('unit_name', 'Course'), "unit_code": config.get('course_id', ''),␣
↪"week_topic": final_plan.get('overall_topic', ''), "deck_title": f"Week␣
↪{week_number}, Lecture {deck_number}"}}
            agenda_section = {"section_type": "Agenda", "content": {"title":␣
↪"Today's Agenda", "items": [item.get('title', 'Untitled Topic') for item in␣
↪session_content_blocks]}}
            summary_section = {"section_type": "Summary", "content": {"title":␣
↪"Summary & Key Takeaways", "placeholder": "Auto-generate based on covered␣
↪topics."}}
            end_section = {"section_type": "End", "content": {"title": "Thank␣
↪You", "text": "Questions?"}}
            main_content_block = {"section_type": "Content", "content_blocks":␣
↪session_content_blocks}

            final_sections_for_deck = [title_section, agenda_section,␣
↪main_content_block, summary_section, end_section]
```

```
            # ************************************************************
            # *** NEW: Assign Sequential IDs for the entire deck ***
            # ************************************************************
            seq_counter = [0] # Use a list for pass-by-reference behavior
            for section in final_sections_for_deck:
                if section.get("section_type") == "Content":
                    # For the main content, iterate through its blocks and␣
↪start the recursion
                    for block in section.get("content_blocks", []):
                        self._assign_sequence_ids_recursively(block,␣
↪seq_counter)
                else:
                    # For simple framework slides, just assign the next ID
                    section['seq_id'] = seq_counter[0]
                    seq_counter[0] += 1
            # ************************************************************
            # *** END NEW CODE ***
            # ************************************************************


            # Calculate Deck Summaries
            total_content_slides = sum(b.get('total_slides_in_branch', 0) - b.
↪get('interactive_activity',{}).get('slides_allocated',0) for b in␣
↪session_content_blocks)
            total_interactive_slides = sum(b.get('interactive_activity',{}).
↪get('slides_allocated',0) for b in session_content_blocks)

            deck_content_time = sum(b.get('time_allocation_minutes', {}).
↪get('total_branch_time', 0) for b in session_content_blocks)

            deck['total_slides_in_deck'] = FRAMEWORK_SLIDES_PER_DECK + sum(b.
↪get('total_slides_in_branch', 0) for b in session_content_blocks)
            deck['slide_count_breakdown'] = {"framework":␣
↪FRAMEWORK_SLIDES_PER_DECK, "content": total_content_slides, "interactive":␣
↪total_interactive_slides}
            deck['time_breakdown_minutes'] = {"framework":␣
↪TIME_FOR_FRAMEWORK_DECK, "content_and_interactive": deck_content_time,␣
↪"total_deck_time": TIME_FOR_FRAMEWORK_DECK + deck_content_time}
            deck['sections'] = final_sections_for_deck
            if 'deck_title' in deck: del deck['deck_title']

        # --- Calculate Grand Totals for the Week ---
```

```
        weekly_slide_summary = {"total_slides_for_week": 0,␣
↪"total_framework_slides": 0, "total_content_slides": 0,␣
↪"total_interactive_slides": 0, "number_of_decks": len(final_plan.
↪get("deck_plans", []))}
        weekly_time_summary = {"total_time_for_week_minutes": 0,␣
↪"total_framework_time": 0, "total_content_and_interactive_time": 0}

        for deck in final_plan.get("deck_plans", []):
            weekly_slide_summary['total_slides_for_week'] += deck.
↪get('total_slides_in_deck', 0)
            for key, value in deck.get('slide_count_breakdown', {}).items():␣
↪weekly_slide_summary[f"total_{key}_slides"] += value
            weekly_time_summary['total_time_for_week_minutes'] += deck.
↪get('time_breakdown_minutes', {}).get('total_deck_time', 0)
            weekly_time_summary['total_framework_time'] += deck.
↪get('time_breakdown_minutes', {}).get('framework', 0)
            weekly_time_summary['total_content_and_interactive_time'] += deck.
↪get('time_breakdown_minutes', {}).get('content_and_interactive', 0)

        # --- Construct Final Ordered Plan ---
        final_ordered_plan = {
            "week": final_plan.get("week"),
            "overall_topic": final_plan.get("overall_topic"),
            "weekly_slide_summary": weekly_slide_summary,
            "weekly_time_summary_minutes": weekly_time_summary,
            "deck_plans": final_plan.get("deck_plans", [])
        }

        return final_ordered_plan

    # --- NEW FUNCTION TO GENERATE MASTER SUMMARY ---
    def generate_and_save_master_plan(self, weekly_plans: List[Dict], config:␣
↪Dict):
        """
        Aggregates summaries from all weekly plans into a single master plan␣
↪file,
        including new grand total metrics.
        """
        print_header("Phase 4: Generating Master Unit Plan", char="#")

        # Initialize the master plan structure with the new fields
        master_plan = {
            "unit_code": config.get('course_id', 'UNKNOWN'),
            "unit_name": config.get('unit_name', 'Unknown Unit'),
            "grand_total_summary": {
                "total_slides_for_unit": 0,
```

```
                "total_framework_slides": 0,
                "total_content_slides": 0,
                "total_interactive_slides": 0,
                "total_number_of_decks": 0,
                "total_time_for_unit_minutes": 0,
                "total_time_for_unit_in_hour": 0, # New
                "average_deck_time_in_min": 0,
                "average_deck_time_in_hour": 0
            },
            "weekly_summaries": []
        }

        grand_totals = master_plan["grand_total_summary"]

        # Loop through each weekly plan to aggregate data
        for plan in sorted(weekly_plans, key=lambda p: p.get('week', 0)):
            # Extract the high-level summary for this week
            summary_entry = {
                "week": plan.get("week"),
                "overall_topic": plan.get("overall_topic"),
                "slide_summary": plan.get("weekly_slide_summary"),
                "time_summary_minutes": plan.get("weekly_time_summary_minutes")
            }
            master_plan["weekly_summaries"].append(summary_entry)

            # Add this week's totals to the grand totals
            slide_summary = plan.get("weekly_slide_summary", {})
            time_summary = plan.get("weekly_time_summary_minutes", {})

            grand_totals["total_slides_for_unit"] += slide_summary.
↪get("total_slides_for_week", 0)
            grand_totals["total_framework_slides"] += slide_summary.
↪get("total_framework_slides", 0)
            grand_totals["total_content_slides"] += slide_summary.
↪get("total_content_slides", 0)
            grand_totals["total_interactive_slides"] += slide_summary.
↪get("total_interactive_slides", 0)
            grand_totals["total_number_of_decks"] += slide_summary.
↪get("number_of_decks", 0)
            grand_totals["total_time_for_unit_minutes"] += time_summary.
↪get("total_time_for_week_minutes", 0)

        # --- NEW: Calculate the final derived grand totals after the loop ---
        if grand_totals["total_time_for_unit_minutes"] > 0:
            grand_totals["total_time_for_unit_in_hour"] =␣
↪round(grand_totals["total_time_for_unit_minutes"] / 60, 2)
```

```
        if grand_totals["total_number_of_decks"] > 0:
            grand_totals["average_deck_time_in_min"] =␣
↪round(grand_totals["total_time_for_unit_minutes"] /␣
↪grand_totals["total_number_of_decks"], 2)

        if grand_totals["total_number_of_decks"] > 0:
            grand_totals["average_deck_time_in_hour"] =␣
↪round((grand_totals["total_time_for_unit_minutes"] /␣
↪grand_totals["total_number_of_decks"]) / 60, 2)


        master_filename = f"{config.get('course_id', 'UNIT')}_master_plan_unit.
↪json"
        output_path = os.path.join(PLAN_OUTPUT_DIR, master_filename)

        try:
            with open(output_path, 'w') as f:
                json.dump(master_plan, f, indent=2)
            logger.info(f"Successfully generated and saved Master Unit Plan to:␣
↪{output_path}")
            print("\n--- Preview of Master Plan ---")
            print(json.dumps(master_plan, indent=2))
            return True
        except Exception as e:
            logger.error(f"Failed to save Master Unit Plan: {e}", exc_info=True)
```

7.2 Content Generator Agent

[39]:
```
# CONTENT_SLIDE_GENERATION_PROMPT = """
# You are an expert university lecturer and instructional designer. Your task␣
↪is to create the content for a single, specific PowerPoint slide as part of␣
↪a larger topic.

# **CONTEXT:**
# - **Main Topic:** "{main_topic_title}"
# - **Overall Raw Text for this Topic:** ```{topic_raw_content}```
# - **Generation Task:** You are generating slide **{current_slide_num}** of a␣
↪total of **{total_slides_for_topic}** slides for this topic.
# - **Content Covered on Previous Slides:** {generation_history}

# **INSTRUCTIONS:**
# 1.  Based on all the context, determine the most logical sub-topic for the␣
↪*next* slide ({current_slide_num}). Do NOT repeat content from previous␣
↪slides.
```

```
# 2.  Create the content for this single slide. You can use one or two "Content␣
 ↪Objects" to structure the slide (e.g., use two objects for a comparison).
# 3.  Your output **MUST** be a single, well-formed JSON object. Do NOT add any␣
 ↪text or explanations outside of the JSON object.

# **JSON OUTPUT STRUCTURE & RULES:**
# {{
#   "title": "string | The main title for the slide, consistent with the main␣
 ↪topic.",
#   "subtitle": "string | A more specific subtitle for this particular slide.␣
 ↪Use null if not needed.",
#   "objects": [
#     {{
#       "content_type": "string | CHOOSE ONE: 'bullet_points', 'table'.",
#       "content_purpose": "string | CHOOSE ONE: 'description', 'explanation',␣
 ↪'timeline', 'process', 'cycle', 'comparison', 'contrast', 'hierarchy',␣
 ↪'case_study', 'example'.",
#       "data": "object | The structured data for the slide. See examples below.
 ↪"
#     }}
#   ]
# }}

# **DATA STRUCTURE EXAMPLES:**
# - For "bullet_points": "data": ["Point 1", {{"text": "Point 2", "children":␣
 ↪["Sub-point 2.1"]}}]
# - For "table": "data": {{"headers": ["Col A"], "rows": [["Data 1"]]}}

# Now, generate the JSON for slide {current_slide_num}.
# """
CONTENT_SLIDE_GENERATION_PROMPT = """
You are an expert university lecturer and instructional designer. Your task is␣
 ↪to process a large block of raw text and intelligently divide it into a␣
 ↪sequence of PowerPoint slides.

**CONTEXT:**
- **Main Topic:** "{main_topic_title}"
- **The ENTIRE Raw Text for this Topic:** ```{topic_raw_content}```
- **Total Slides to Create:** You must cover all the key concepts in the raw␣
 ↪text over a total of **{total_slides_for_topic}** slides.
- **Current Task:** You are generating slide **{current_slide_num}** of␣
 ↪{total_slides_for_topic}.
- **Topics Already Covered on Previous Slides:** {generation_history}

**YOUR GOAL AND INSTRUCTIONS:**
1.  **Analyze the ENTIRE raw text.** Identify the distinct sub-topics within it.
```

```
2.  **Review the topics already covered** in the `generation_history`.
3.  **Identify the NEXT logical sub-topic from the raw text that has NOT been␣
 ↪covered yet.** Your primary goal is to ensure comprehensive coverage without␣
 ↪repetition.
4.  **Create the content for a single slide** that is focused *only* on this␣
 ↪new, uncovered sub-topic.
5.  Your output **MUST** be a single, well-formed JSON object. Do not add any␣
 ↪text or explanations outside the JSON object.

**JSON OUTPUT STRUCTURE & RULES:**
{{
  "title": "string | The main title for the slide, consistent with the main␣
 ↪topic.",
  "subtitle": "string | A more specific subtitle for this particular slide. Use␣
 ↪null if not needed.",
  "objects": [
    {{
      "content_type": "string | CHOOSE ONE: 'bullet_points', 'table'.",
      "content_purpose": "string | CHOOSE ONE: 'description', 'explanation',␣
 ↪'timeline', 'process', 'cycle', 'comparison', 'contrast', 'hierarchy',␣
 ↪'case_study', 'example'.",
      "data": "object | The structured data for the slide. See examples below."
    }}
  ]
}}

**DATA STRUCTURE EXAMPLES:**
- For "bullet_points": "data": ["Point 1", {{"text": "Point 2", "children":␣
 ↪["Sub-point 2.1"]}}]
- For "table": "data": {{"headers": ["Col A"], "rows": [["Data 1"]]}}

Now, generate the JSON for slide {current_slide_num}.
"""




INTERACTIVE_ACTIVITY_PROMPT = """
You are an engaging university lecturer creating an interactive slide to test␣
 ↪student understanding.

**CONTEXT:**
- **Topic for this Activity:** "{topic_title}"
- **Raw Text Content for this Topic:** ```{topic_raw_content}```
```

```
**INSTRUCTIONS:**
1.  Create a single, insightful multiple-choice question that assesses a key␣
 ↪concept from the provided raw text.
2.  The question must have exactly 4 options. Provide the correct answer and a␣
 ↪brief, clear explanation.
3.  Your output **MUST** be a single, well-formed JSON object that adheres␣
 ↪strictly to the structure below.

**JSON OUTPUT STRUCTURE:**
{{
  "title": "Let's Apply This!",
  "subtitle": "Knowledge Check: {topic_title}",
  "objects": [
    {{
      "content_type": "multiple_choice_question",
      "content_purpose": "knowledge_check",
      "data": {{
        "question_text": "string | The question to be asked.",
        "options": [
          {{"label": "A", "text": "string"}},
          {{"label": "B", "text": "string"}},
          {{"label": "C", "text": "string"}},
          {{"label": "D", "text": "string"}}
        ],
        "correct_answer": {{
          "label": "string | e.g., 'B'",
          "explanation": "string | A brief explanation of why this answer is␣
 ↪correct."
        }}
      }}
    }}
  ]
}}

Now, generate the JSON for the interactive activity.
"""

# ================================================================================
# NEW, CONTEXT-AWARE SUMMARY GENERATION PROMPT
# This prompt provides the LLM with the content of all slides for a rich␣
 ↪summary.
# ================================================================================

SUMMARY_GENERATION_PROMPT = """
You are an expert university lecturer crafting the final, conclusive summary␣
 ↪slide for a lecture.
```

```
**CONTEXT:**
- **Overall Lecture Topic:** "{lecture_topic}"
- **Detailed Content of Slides Presented:**
```json
{slide_content_details}
INSTRUCTIONS:
Analyze the detailed content of all the slides provided in the JSON context.
Synthesize the most critical points into 3-5 concise, high-level takeaways. Do␣
 ↪not just list the subtitles. Capture the core lesson of each major section.
Your output MUST be a single, well-formed JSON object that adheres to the␣
 ↪standard slide structure below. This ensures it can be rendered correctly.

JSON OUTPUT STRUCTURE:
{{
"title": "Summary & Key Takeaways",
"subtitle": null,
"objects": [
{{
  "content_type": "bullet_points",
  "content_purpose": "description",
  "data": [
  "string | Key takeaway 1.",
  "string | Key takeaway 2.",
  "string | Key takeaway 3.",
  "string | Key takeaway 4.",
  "string | Key takeaway 5."
]
}}
]
}}
Now, generate the JSON for the final summary slide.
"""
```

[45]:
```
# Content Generator


"""
Imporvement to avoid repetition split the data provide? --> search sentences␣
 ↪or by paragraps
To create more interactive activities could run a prohram to ramdom select the␣
 ↪prompt or 2 steps analysis - check data then select the prompt to run / more␣
 ↪capable llm probably handle repetition
make stronger structure retriver to secure structure for presenter
"""
```
```

```python
import os
import json
import re
import logging
from typing import List, Dict, Any, Optional

# Third-party imports (ensure these are installed)
try:
    import ollama
    from tenacity import retry, stop_after_attempt, wait_exponential
except ImportError:
    print("Please install required libraries: pip install ollama tenacity")
    ollama = None

# Basic Logger
logging.basicConfig(level=logging.INFO, format='%(asctime)s - %(levelname)s - %(message)s')
logger = logging.getLogger(__name__)


class ContentAgent:
    """
    Processes a hierarchical content plan, using specialized LLM prompts to generate
    a sequence of individual, richly-structured slides.
    """
    def __init__(self, master_config: Dict, vector_store: Optional[Any] = None):
        self.config = master_config['processed_settings']
        self.unit_outline = master_config['unit_outline']
        self.book_toc = master_config['book_toc']
        #self.teaching_flows = master_config['teaching_flows']
        #self.layaut_slides = master_config['layaut_slides']
        self.vector_store = vector_store
        self.client = ollama.Client(host=master_config.get('OLLAMA_HOST', 'http://localhost:11434'))
        self.model = master_config.get('OLLAMA_MODEL', 'qwen3:8b')
        # --- Added for Progress Tracking ---
        self.total_slides_to_generate = 0
        self.llm_calls_made = 0
        logger.info(f"Data-Driven Content Agent initialized with model '{self.model}'.")



    # Define the desired key order as a class attribute
        self.key_order = [
```

```
            "title", "toc_id",
            "chunk_count",
            "total_chunks_in_branch",
            "budget_slides_content",
            "direct_slides_content",
            "total_slides_in_branch",
            "time_allocation_minutes",
            "chunks_sorted",
            "content",
            "llm_generated_content",
            "children",
            "interactive_activity"
        ]

        logger.info("Data-Driven Content Agent initialized successfully.")

    # --- Key Reordering Logic (REFINED) ---
    def _reorder_keys_recursively(self, node: dict) -> dict:
        """
        Recursively traverses a dictionary (a node in the plan) and reorders␣
↪its keys
        according to a predefined order for maximum readability.
        """
        if not isinstance(node, dict):
            return node

        # 1. Recurse first to ensure nested structures are already ordered.
        if 'children' in node and isinstance(node.get('children'), list):
            node['children'] = [self._reorder_keys_recursively(child) for child␣
↪in node['children']]

        if 'interactive_activity' in node and isinstance(node.
↪get('interactive_activity'), dict):
            node['interactive_activity'] = self.
↪_reorder_keys_recursively(node['interactive_activity'])

        # 2. Build a new dictionary for the current node with the correct key␣
↪order.
        reordered_node = {}

        # Add keys that are in our desired order
        for key in self.key_order:
            if key in node:
                reordered_node[key] = node[key]

        # Add any remaining keys that were not in the order list (as a fallback)
        for key, value in node.items():
```

```
            if key not in reordered_node:
                reordered_node[key] = value

        return reordered_node



    # --- Phase 5: Raw Content Population ---
    def retrieve_content_for_toc_id(self, toc_id: int) -> dict:
        if not isinstance(toc_id, int):
            logger.warning(f"Invalid toc_id: {toc_id}. Must be an integer.")
            return {"chunks_sorted": [], "content": ""}
        try:
            results = self.vector_store.get(where={"toc_id": toc_id},␣
↪include=["documents", "metadatas"])
            if not results or not results.get('ids'):
                logger.warning(f"No chunks found in the database for toc_id =␣
↪{toc_id}")
                return {"chunks_sorted": [], "content": ""}
            sorted_items = sorted(zip(results['documents'],␣
↪results['metadatas']), key=lambda item: item[1].get('chunk_id', 0))
            sorted_docs = [item[0] for item in sorted_items]
            sorted_chunk_ids = [item[1].get('chunk_id') for item in␣
↪sorted_items]
            reassembled_text = "\n\n".join(sorted_docs)
            return {"chunks_sorted": sorted_chunk_ids, "content":␣
↪reassembled_text}
        except Exception as e:
            logger.error(f"An error occurred during retrieval for toc_id␣
↪{toc_id}: {e}", exc_info=True)
            return {"chunks_sorted": [], "content": ""}

    def populate_content_recursively(self, node: dict):

        if 'toc_id' in node and 'content' not in node:
            content_data = self.retrieve_content_for_toc_id(node['toc_id'])
            node.update(content_data)
        if 'children' in node and isinstance(node.get('children'), list):
            for child in node['children']:
                self.populate_content_recursively(child)

    def generate_content_for_plan(self, final_plan_path: str, output_dir: str)␣
↪-> bool:

        logger.info(f"PHASE 5: Populating raw content for: {final_plan_path}")
        try:
```

```
            with open(final_plan_path, 'r', encoding='utf-8') as f:
                plan_data = json.load(f)
        except (FileNotFoundError, json.JSONDecodeError) as e:
            logger.error(f"FATAL: Could not read or decode plan file␣
↪{final_plan_path}. Error: {e}")
            return False

        for deck in plan_data.get('deck_plans', []):
            for section in deck.get('sections', []):
                if section.get('section_type') == 'Content':
                    for content_block in section.get('content_blocks', []):
                        self.populate_content_recursively(content_block)

        base_filename = os.path.basename(final_plan_path)
        output_path = os.path.join(output_dir, base_filename)
        os.makedirs(output_dir, exist_ok=True)

        logger.info("Reordering keys for Fetched clean output...")
        fetched_ordered_plan = self._reorder_keys_recursively(plan_data)

        try:
            with open(output_path, 'w', encoding='utf-8') as f:
                # CORRECTED: Save the 'fetched_ordered_plan' variable
                json.dump(fetched_ordered_plan, f, indent=2, ensure_ascii=False)
            logger.info(f"Successfully saved content-enriched plan to:␣
↪{output_path}")
            return True
        except Exception as e:
            logger.error(f"Failed to save the content-enriched plan to␣
↪{output_path}: {e}", exc_info=True)
            return False

    # --- Phase 6: LLM Content Generation & Final Formatting ---

    @retry(stop=stop_after_attempt(3), wait=wait_exponential(min=2, max=10))
    def _call_ollama_with_retry(self, prompt: str) -> str:
        # --- Incremented Counter Here ---
        self.llm_calls_made += 1
        logger.info(f"Calling Ollama model '{self.model}' (Call #{self.
↪llm_calls_made})...")
        response = self.client.chat(model=self.model, messages=[{"role":␣
↪"user", "content": prompt}], format="json", options={"temperature": 0.2})
        if not response or 'message' not in response or not response['message'].
↪get('content'):
            raise ValueError("Ollama returned an empty or invalid response.")
        return response['message']['content']
```

```
    def _parse_llm_json_output(self, content: str) -> Optional[Dict]:
        try:
            match = re.search(r'\{.*\}', content, re.DOTALL)
            if not match: return None
            return json.loads(match.group(0))
        except (json.JSONDecodeError, TypeError): return None

    def _process_content_node_recursively(self, node: dict):
        """
        Recursively processes a content node by generating its own slides,
        processing its children, and then generating its interactive activity.
        This method modifies the 'node' dictionary in-place.
        """
        num_slides_to_gen = node.get('direct_slides_content', 0)

        if num_slides_to_gen > 0:
            generated_slides = []
            generation_history = "None. This is the first slide."
            for i in range(num_slides_to_gen):
                # --- Updated Progress Log ---
                progress_log = f"[Progress: {self.llm_calls_made}/{self.
↪total_slides_to_generate}]"
                logger.info(f"{progress_log} Generating slide {i+1}/
↪{num_slides_to_gen} for topic: '{node.get('title')}'")
                try:
                    prompt = CONTENT_SLIDE_GENERATION_PROMPT.format(
                        main_topic_title=node.get('title'),
                        topic_raw_content=node.get('content', ''),
                        current_slide_num=i + 1,
                        total_slides_for_topic=num_slides_to_gen,
                        generation_history=generation_history
                    )
                    llm_str = self._call_ollama_with_retry(prompt)
                    slide_content = self._parse_llm_json_output(llm_str)
                    if slide_content:
                        slide_object = {
                            "seq_id": float(f"{node.get('seq_id', 0)}.{i + 1}"),
                            "llm_generated_content": slide_content
                        }
                        generated_slides.append(slide_object)
                        generation_history += f"\n- Slide {i+1} subtitle:␣
↪{slide_content.get('subtitle', 'N/A')}"
                except Exception as e:
                    logger.error(f"LLM call failed for slide {i+1} of '{node.
↪get('title')}': {e}")
```

```
            node['slides'] = generated_slides

        for child_node in node.get('children', []):
            self._process_content_node_recursively(child_node)

        if 'interactive_activity' in node:
            # --- Updated Progress Log for Interactive Slide ---
            progress_log = f"[Progress: {self.llm_calls_made}/{self.
↪total_slides_to_generate}]"
            logger.info(f"{progress_log} Generating interactive slide for topic:
↪ '{node.get('title')}'")
            try:
                prompt = INTERACTIVE_ACTIVITY_PROMPT.format(
                    topic_title=node.get('title'),
                    topic_raw_content=node.get('content', '')
                )
                llm_str = self._call_ollama_with_retry(prompt)
                interactive_content = self._parse_llm_json_output(llm_str)
                if interactive_content:
                    node['interactive_activity']['llm_generated_content'] =␣
↪interactive_content
            except Exception as e:
                logger.error(f"LLM call failed for interactive activity on␣
↪'{node.get('title')}': {e}")

    def _count_total_slides(self, plan_data: Dict):
        """Recursively counts the total number of slides to be generated."""
        count = 0
        for deck in plan_data.get('deck_plans', []):
            # Count summary slide
            if any(s.get('section_type') == 'Summary' for s in deck.
↪get('sections', [])):
                count += 1

            # Count content slides
            for section in deck.get('sections', []):
                if section.get('section_type') == 'Content':
                    for content_block in section.get('content_blocks', []):

                        def _recursive_count(node):
                            nonlocal count
                            count += node.get('direct_slides_content', 0)
                            if 'interactive_activity' in node:
                                count += 1
                            for child in node.get('children', []):
                                _recursive_count(child)
```

```
                        _recursive_count(content_block)
        return count

    def generate_llm_content_for_plan(self, content_plan_path: str,␣
↪llm_output_dir: str) -> bool:
        """
        Orchestrates the LLM content generation for the entire plan file,
        including a rich, context-aware summary.
        """
        logger.info(f" PHASE 6: Generating LLM content for: {os.path.
↪basename(content_plan_path)} output file path: {llm_output_dir}")
        try:
            with open(content_plan_path, 'r', encoding='utf-8') as f:
                plan_data = json.load(f)
        except (FileNotFoundError, json.JSONDecodeError) as e:
            logger.error(f"FATAL: Could not read plan file {content_plan_path}.␣
↪Error: {e}")
            return False

        # --- Pre-calculate total slides for progress tracking ---
        self.total_slides_to_generate = self._count_total_slides(plan_data)
        self.llm_calls_made = 0 # Reset counter for the run
        logger.info(f"Found a total of {self.total_slides_to_generate} slides␣
↪to generate across all decks.")

        summary_prompt_template = SUMMARY_GENERATION_PROMPT

        for deck in plan_data.get('deck_plans', []):
            all_slide_contents_for_summary = []

            for section in deck.get('sections', []):
                if section.get('section_type') == 'Content':
                    for content_block in section.get('content_blocks', []):
                        self._process_content_node_recursively(content_block)

            for section in deck.get('sections', []):
                 if section.get('section_type') == 'Content':
                    for content_block in section.get('content_blocks', []):
                        def _collect_slide_content(node):
                            for slide in node.get('slides', []):
                                all_slide_contents_for_summary.append(slide.
↪get('llm_generated_content'))
                            if 'interactive_activity' in node and␣
↪'llm_generated_content' in node['interactive_activity']:
                                all_slide_contents_for_summary.
↪append(node['interactive_activity'].get('llm_generated_content'))
```

```
                            for child in node.get('children', []):
                                _collect_slide_content(child)
                        _collect_slide_content(content_block)

            if summary_prompt_template and all_slide_contents_for_summary:
                summary_section = next((s for s in deck['sections'] if s.
↪get('section_type') == 'Summary'), None)

                if summary_section:
                    # --- Updated Progress Log for Summary Slide ---
                    progress_log = f"[Progress: {self.llm_calls_made}/{self.
↪total_slides_to_generate}]"
                    logger.info(f"{progress_log} Generating summary for Deck␣
↪{deck.get('deck_number', 'N/A')}...")
                    try:
                        slide_details_str = json.
↪dumps(all_slide_contents_for_summary, indent=2)
                        prompt = summary_prompt_template.format(
                            lecture_topic=plan_data.get('overall_topic', 'This␣
↪Lecture'),
                            slide_content_details=slide_details_str
                        )
                        llm_str = self._call_ollama_with_retry(prompt)
                        summary_content = self._parse_llm_json_output(llm_str)
                        if summary_content:
                            summary_section['llm_generated_content'] =␣
↪summary_content
                        else:
                            logger.warning("Failed to parse summary content␣
↪from LLM.")
                            summary_section['llm_generated_content'] = {"title":
↪ "Summary & Key Takeaways", "subtitle": None, "objects": [{"content_type":␣
↪"bullet_points", "content_purpose": "description", "data": ["Summary could␣
↪not be generated."]}]}
                    except Exception as e:
                        logger.error(f"LLM call failed for deck summary: {e}",␣
↪exc_info=True)

        base_filename = os.path.basename(content_plan_path).replace('.json',␣
↪'_llm_generated.json')
        output_path = os.path.join(llm_output_dir, base_filename)
        os.makedirs(llm_output_dir, exist_ok=True)
        try:
            with open(output_path, 'w', encoding='utf-8') as f:
                json.dump(plan_data, f, indent=2, ensure_ascii=False)
```

```
            logger.info(f"Successfully saved final LLM-enriched plan to:␣
 ↪{output_path}")
            return True
        except Exception as e:
            logger.error(f"Failed to save the final LLM-enriched plan: {e}",␣
 ↪exc_info=True)
            return False
# ============================================================================
# TEST EXECUTION BLOCK
# ============================================================================
# if __name__ == '__main__' and ollama is not None:
#     print_header("Content Agent Test Runner", char="=")

#     # --- 1. Simulate Environment and Paths ---
#     # These would normally come from Cell 1 in your notebook
#     TEST_BASE_DIR = "/home/sebas_dev_linux/projects/course_generator/
 ↪test_output"
#     CONTENT_OUTPUT_DIR = os.path.join(TEST_BASE_DIR, "generated_content")
#     CONTENT_LLM_OUTPUT_DIR = os.path.join(TEST_BASE_DIR,␣
 ↪"generated_content_llm")
#     os.makedirs(CONTENT_OUTPUT_DIR, exist_ok=True)
#     os.makedirs(CONTENT_LLM_OUTPUT_DIR, exist_ok=True)

#     # --- 2. Create a Dummy Phase 5 Input File ---
#     # This simulates the file that the ContentAgent would read.
#     dummy_input_content = {
#       "week": 1,
#       "overall_topic": "Understanding the Digital Forensics Profession and␣
 ↪Investigations.",
#       "deck_plans": [
#         {
#           "sections": [
#             { "section_type": "Title",
#               "seq_id": 0,
#               "content": {}
#             },
#             { "section_type": "Agenda", "seq_id": 1, "content": {} },
#             {
#               "section_type": "Content",
#               "content_blocks": [
#                 {
#                   "title": "An Overview of Digital Forensics",
#                   "toc_id": 9,
#                   "seq_id": 2,
#                   "direct_slides_content": 2, # Requesting 2 slides for this␣
 ↪topic
```

```
#                     "content": "Digital forensics is the application of␣
 ↪computer science... It involves analyzing digital evidence... The Fourth␣
 ↪Amendment protects against unreasonable searches..."
#                   },
#                   {
#                     "title": "A Brief History of Digital Forensics",
#                     "toc_id": 11,
#                     "seq_id": 5,
#                     "direct_slides_content": 1, # Requesting 1 slide for this␣
 ↪topic
#                     "content": "The history begins in the 1970s with mainframe␣
 ↪crimes... In the 1980s, PC use grew... Specialized tools like EnCase emerged␣
 ↪in the 90s..."
#                   }
#                 ]
#               },
#               { "section_type": "Summary", "seq_id": 12, "content": {} }
#             ]
#           }
#         ]
#     }
#     dummy_input_path = os.path.join(CONTENT_OUTPUT_DIR,␣
 ↪"Week1_plan_content_enriched.json")
#     with open(dummy_input_path, 'w', encoding='utf-8') as f:
#         json.dump(dummy_input_content, f, indent=2)
#     logger.info(f"Created dummy input file at: {dummy_input_path}")

#     # --- 3. Set up a Dummy Master Configuration ---
#     master_config = {
#         "OLLAMA_HOST": "http://localhost:11434",
#         "OLLAMA_MODEL": "mistral:latest",#"deepseek-r1:8b", #"qwen3:8b", # Or␣
 ↪"qwen3:8b", etc.
#         "processed_settings": {"teaching_flow_id": "standard_lecture"},
#         "teaching_flows": {
#             "standard_lecture": {
#                 "prompts": {
#                     "summary_generation": "Summarize these topics:␣
 ↪{topic_list}"
#                 }
#             }
#         }
#     }
#     CONTENT_TO_PROCESS = '/home/sebas_dev_linux/projects/course_generator/
 ↪test_output/generated_content/test_generation_before_LLM.json'
#     # --- 4. Instantiate and Run the Agent ---
#     try:
```

```
#         content_agent = ContentAgent(master_config)

#         # Run the agent in test_mode to only process the first deck
#         success = content_agent.generate_llm_content_for_plan(
#             content_plan_path=CONTENT_TO_PROCESS,
#             llm_output_dir=CONTENT_LLM_OUTPUT_DIR,

#         )

#         if success:
#             logger.info(" Test run completed successfully!")
#             logger.info(f"Check the output file in the␣
 ↪'{CONTENT_LLM_OUTPUT_DIR}' directory.")
#         else:
#             logger.error(" Test run failed.")

#     except NameError:
#         logger.error("Ollama library not found. Skipping test execution.")
#     except Exception as e:
#         logger.error(f"An unexpected error occurred during the test run:␣
 ↪{e}", exc_info=True)
```

7.3 Presentation Agent

test Presenter

we need to ensure the presenter seq_id flow: Parent -> Child 1 -> Child 1's Activity -> Child 2 -> Child 2's Activity -> Parent's Activity.

https://aistudio.google.com/prompts/18YaU5pG96eFMbM1l6yeG1v9j63PkLc5H

[41]:
```
# PresentationAgent


# import os
# import json
# import logging
# import random
# from pptx import Presentation
# from pptx.util import Inches
# from pptx.enum.shapes import PP_PLACEHOLDER
# from pptx.enum.text import MSO_AUTO_SIZE

# # --- Basic Setup ---
# logging.basicConfig(level=logging.INFO, format='%(asctime)s - %(levelname)s -␣
 ↪%(message)s')
# logger = logging.getLogger(__name__)
```

```
# # --- Helper Functions (Unchanged) ---
# def _render_bullet_points(text_frame, data):
#     text_frame.clear(); text_frame.auto_size = MSO_AUTO_SIZE.TEXT_TO_FIT_SHAPE
#     def add_points(points, level):
#         for point in points:
#             if isinstance(point, dict):
#                 p = text_frame.add_paragraph(); p.text = point.get('text',␣
 ↪''); p.level = level
#                 if 'children' in point: add_points(point['children'], level +␣
 ↪1)
#             else:
#                 p = text_frame.add_paragraph(); p.text = str(point); p.level␣
 ↪= level
#     add_points(data, 0)
#     if text_frame.paragraphs and not text_frame.paragraphs[0].text:
#         p = text_frame.paragraphs[0]; p._p.getparent().remove(p._p)

# def _render_table(slide, placeholder, data):
#     headers, rows_data = data.get('headers', []), data.get('rows', [])
#     if not headers or not rows_data: return
#     num_rows, num_cols = len(rows_data) + 1, len(headers)
#     table_shape = slide.shapes.add_table(num_rows, num_cols, placeholder.
 ↪left, placeholder.top, placeholder.width, placeholder.height)
#     table = table_shape.table
#     for i, header in enumerate(headers):
#         cell = table.cell(0, i); cell.text = header
#         cell.text_frame.auto_size = MSO_AUTO_SIZE.TEXT_TO_FIT_SHAPE; cell.
 ↪text_frame.paragraphs[0].font.bold = True
#     for r_idx, row_data in enumerate(rows_data):
#         for c_idx, cell_text in enumerate(row_data):
#             cell = table.cell(r_idx + 1, c_idx); cell.text = str(cell_text)
#             cell.text_frame.auto_size = MSO_AUTO_SIZE.TEXT_TO_FIT_SHAPE
#     sp = placeholder.element; sp.getparent().remove(sp)

# def _render_multiple_choice_question(slide, placeholders, data):
#     main_ph = next((p for p in placeholders if p.placeholder_format.type ==␣
 ↪PP_PLACEHOLDER.OBJECT), None)
#     answer_ph = next((p for p in placeholders if p.placeholder_format.type ==␣
 ↪PP_PLACEHOLDER.BODY), None)
#     if not main_ph: return
#     tf = main_ph.text_frame; tf.clear(); tf.auto_size = MSO_AUTO_SIZE.
 ↪TEXT_TO_FIT_SHAPE
#     p = tf.add_paragraph(); p.text = data.get('question_text', ''); p.font.
 ↪bold = True; p.level = 0
#     for option in data.get('options', []):
```

```
#         p = tf.add_paragraph(); p.text = f"{option.get('label', '')}) {option.
 ↪get('text', '')}"; p.level = 1
#     if answer_ph:
#         answer_ph.text_frame.clear()
#         explanation = data.get('correct_answer', {}).get('explanation', '')
#         answer_ph.text_frame.text = f"Answer: {data.get('correct_answer', {}).
 ↪get('label', '')}. {explanation}"

# def _render_matching_activity(slide, placeholders, data):
#     object_placeholders = [p for p in placeholders if p.placeholder_format.
 ↪type == PP_PLACEHOLDER.OBJECT]
#     if len(object_placeholders) < 2: return
#     terms_ph, defs_ph = object_placeholders[0], object_placeholders[1]
#     terms_tf, defs_tf = terms_ph.text_frame, defs_ph.text_frame
#     terms_tf.clear(); defs_tf.clear(); terms_tf.auto_size = MSO_AUTO_SIZE.
 ↪TEXT_TO_FIT_SHAPE; defs_tf.auto_size = MSO_AUTO_SIZE.TEXT_TO_FIT_SHAPE
#     pairs = data.get('pairs', [])
#     if not pairs: return
#     terms = [f"{i+1}. {p['term']}" for i, p in enumerate(pairs)]
#     definitions = [p['definition'] for p in pairs]
#     random.shuffle(definitions)
#     for term in terms: p = terms_tf.add_paragraph(); p.text = term; p.level =␣
 ↪0
#     for i, definition in enumerate(definitions): p = defs_tf.add_paragraph();␣
 ↪p.text = f"{chr(65+i)}. {definition}"; p.level = 0

# class PresentationAgent:
#     def __init__(self, template_path: str, layout_config_path: str):
#         self.template_path = template_path
#         self.prs_for_layouts = Presentation(template_path)
#         with open(layout_config_path, 'r', encoding='utf-8') as f:
#             self.user_selections = json.load(f)['user_selections']
#         logger.info(f"Agent initialized with template '{os.path.
 ↪basename(template_path)}'")

#     def _get_layout(self, layout_key: str):
#         selection = self.user_selections.get(layout_key, self.
 ↪user_selections['Content'])
#         layout_index = selection['selected_layout_index']
#         return self.prs_for_layouts.slide_layouts[layout_index]

#     def _determine_layout_key(self, item_data):
#         llm_content = item_data.get('llm_generated_content', {})
#         objects = llm_content.get('objects', [])
#         has_subtitle = bool(llm_content.get('subtitle'))
```

```
#         # <<< NEW LOGIC: Determine layout based on subtitles and object count
#         if objects and objects[0]['content_type'] in␣
 ↪['multiple_choice_question', 'matching_activity']:
#             return "Application_Two_Column" if len(objects) > 1 or␣
 ↪objects[0]['content_type'] == 'matching_activity' else "Application"

#         if len(objects) >= 2:
#             return "Content_Two_Column_child" if has_subtitle else␣
 ↪"Content_Two_Column"
#         else:
#             return "Content_child" if has_subtitle else "Content"

#     def create_presentation_from_plan(self, plan_json_path: str, output_path:␣
 ↪str):
#         logger.info(f"Creating presentation from plan: '{os.path.
 ↪basename(plan_json_path)}'")
#         with open(plan_json_path, 'r', encoding='utf-8') as f: plan_data =␣
 ↪json.load(f)
#         prs = Presentation(self.template_path)
#         while len(prs.slides):
#             rId = prs.slides._sldIdLst[0].rId; prs.part.drop_rel(rId); del␣
 ↪prs.slides._sldIdLst[0]
#         slide_items = self._collect_slide_items(plan_data)
#         for item in sorted(slide_items, key=lambda x: x.get('seq_id', 999)):␣
 ↪self._add_slide_for_item(prs, item)
#         prs.save(output_path)
#         logger.info(f"Successfully created presentation: '{output_path}'")

#     def _collect_slide_items(self, plan_data):
#         items = []
#         for deck in plan_data.get('deck_plans', []):
#             for section in deck.get('sections', []):
#                 if section.get('section_type') == 'Content':
#                     for block in section.get('content_blocks', []):
#                         for slide_item in block.get('slides', []):
#                             items.append({'item_type': 'Content', 'data':␣
 ↪slide_item})
#                 else:
#                     items.append({'item_type': section['section_type'],␣
 ↪'data': section})
#         return items

#     def _add_slide_for_item(self, prs: Presentation, item: dict):
#         item_type = item['item_type']
#         layout_key = self._determine_layout_key(item['data']) if item_type ==␣
 ↪'Content' else item_type
```

```
#         slide = prs.slides.add_slide(self._get_layout(layout_key))
#         render_map = {
#             'Title': self._render_title_slide, 'Agenda': self.
 ↪_render_agenda_slide,
#             'Summary': self._render_summary_slide, 'End': self.
 ↪_render_end_slide,
#             'Divider': self._render_divider_slide, 'Content': self.
 ↪_render_content_slide
#         }
#         if item_type in render_map:
#             render_map[item_type](slide, item['data'])

#     def _render_title_slide(self, slide, data):
#         content = data.get('content', {})
#         if slide.shapes.title:
#             slide.shapes.title.text = content.get('week_topic', 'Untitled␣
 ↪Topic')
#         subtitle_ph = next((p for p in slide.placeholders if p.
 ↪placeholder_format.type == PP_PLACEHOLDER.SUBTITLE), None)
#         if subtitle_ph:
#             subtitle_ph.text = f"{content.get('deck_title', '')}\n{content.
 ↪get('unit_name', '')} - {content.get('unit_code', '')}"

#     def _render_agenda_slide(self, slide, data):
#         content = data.get('content', {})
#         if slide.shapes.title: slide.shapes.title.text = content.get('title',␣
 ↪'Agenda')
#         body_ph = next((p for p in slide.placeholders if p.placeholder_format.
 ↪type == PP_PLACEHOLDER.OBJECT), None)
#         if body_ph: _render_bullet_points(body_ph.text_frame, content.
 ↪get('items', []))

#     def _render_summary_slide(self, slide, data):
#         if slide.shapes.title: slide.shapes.title.text = "Summary & Key␣
 ↪Takeaways"
#         if 'llm_generated_content' in data: self._render_content_slide(slide,␣
 ↪data)

#     def _render_end_slide(self, slide, data):
#         content = data.get('content', {})
#         if slide.shapes.title: slide.shapes.title.text = content.get('text',␣
 ↪'Questions?')

#     def _render_divider_slide(self, slide, data):
#         content = data.get('content', {})
```

```
#         if slide.shapes.title: slide.shapes.title.text = content.get('title',␣
 ↪'Section Divider')

#     def _render_content_slide(self, slide, data):
#         content = data.get('llm_generated_content', {})

#         # <<< NEW LOGIC: Populate title and subtitle in separate placeholders
#         if slide.shapes.title:
#             slide.shapes.title.text = content.get('title', '')
#             slide.shapes.title.text_frame.auto_size = MSO_AUTO_SIZE.
 ↪TEXT_TO_FIT_SHAPE

#         subtitle_ph = next((p for p in slide.placeholders if p.
 ↪placeholder_format.type == PP_PLACEHOLDER.SUBTITLE), None)
#         if subtitle_ph:
#             subtitle_ph.text = content.get('subtitle', '')
#             subtitle_ph.text_frame.auto_size = MSO_AUTO_SIZE.TEXT_TO_FIT_SHAPE

#         body_placeholders = sorted([p for p in slide.placeholders if p.
 ↪placeholder_format.type not in [PP_PLACEHOLDER.TITLE, PP_PLACEHOLDER.
 ↪SUBTITLE]], key=lambda p: p.left)
#         objects = content.get('objects', [])

#         for obj, placeholder in zip(objects, body_placeholders):
#             content_type, obj_data = obj.get('content_type'), obj.get('data')
#             renderers = {
#                 'bullet_points': lambda: _render_bullet_points(placeholder.
 ↪text_frame, obj_data),
#                 'table': lambda: _render_table(slide, placeholder, obj_data),
#                 'multiple_choice_question': lambda:␣
 ↪_render_multiple_choice_question(slide, body_placeholders, obj_data),
#                 'matching_activity': lambda: _render_matching_activity(slide,␣
 ↪body_placeholders, obj_data)
#             }
#             if content_type in renderers:
#                 renderers[content_type]()
#                 if content_type in ['multiple_choice_question',␣
 ↪'matching_activity']: break

import os
import json
import logging
import random
from pptx import Presentation
from pptx.util import Inches, Pt
from pptx.enum.shapes import PP_PLACEHOLDER_TYPE, MSO_SHAPE_TYPE
```

```
from pptx.enum.text import MSO_AUTO_SIZE

# --- Basic Setup ---
logging.basicConfig(level=logging.INFO, format='%(asctime)s - %(levelname)s -␣
 ↪%(message)s')
logger = logging.getLogger(__name__)

# --- Helper function to find specific placeholders ---
def _get_placeholder(slide, ph_type):
    for shape in slide.placeholders:
        if shape.placeholder_format.type == ph_type:
            return shape
    return None

# --- Rendering Helper Functions ---
def _render_bullet_points(text_frame, data):
    text_frame.clear(); text_frame.auto_size = MSO_AUTO_SIZE.TEXT_TO_FIT_SHAPE
    def add_points(points, level):
        for point in points:
            if isinstance(point, dict):
                p = text_frame.add_paragraph(); p.text = point.get('text', '');␣
 ↪p.level = level
                if 'children' in point: add_points(point['children'], level + 1)
            else:
                p = text_frame.add_paragraph(); p.text = str(point); p.level =␣
 ↪level
    if isinstance(data, list): add_points(data, 0)
    if text_frame.paragraphs and not text_frame.paragraphs[0].text:
        p = text_frame.paragraphs[0]; p._p.getparent().remove(p._p)

def _render_table(slide, placeholder, data):
    headers, rows_data = data.get('headers', []), data.get('rows', [])
    if not headers or not rows_data: return
    table_shape = slide.shapes.add_table(len(rows_data) + 1, len(headers),␣
 ↪placeholder.left, placeholder.top, placeholder.width, placeholder.height)
    table = table_shape.table
    for i, header in enumerate(headers):
        cell = table.cell(0, i); cell.text = header; cell.text_frame.
 ↪paragraphs[0].font.bold = True
    for r_idx, row_data in enumerate(rows_data):
        for c_idx, cell_text in enumerate(row_data):
            table.cell(r_idx + 1, c_idx).text = str(cell_text)
    sp = placeholder.element; sp.getparent().remove(sp)

def _render_mcq(placeholder, data, answer_placeholder=None):
    """
    Renders the MCQ question into the main placeholder and the detailed
```

```
    answer with explanation into the dedicated answer_placeholder.
    """
    # --- Part 1: Render the Question and Options ---
    tf = placeholder.text_frame
    tf.clear()
    tf.auto_size = MSO_AUTO_SIZE.TEXT_TO_FIT_SHAPE

    # Add the question text
    p_question = tf.add_paragraph()
    p_question.text = data.get('question_text', '')
    p_question.font.bold = True

    # Add the options
    for option in data.get('options', []):
        p_option = tf.add_paragraph()
        p_option.text = f"{option.get('label', '')}) {option.get('text', '')}"
        p_option.level = 1

    # --- Part 2: Render the Detailed Answer and Explanation ---
    if answer_placeholder:
        answer_info = data.get('correct_answer', {})
        answer_label = answer_info.get('label', 'N/A')
        answer_explanation = answer_info.get('explanation', 'No explanation␣
↪provided.')

        # Get the text frame of the answer placeholder
        answer_tf = answer_placeholder.text_frame
        answer_tf.clear()

        # Add "Correct Answer: [Label]" in bold
        p_answer_label = answer_tf.add_paragraph()
        p_answer_label.text = f" Answer: {answer_label} - {answer_explanation}"
        #p_answer_label.font.bold = True

        # # Add the explanation text on the next line
        # p_answer_explanation = answer_tf.add_paragraph()
        # p_answer_explanation.text = answer_explanation

def _render_matching(placeholders, data):
    """Renders a matching activity into a pair of placeholders."""
    if len(placeholders) < 2: return
    terms_ph, defs_ph = placeholders[0], placeholders[1]
    terms_tf, defs_tf = terms_ph.text_frame, defs_ph.text_frame
    terms_tf.clear(); defs_tf.clear()
    pairs = data.get('pairs', [])
    if not pairs: return
    terms = [f"{i+1}. {p['term']}" for i, p in enumerate(pairs)]
```

```
    definitions = [p['definition'] for p in pairs]
    random.shuffle(definitions)
    for term in terms: terms_tf.add_paragraph().text = term
    for i, definition in enumerate(definitions): defs_tf.add_paragraph().text =␣
 ↪f"{chr(65+i)}. {definition}"


def _render_text_with_smart_fit(text_frame, data):
    """
    Renders text and proactively adjusts the font size based on content length
    to prevent overflow.
    """
    text_frame.clear()
    # We still set this as a fallback for moderate cases
    text_frame.auto_size = MSO_AUTO_SIZE.TEXT_TO_FIT_SHAPE

    # --- Character Counting Logic ---
    # Convert the data (which can be a list of strings or dicts) into a single␣
 ↪string to measure its length.
    json_string = json.dumps(data)
    char_count = len(json_string)

    # --- Define Font Size Tiers ---
    # These values may need tuning based on your template's placeholder sizes␣
 ↪and fonts.
    # Format: (character_threshold, font_size_in_points)
    font_tiers = [
        (1200, Pt(16)), # Very long text -> smallest font
        (750,  Pt(18)), # Long text
        (450,  Pt(20)), # Medium text
        (0,    Pt(22))  # Short text -> default/largest font
    ]

    selected_font_size = None
    for threshold, size in font_tiers:
        if char_count >= threshold:
            selected_font_size = size
            break

    # --- Recursive Function to Add Points and Apply Font Size ---
    def add_points_with_font(points, level):
        for point in points:
            # Determine the text for the current point
            text_to_add = ""
            if isinstance(point, dict):
                text_to_add = point.get('text', '')
            else:
```

```
                text_to_add = str(point)

            # Add the paragraph and set its properties
            p = text_frame.add_paragraph()
            p.text = text_to_add
            p.level = level
            if selected_font_size:
                p.font.size = selected_font_size

            # Recurse for children if they exist
            if isinstance(point, dict) and 'children' in point:
                add_points_with_font(point['children'], level + 1)

    # Start the rendering process
    if isinstance(data, list):
        add_points_with_font(data, 0)

    # Clean up the initial empty paragraph if it exists and is unused
    if text_frame.paragraphs and not text_frame.paragraphs[0].text:
        p_to_remove = text_frame.paragraphs[0]
        p_to_remove._p.getparent().remove(p_to_remove._p)


class PresentationAgent:
    def __init__(self, template_path: str, layout_config_path: str):
        self.template_path = template_path
        self.prs_for_layouts = Presentation(template_path)
        with open(layout_config_path, 'r', encoding='utf-8') as f:
            self.user_selections = json.load(f)['user_selections']
        logger.info(f"Agent initialized with template '{os.path.
↪basename(template_path)}'")

    def _get_layout(self, layout_key: str):
        selection = self.user_selections.get(layout_key, self.
↪user_selections['Content'])
        return self.prs_for_layouts.
↪slide_layouts[selection['selected_layout_index']]

    def _collect_and_sort_slides(self, plan_data: dict) -> list:
        all_slide_items = []

        # This is the same recursive helper function, it is correct.
        def _recursive_harvester(node: dict, parent_title: str = None):
            is_child_node = bool(parent_title)
            for slide in node.get('slides', []):
                slide['parent_title'] = parent_title
                slide['is_child'] = is_child_node
```

```
                all_slide_items.append({'item_type': 'Content', 'data': slide})

            for child in node.get('children', []):
                # Pass the current node's title as the parent title for the␣
↪next level
                _recursive_harvester(child, parent_title=node.get('title'))

            # This part correctly harvests activities from ANY node (parent or␣
↪child)
            if 'interactive_activity' in node and 'llm_generated_content' in␣
↪node['interactive_activity']:
                activity = node['interactive_activity']
                activity['parent_title'] = parent_title
                # IMPORTANT: The 'is_child' status depends on whether a␣
↪parent_title was passed in.
                activity['is_child'] = is_child_node
                # We add the main node's title to the activity data so it can␣
↪be used if needed
                activity['main_topic_title'] = node.get('title')
                all_slide_items.append({'item_type': 'Content', 'data':␣
↪activity})

        # This is the main loop that processes the plan
        for deck in plan_data.get('deck_plans', []):
            # Framework slides are added as before
            framework_start = ([{'item_type': 'Title', 'data': s} for s in␣
↪deck['sections'] if s['section_type'] == 'Title'] +
                               [{'item_type': 'Agenda', 'data': s} for s in␣
↪deck['sections'] if s['section_type'] == 'Agenda'])
            all_slide_items.extend(framework_start)

            # --- MODIFICATION IS HERE ---
            # We now loop through the content blocks and call the harvester on␣
↪each one.
            content_sections = [s for s in deck['sections'] if␣
↪s['section_type'] == 'Content']
            if content_sections:
                for block in content_sections[0].get('content_blocks', []):
                    # We start the recursion for each main content block
                    # We pass the block's own title as the parent_title for its␣
↪direct activities
                    _recursive_harvester(block, parent_title=None) # Start with␣
↪no parent

            framework_end = ([{'item_type': 'Summary', 'data': s} for s in␣
↪deck['sections'] if s['section_type'] == 'Summary'] +
```

```
                            [{'item_type': 'End', 'data': s} for s in␣
↪deck['sections'] if s['section_type'] == 'End'])
            all_slide_items.extend(framework_end)

        return sorted(all_slide_items, key=lambda x: x['data'].get('seq_id',␣
↪999))

    def _determine_layout_key(self, slide_data: dict) -> str:
        # This logic is correct
        llm_content = slide_data.get('llm_generated_content', {})
        objects = llm_content.get('objects', [])
        if not objects: return "Content"
        first_obj_type = objects[0].get('content_type')
        if first_obj_type in ['multiple_choice_question', 'matching_activity']:
            return "Application_Two_Column " if first_obj_type ==␣
↪'matching_activity' else "Application"
        is_child = slide_data.get('is_child', False)
        if len(objects) >= 2:
            return "Content_Two_Column_child" if is_child else␣
↪"Content_Two_Column"
        return "Content_child" if is_child else "Content"

    def create_presentation_from_plan(self, plan_json_path: str, output_dir:␣
↪str):
        # This orchestrator is correct
        logger.info(f"Creating presentation from plan: '{os.path.
↪basename(plan_json_path)}'")
        with open(plan_json_path, 'r', encoding='utf-8') as f: plan_data = json.
↪load(f)
        final_slide_list = self._collect_and_sort_slides(plan_data)
        prs = Presentation(self.template_path)
        while len(prs.slides):
            rId = prs.slides._sldIdLst[0].rId; prs.part.drop_rel(rId); del prs.
↪slides._sldIdLst[0]
        logger.info(f"Collected a total of {len(final_slide_list)} slides to␣
↪render.")
        for item in final_slide_list:
            self._add_slide_for_item(prs, item)
        unit_code = plan_data.get('deck_plans', [{}])[0].get('sections',␣
↪[{}])[0].get('content', {}).get('unit_code', 'UNIT')
        week_num = plan_data.get('week', 'X')
        deck_num = plan_data.get('deck_plans', [{}])[0].get('deck_number', 'Y')
        output_filename =␣
↪f"{unit_code}_Week{week_num}_Deck{deck_num}_Presentation.pptx"
        output_path = os.path.join(output_dir, output_filename)
        prs.save(output_path)
```

```
        logger.info(f"Successfully created presentation: '{output_path}'")

    def _add_slide_for_item(self, prs: Presentation, item: dict):
        item_type = item['item_type']
        item_data = item['data']
        layout_key = self._determine_layout_key(item_data) if item_type ==␣
↪'Content' else item_type
        slide = prs.slides.add_slide(self._get_layout(layout_key))
        render_map = {
            'Title': self._render_title_slide, 'Agenda': self.
↪_render_agenda_slide,
            'Summary': self._render_summary_slide, 'End': self.
↪_render_end_slide,
            'Divider': self._render_divider_slide, 'Content': self.
↪_render_content_slide
        }
        if item_type in render_map: render_map[item_type](slide, item_data)

    def _render_title_slide(self, slide, data):
        content = data.get('content', {})
        title_ph = _get_placeholder(slide, PP_PLACEHOLDER_TYPE.CENTER_TITLE)
        if title_ph: title_ph.text = content.get('week_topic', '')
        subtitle_ph = _get_placeholder(slide, PP_PLACEHOLDER_TYPE.SUBTITLE)
        if subtitle_ph: subtitle_ph.text = f"{content.get('deck_title',␣
↪'')}\n{content.get('unit_name', '')} | {content.get('unit_code', '')}"

    def _render_agenda_slide(self, slide, data):
        content = data.get('content', {})
        title_ph = _get_placeholder(slide, PP_PLACEHOLDER_TYPE.TITLE)
        if title_ph: title_ph.text = content.get('title', 'Agenda')
        body_ph = _get_placeholder(slide, PP_PLACEHOLDER_TYPE.OBJECT)
        if body_ph: _render_bullet_points(body_ph.text_frame, content.
↪get('items', []))

    def _render_summary_slide(self, slide, data):
        """Directly renders the summary slide content without calling another␣
↪function."""
        title_ph = _get_placeholder(slide, PP_PLACEHOLDER_TYPE.TITLE)
        if title_ph:
            title_ph.text = "Summary & Key Takeaways"

        body_ph = _get_placeholder(slide, PP_PLACEHOLDER_TYPE.OBJECT)
        if not body_ph:
            logger.warning("Summary slide layout is missing a main body/object␣
↪placeholder.")
            return
```

```
        # The summary data is nested in the 'llm_generated_content' key
        content = data.get('llm_generated_content', {})
        if content and 'objects' in content and content['objects']:
            # The summary is always bullet points, so we can call the helper␣
↪directly.
            summary_data = content['objects'][0].get('data', [])
            _render_bullet_points(body_ph.text_frame, summary_data)

    def _render_divider_slide(self, slide, data):
        content = data.get('content', {})
        title_ph = _get_placeholder(slide, PP_PLACEHOLDER_TYPE.TITLE)
        if title_ph: title_ph.text = content.get('title', '')

    def _render_end_slide(self, slide, data):
        content = data.get('content', {})
        title_ph = _get_placeholder(slide, PP_PLACEHOLDER_TYPE.TITLE)
        if title_ph: title_ph.text = content.get('text', 'Thank You & Questions?
↪')

    def _render_summary_slide(self, slide, data):
        title_ph = _get_placeholder(slide, PP_PLACEHOLDER_TYPE.TITLE)
        if title_ph: title_ph.text = "Summary & Key Takeaways"
        body_ph = _get_placeholder(slide, PP_PLACEHOLDER_TYPE.OBJECT)
        if body_ph and 'llm_generated_content' in data:
            summary_data = data['llm_generated_content'].get('objects',␣
↪[{}])[0].get('data', [])
            _render_bullet_points(body_ph.text_frame, summary_data)

    # <<< THIS IS THE CORRECTED METHOD >>>
    def _render_content_slide(self, slide, data):
        content = data.get('llm_generated_content', {})
        if not content: return

        # --- Step 1: Determine Layout and Content ---
        layout_key = self._determine_layout_key(data)
        title_ph = _get_placeholder(slide, PP_PLACEHOLDER_TYPE.TITLE)
        subtitle_ph = _get_placeholder(slide, PP_PLACEHOLDER_TYPE.SUBTITLE)

        # --- Step 2: Handle Title and Subtitle Rendering ---
        # This logic now correctly covers all slide types.
        if "Application" in layout_key:
            # For ANY interactive slide, the main title is the specific topic.
            if title_ph:
                # The subtitle from the JSON is the most descriptive title.
                title_ph.text = content.get('subtitle', 'Knowledge Check')
            # The subtitle is the generic call to action.
```

```
            if subtitle_ph:
                subtitle_ph.text = content.get('title', "Let's Apply This!")

        elif data.get('is_child'):
            # For a standard child slide, use the parent-child hierarchy.
            if title_ph: title_ph.text = data.get('parent_title', '')
            if subtitle_ph: subtitle_ph.text = content.get('title', '')

        else:
            # For a standard parent slide.
            if title_ph: title_ph.text = content.get('title', '')
            if subtitle_ph: subtitle_ph.text = content.get('subtitle', '')

        # --- Step 3: Render Body Objects ---
        # Find the main placeholder for the question and the dedicated␣
↪placeholder for the answer.
        body_placeholders = sorted([p for p in slide.placeholders if p.
↪placeholder_format.type == PP_PLACEHOLDER_TYPE.OBJECT], key=lambda p: p.left)
        answer_placeholder = _get_placeholder(slide, PP_PLACEHOLDER_TYPE.BODY)
        objects = content.get('objects', [])

        for i, obj in enumerate(objects):
            if i >= len(body_placeholders): break
            placeholder = body_placeholders[i]
            content_type, obj_data = obj.get('content_type'), obj.get('data')

            # THIS IS THE CHANGE:
            if content_type in ['bullet_points', 'description', 'explanation',␣
↪'timeline', 'process', 'cycle', 'hierarchy', 'case_study', 'example']:
                _render_text_with_smart_fit(placeholder.text_frame, obj_data) #␣
↪Use the new function
            elif content_type == 'table':
                _render_table(slide, placeholder, obj_data)
            elif content_type == 'multiple_choice_question':
                _render_mcq(placeholder, obj_data, answer_placeholder)
                # The notes slide can still contain the detailed answer for␣
↪reference.
                if slide.has_notes_slide:
                    answer_info = obj_data.get('correct_answer', {})
                    slide.notes_slide.notes_text_frame.text = f"Answer:␣
↪{answer_info.get('label', '')}. {answer_info.get('explanation', '')}"

            elif content_type == 'matching_activity':
                _render_matching(body_placeholders, obj_data)
                break # Matching uses all placeholders.
```

```
            else: # All other content types
                _render_bullet_points(placeholder.text_frame, obj_data)



# # --- Main Execution Block ---  No delete no change it is ready for test the␣
 ↪outputfrom ContentAgent
# if __name__ == "__main__":
#     # Define project paths
#     PROJECT_BASE_DIR = "/home/sebas_dev_linux/projects/course_generator"
#     CONFIG_DIR = os.path.join(PROJECT_BASE_DIR, "configs")
#     DATA_DIR = os.path.join(PROJECT_BASE_DIR, "data")
#     # This is the INPUT directory for the final JSON plan
#     INPUT_CONTENT_DIR = os.path.join(PROJECT_BASE_DIR, "test_output",␣
 ↪"generated_content_llm")
#     # This is the OUTPUT directory for the final .pptx file
#     FINAL_PRESENTATION_DIR = os.path.join(PROJECT_BASE_DIR,␣
 ↪"final_presentations")

#     # Create directories if they don't exist
#     os.makedirs(CONFIG_DIR, exist_ok=True)
#     os.makedirs(DATA_DIR, exist_ok=True)
#     os.makedirs(INPUT_CONTENT_DIR, exist_ok=True)
#     os.makedirs(FINAL_PRESENTATION_DIR, exist_ok=True)

#     # --- Define file paths ---
#     LAYOUT_MAPPING_PATH = os.path.join(CONFIG_DIR, "layout_mapping_test_Mod.
 ↪json")
#     # The specific JSON file we want to process
#     TEST_CONTENT_PATH = os.path.join(INPUT_CONTENT_DIR,␣
 ↪"test_generation_before_LLM_llm_generated.json")
#     SLIDE_TEMPLATE_PATH = os.path.join(DATA_DIR, "slide_style/
 ↪slide_style_test_2.pptx")

#     print("--- Presentation Agent Test Runner (Final Version) ---")

#     # Check for required files
#     required_files = [SLIDE_TEMPLATE_PATH, LAYOUT_MAPPING_PATH,␣
 ↪TEST_CONTENT_PATH]
#     if not all(os.path.exists(p) for p in required_files):
#         logger.error("CRITICAL: One or more required files not found. Please␣
 ↪check paths:")
#         logger.error(f"  Template: {SLIDE_TEMPLATE_PATH} {'(Found)' if os.
 ↪path.exists(SLIDE_TEMPLATE_PATH) else '(MISSING)'}")
#         logger.error(f"  Layout Map: {LAYOUT_MAPPING_PATH} {'(Found)' if os.
 ↪path.exists(LAYOUT_MAPPING_PATH) else '(MISSING)'}")
```

```
#         logger.error(f"  Content Plan: {TEST_CONTENT_PATH} {'(Found)' if os.
 ↪path.exists(TEST_CONTENT_PATH) else '(MISSING)'}")
#     else:
#         try:
#             presentation_agent = PresentationAgent(
#                 template_path=SLIDE_TEMPLATE_PATH,
#                 layout_config_path=LAYOUT_MAPPING_PATH
#             )

#             # <<< THIS IS THE CORRECTED LINE >>>
#             # We pass the DIRECTORY where the output should be saved.
#             presentation_agent.create_presentation_from_plan(
#                 plan_json_path=TEST_CONTENT_PATH,
#                 output_dir=FINAL_PRESENTATION_DIR
#             )

#             logger.info("--- Test script finished successfully. ---")
#             logger.info(f"Check for the generated .pptx file in:␣
 ↪{FINAL_PRESENTATION_DIR}")

#         except Exception as e:
#             logger.error(f"An unexpected error occurred during the test run:␣
 ↪{e}", exc_info=True)
```

7.4 Orquestrator (Addressing pain points)

Description:

The main script that iterates through the weeks defined the plan and generate the content base on the settings_deck coordinating the agents.

Parameters and concideration - 1 hour in the setting session_time_duration_in_hour - is 18-20 slides at the time so it is require to calculate this according to the given value but this also means per session so sessions_per_week is a multiplicator factor that
- if apply_topic_interactive is available will add an extra slide and add extra 5 min time but to determine this is required to plan all the content first and then calculate then provide a extra time

settings_deck.json

{ “course_id”: ““,”unit_name”: ““,”interactive”: true, “interactive_deep”: false, “teaching_flow_id”: “Standard Lecture Flow”, “parameters_slides”: { “slides_per_hour”: 18, “time_per_content_slides_min”: 3, “time_per_interactive_slide_min”: 5, “time_for_framework_slides_min”: 6 }, “week_session_setup”: { “sessions_per_week”: 1, “distribution_strategy”: “even”, “session_time_duration_in_hour”: 2, “interactive_time_in_hour”: 0, “total_session_time_in_hours”: 0 }, “slide_count_strategy”: { “method”: “per_week”, “target_total_slides”: 0, “slides_content_per_session”: 0, “interactive_slides_per_week”: 0, “interactive_slides_per_session”: 0, “total_slides_deck_week”: 0, “total_slides_session”: 0
},
“generation_scope”: { “weeks”: [1] } }

teaching_flows.json

{ “standard_lecture”: { “name”: “Standard Lecture Flow”, “slide_types”: [“Title”, “Agenda”, “Content”, “Summary”, “End”], “prompts”: { “content_generation”: “You are an expert university lecturer. Your audience is undergraduate students. Based on the following context, create a slide that provides a detailed explanation of the topic ‘{sub_topic}’. The content should be structured with bullet points for key details. Your output MUST be a single JSON object with a ‘title’ (string) and ‘content’ (list of strings) key.”, “summary_generation”: “You are an expert university lecturer creating a summary slide. Based on the following list of topics covered in this session, generate a concise summary of the key takeaways. The topics are: {topic_list}. Your output MUST be a single JSON object with a ‘title’ (string) and ‘content’ (list of strings) key.” }, “slide_schemas”: { “Content”: {“title”: “string”, “content”: “list[string]”}, “Summary”: {“title”: “string”, “content”: “list[string]”} } }, “apply_topic_interactive”: { “name”: “Interactive Lecture Flow”, “slide_types”: [“Title”, “Agenda”, “Content”, “Application”, “Summary”, “End”], “prompts”: { “content_generation”: “You are an expert university lecturer in Digital Forensics. Your audience is undergraduate students. Based on the provided context, create a slide explaining the concept of ‘{sub_topic}’. The content should be clear, concise, and structured with bullet points for easy understanding. Your output MUST be a single JSON object with a ‘title’ (string) and ‘content’ (list of strings) key.”, “application_generation”: “You are an engaging university lecturer creating an interactive slide. Based on the concept of ‘{sub_topic}’, create a multiple-choice question with exactly 4 options (A, B, C, D) to test understanding. The slide title must be ‘Let’s Apply This:’. Clearly indicate the correct answer within the content. Your output MUST be a single JSON object with a ‘title’ (string) and ‘content’ (list of strings) key.”, “summary_generation”: “You are an expert university lecturer creating a summary slide. Based on the following list of concepts and applications covered in this session, generate a concise summary of the key takeaways. The topics are: {topic_list}. Your output MUST be a single JSON object with a ‘title’ (string) and ‘content’ (list of strings) key.” }, “slide_schemas”: { “Content”: {“title”: “string”, “content”: “list[string]”}, “Application”: {“title”: “string”, “content”: “list[string]”}, “Summary”: {“title”: “string”, “content”: “list[string]”} } } }

7.4.1 Main Integration

[ ]:
```
TEST_MODE_ENABLED = False

# Cell 11: --- Main Orchestration Block (with Phase 5 & 6) ---
print_header("Main Orchestrator Initialized", char="*")

try:
    # 1. Connect to DB
    vector_store = Chroma(
        persist_directory=CHROMA_PERSIST_DIR,
        embedding_function=OllamaEmbeddings(model=EMBEDDING_MODEL_OLLAMA),
        collection_name=CHROMA_COLLECTION_NAME
    )
    logger.info("Database connection successful.")

    # Phase 1: Configuration and Scoping
    master_config = process_and_load_configurations()
```

```
    if master_config:
        all_final_plans = []

        # Phase 2 & 3: Create and Finalize Draft Plans
        print_header("Phase 2 & 3: Generating and Finalizing Weekly Plans  ",␣
↪char="-")
        planning_agent = PlanningAgent(master_config, vector_store=vector_store)

        weeks_to_generate =␣
↪master_config['processed_settings']['generation_scope']['weeks']
        logger.info(f"Found {len(weeks_to_generate)} week(s) to plan:␣
↪{weeks_to_generate}")

        if TEST_MODE_ENABLED:
            logger.info("--- TEST MODE ACTIVE: Processing only the first week␣
↪in scope. ---")
            weeks_to_generate = weeks_to_generate[:1]

        for week in weeks_to_generate:
            draft_plan = planning_agent.create_content_plan_for_week(week)
            if draft_plan:
                final_plan = planning_agent.
↪finalize_and_calculate_time_plan(draft_plan,␣
↪master_config['processed_settings'])
                all_final_plans.append(final_plan)

                # Save both draft and final for comparison
                draft_filename = f"{master_config['processed_settings'].
↪get('course_id')}_Week{week}_plan_draft.json"
                final_filename = f"{master_config['processed_settings'].
↪get('course_id')}_Week{week}_plan_final.json"


                with open(os.path.join(PLAN_OUTPUT_DIR, draft_filename), 'w')␣
↪as f:
                    json.dump(draft_plan, f, indent=2)

                with open(os.path.join(PLAN_OUTPUT_DIR, final_filename), 'w')␣
↪as f:
                    json.dump(final_plan, f, indent=2)

                logger.info(f"  Successfully saved FINAL plan for Week {week}␣
↪to: {os.path.join(PLAN_OUTPUT_DIR, final_filename)}")

            else:
```

```
                logger.error(f" Failed to generate draft plan for Week {week}.")

        # Phase 4: Generate Master Summary Plan
        master_plan_generated = False
        if all_final_plans:
            print_header(" Phase 4: Generating Master Unit Plan  ", char="-")
            master_plan_generated = planning_agent.
↪generate_and_save_master_plan(all_final_plans,␣
↪master_config['processed_settings'])
            master_plan_generated = True
        else:
            logger.warning("No weekly plans were generated, skipping master␣
↪plan creation.")

        # Initialize ContentAgent once for subsequent phases
        content_agent = ContentAgent(master_config, vector_store=vector_store)

        phase_5_successful = False

        # Phase 5: Fetching Raw Content
        if master_plan_generated:
            print_header("Phase 5: Populating Plans with Raw Content", char="#")
            successful_weeks_phase5 = []
            for week in weeks_to_generate:
                final_filename = f"{master_config['processed_settings'].
↪get('course_id')}_Week{week}_plan_final.json"
                full_plan_path = os.path.join(PLAN_OUTPUT_DIR, final_filename)

                if os.path.exists(full_plan_path):
                    if content_agent.generate_content_for_plan(full_plan_path,␣
↪CONTENT_OUTPUT_DIR):
                        successful_weeks_phase5.append(week)
                else:
                    logger.warning(f" Skipping content population for Week␣
↪{week} as its plan file was not found.")

            if successful_weeks_phase5:
                phase_5_successful = True
                logger.info(f" Phase 5 completed for weeks:␣
↪{successful_weeks_phase5}")

        # # Phase 6: Generating Slide Content with LLM
        phase_6_successful = False
        if phase_5_successful:
            print_header(" Phase 6: Generating Slide Content with LLM",␣
↪char="#")
```

```
            successful_weeks_phase6 = []
            for week in weeks_to_generate:
                # The input for phase 6 is the output of phase 5
                content_enriched_filename =␣
↪f"{master_config['processed_settings'].
↪get('course_id')}_Week{week}_plan_final.json"
                content_plan_path = os.path.join(CONTENT_OUTPUT_DIR,␣
↪content_enriched_filename)

                if os.path.exists(content_plan_path):
                     if content_agent.
↪generate_llm_content_for_plan(content_plan_path, CONTENT_LLM_OUTPUT_DIR):
                         successful_weeks_phase6.append(week)
                else:
                    logger.warning(f"Skipping LLM generation for Week {week} as␣
↪its content-enriched file was not found.")

            if successful_weeks_phase6:
                phase_6_successful = True
                logger.info(f" Phase 6 completed for weeks:␣
↪{successful_weeks_phase6}")


        # Phase 7 Phase 7: Generating Final PowerPoint Files
        #LAYOUT_MAPPING_PATH = os.path.join(CONFIG_DIR,␣
↪"layout_mapping_test_Mod.json") #   Seted up




        if phase_6_successful:
            print_header(" Phase 7: Generating Final PowerPoint Files",␣
↪char="#")
            presentation_agent =␣
↪PresentationAgent(template_path=SLIDE_TEMPLATE_PATH, layout_config_path =␣
↪LAYOUT_MAPPING_PATH)

            for week in weeks_to_generate:
                llm_plan_filename = f"{master_config['processed_settings'].
↪get('course_id')}_Week{week}_plan_final_llm_generated.json"
                llm_plan_path = os.path.join(CONTENT_LLM_OUTPUT_DIR,␣
↪llm_plan_filename)

                if os.path.exists(llm_plan_path):
                    presentation_agent.
↪create_presentation_from_plan(llm_plan_path, FINAL_PRESENTATION_DIR)
```

```
                else:
                    logger.warning(f"Skipping presentation generation for Week␣
 ↪{week} as its LLM-enriched plan was not found.")

            logger.info(f" Phase 7 completed  ")
        else:
            logger.warning("Skipping Phase 7 because prior phases failed or␣
 ↪were skipped.")

except Exception as e:
    logger.error(f"An unexpected error occurred during the main orchestration:␣
 ↪{e}", exc_info=True)
```

```
2025-07-18 21:59:49,592 - INFO - Database connection successful.
2025-07-18 21:59:49,592 - INFO - Loading all necessary configuration and data
files…
2025-07-18 21:59:49,595 - INFO - All files loaded successfully.
2025-07-18 21:59:49,595 - INFO - Pre-processing settings_deck for definitive
plan…
2025-07-18 21:59:49,595 - INFO - Applying overrides if specified…
2025-07-18 21:59:49,596 - INFO - Loaded from settings: 'interactive' is True.
Set teaching_flow_id to 'apply_topic_interactive'.
2025-07-18 21:59:49,596 - INFO - Calculating preliminary slide budget based on
session time…
2025-07-18 21:59:49,596 - INFO -  Duration Hours 2.
2025-07-18 21:59:49,597 - INFO -  Sessions per week 1.
2025-07-18 21:59:49,598 - INFO -  Preliminary weekly content slide target
calculated: # 32 slides.
2025-07-18 21:59:49,598 - INFO - Saving preliminary processed configuration to:
/home/sebas_dev_linux/projects/course_generator/configs/processed_settings.json
2025-07-18 21:59:49,599 - INFO - File saved successfully.
2025-07-18 21:59:49,599 - INFO - Master configuration object is ready for the
Planning Agent.
2025-07-18 21:59:49,601 - INFO - Data-Driven PlanningAgent initialized
successfully.
2025-07-18 21:59:49,602 - INFO - Found 12 week(s) to plan: [1, 2, 3, 4, 5, 6, 7,
8, 9, 10, 11, 12]
2025-07-18 21:59:49,659 - INFO - Partitioning strategy: Distributing 7 top-level
sections across 1 decks.
2025-07-18 21:59:49,659 - INFO - --- Planning Deck 1/1 | Topics: ['An Overview
of Digital Forensics', 'Preparing for Digital Investigations', 'Maintaining
Professional Conduct', 'Preparing a Digital Forensics Investigation',
'Procedures for Private-Sector High-Tech Investigations', 'Understanding Data
Recovery Workstations and Software', 'Conducting an Investigation'] | Weight:
521 chunks | Slide Budget: 31 ---
2025-07-18 21:59:49,661 - INFO -  Successfully saved FINAL plan for Week 1 to:
/home/sebas_dev_linux/projects/course_generator/data_output/2.
```

```
generated_plans/ICT312/ICT312_Week1_plan_final.json
2025-07-18 21:59:49,713 - INFO - Partitioning strategy: Distributing 4 top-level
sections across 1 decks.
2025-07-18 21:59:49,713 - INFO - --- Planning Deck 1/1 | Topics: ['Understanding
Forensics Lab Accreditation Requirements', 'Determining the Physical
Requirements for a Digital Forensics Lab', 'Selecting a Basic Forensic
Workstation', 'Building a Business Case for Developing a Forensics Lab'] |
Weight: 309 chunks | Slide Budget: 30 ---
2025-07-18 21:59:49,715 - INFO -  Successfully saved FINAL plan for Week 2 to:
/home/sebas_dev_linux/projects/course_generator/data_output/2.
generated_plans/ICT312/ICT312_Week2_plan_final.json
2025-07-18 21:59:49,764 - INFO - Partitioning strategy: Distributing 8 top-level
sections across 1 decks.
2025-07-18 21:59:49,764 - INFO - --- Planning Deck 1/1 | Topics: ['Understanding
Storage Formats for Digital Evidence', 'Determining the Best Acquisition
Method', 'Contingency Planning for Image Acquisitions', 'Using Acquisition
Tools', 'Validating Data Acquisitions', 'Performing RAID Data Acquisitions',
'Using Remote Network Acquisition Tools', 'Using Other Forensics Acquisition
Tools'] | Weight: 362 chunks | Slide Budget: 30 ---
2025-07-18 21:59:49,767 - INFO -  Successfully saved FINAL plan for Week 3 to:
/home/sebas_dev_linux/projects/course_generator/data_output/2.
generated_plans/ICT312/ICT312_Week3_plan_final.json

********************************************************************************
                        Main Orchestrator Initialized
********************************************************************************

--------------------------------------------------------------------------------
                   Phase 1: Configuration and Scoping Process
--------------------------------------------------------------------------------

--------------------------------------------------------------------------------
                        Phase 1 Configuration Complete
--------------------------------------------------------------------------------

--------------------------------------------------------------------------------
             Phase 2 & 3: Generating and Finalizing Weekly Plans
--------------------------------------------------------------------------------

********************************************************************************
                                Planning Week 1
********************************************************************************

********************************************************************************
                                Planning Week 2
********************************************************************************
```

```
********************************************************************************
                                  Planning Week 3
********************************************************************************

********************************************************************************
                                  Planning Week 4
********************************************************************************
2025-07-18 21:59:49,807 - INFO - Partitioning strategy: Distributing 8 top-level
sections across 1 decks.
2025-07-18 21:59:49,808 - INFO - --- Planning Deck 1/1 | Topics: ['Identifying
Digital Evidence', 'Collecting Evidence in Private-Sector Incident Scenes',
'Processing Law Enforcement Crime Scenes', 'Preparing for a Search', 'Securing a
Digital Incident or Crime Scene', 'Seizing Digital Evidence at the Scene',
'Storing Digital Evidence', 'Obtaining a Digital Hash'] | Weight: 305 chunks |
Slide Budget: 29 ---
2025-07-18 21:59:49,810 - INFO -  Successfully saved FINAL plan for Week 4 to:
/home/sebas_dev_linux/projects/course_generator/data_output/2.
generated_plans/ICT312/ICT312_Week4_plan_final.json
2025-07-18 21:59:49,869 - INFO - Partitioning strategy: Distributing 7 top-level
sections across 1 decks.
2025-07-18 21:59:49,870 - INFO - --- Planning Deck 1/1 | Topics: ['Understanding
File Systems', 'Exploring Microsoft File Structures', 'Examining NTFS Disks',
'Understanding Whole Disk Encryption', 'Understanding the Windows Registry',
'Understanding Microsoft Startup Tasks', 'Understanding Virtual Machines'] |
Weight: 449 chunks | Slide Budget: 31 ---
2025-07-18 21:59:49,872 - INFO -  Successfully saved FINAL plan for Week 5 to:
/home/sebas_dev_linux/projects/course_generator/data_output/2.
generated_plans/ICT312/ICT312_Week5_plan_final.json
2025-07-18 21:59:49,919 - INFO - Partitioning strategy: Distributing 4 top-level
sections across 1 decks.
2025-07-18 21:59:49,920 - INFO - --- Planning Deck 1/1 | Topics: ['Evaluating
Digital Forensics Tool Needs', 'Digital Forensics Software Tools', 'Digital
Forensics Hardware Tools', 'Validating and Testing Forensics Software'] |
Weight: 293 chunks | Slide Budget: 30 ---
2025-07-18 21:59:49,923 - INFO -  Successfully saved FINAL plan for Week 6 to:
/home/sebas_dev_linux/projects/course_generator/data_output/2.
generated_plans/ICT312/ICT312_Week6_plan_final.json
2025-07-18 21:59:49,981 - INFO - Partitioning strategy: Distributing 7 top-level
sections across 1 decks.
2025-07-18 21:59:49,981 - INFO - --- Planning Deck 1/1 | Topics: ['Examining
Linux File Structures', 'Understanding Macintosh File Structures', 'Using Linux
Forensics Tools', 'Recognizing a Graphics File', 'Understanding Data
Compression', 'Identifying Unknown File Formats', 'Understanding Copyright
Issues with Graphics'] | Weight: 478 chunks | Slide Budget: 30 ---
2025-07-18 21:59:49,984 - INFO -  Successfully saved FINAL plan for Week 7 to:
/home/sebas_dev_linux/projects/course_generator/data_output/2.
generated_plans/ICT312/ICT312_Week7_plan_final.json
```

```
********************************************************************************
                                Planning Week 5
********************************************************************************

********************************************************************************
                                Planning Week 6
********************************************************************************

********************************************************************************
                                Planning Week 7
********************************************************************************

********************************************************************************
                                Planning Week 8
********************************************************************************
2025-07-18 21:59:50,055 - INFO - Partitioning strategy: Distributing 6 top-level
sections across 1 decks.
2025-07-18 21:59:50,056 - INFO - --- Planning Deck 1/1 | Topics: ['Determining
What Data to Collect and Analyze', 'Validating Forensic Data', 'Addressing Data-
Hiding Techniques', 'An Overview of Virtual Machine Forensics', 'Performing Live
Acquisitions', 'Network Forensics Overview'] | Weight: 491 chunks | Slide
Budget: 30 ---
2025-07-18 21:59:50,058 - INFO -   Successfully saved FINAL plan for Week 8 to:
/home/sebas_dev_linux/projects/course_generator/data_output/2.
generated_plans/ICT312/ICT312_Week8_plan_final.json
2025-07-18 21:59:50,097 - INFO - Partitioning strategy: Distributing 6 top-level
sections across 1 decks.
2025-07-18 21:59:50,098 - INFO - --- Planning Deck 1/1 | Topics: ['Exploring the
Role of E-mail in Investigations', 'Exploring the Roles of the Client and Server
in E-mail', 'Investigating E-mail Crimes and Violations', 'Understanding E-mail
Servers', 'Using Specialized E-mail Forensics Tools', 'Applying Digital
Forensics Methods to Social Media Communications'] | Weight: 282 chunks | Slide
Budget: 30 ---
2025-07-18 21:59:50,099 - INFO -   Successfully saved FINAL plan for Week 9 to:
/home/sebas_dev_linux/projects/course_generator/data_output/2.
generated_plans/ICT312/ICT312_Week9_plan_final.json
2025-07-18 21:59:50,175 - INFO - Partitioning strategy: Distributing 9 top-level
sections across 1 decks.
2025-07-18 21:59:50,175 - INFO - --- Planning Deck 1/1 | Topics: ['Understanding
Mobile Device Forensics', 'Understanding Acquisition Procedures for Mobile
Devices', 'Understanding Forensics in the Internet of Anything', 'An Overview of
Cloud Computing', 'Legal Challenges in Cloud Forensics', 'Technical Challenges
in Cloud Forensics', 'Acquisitions in the Cloud', 'Conducting a Cloud
Investigation', 'Tools for Cloud Forensics'] | Weight: 431 chunks | Slide
Budget: 30 ---
2025-07-18 21:59:50,178 - INFO -   Successfully saved FINAL plan for Week 10 to:
```

```
/home/sebas_dev_linux/projects/course_generator/data_output/2.
generated_plans/ICT312/ICT312_Week10_plan_final.json
2025-07-18 21:59:50,247 - INFO - Partitioning strategy: Distributing 7 top-level
sections across 1 decks.
2025-07-18 21:59:50,248 - INFO - --- Planning Deck 1/1 | Topics: ['Understanding
the Importance of Reports', 'Guidelines for Writing Reports', 'Generating Report
Findings with Forensics Software Tools', 'Preparing for Testimony', 'Testifying
in Court', 'Preparing for a Deposition or Hearing', 'Preparing Forensics
Evidence for Testimony'] | Weight: 553 chunks | Slide Budget: 30 ---
2025-07-18 21:59:50,250 - INFO -  Successfully saved FINAL plan for Week 11 to:
/home/sebas_dev_linux/projects/course_generator/data_output/2.
generated_plans/ICT312/ICT312_Week11_plan_final.json


********************************************************************************
                                Planning Week 9
********************************************************************************

********************************************************************************
                                Planning Week 10
********************************************************************************

********************************************************************************
                                Planning Week 11
********************************************************************************

********************************************************************************
                                Planning Week 12
********************************************************************************
2025-07-18 21:59:50,293 - INFO - Partitioning strategy: Distributing 4 top-level
sections across 1 decks.
2025-07-18 21:59:50,293 - INFO - --- Planning Deck 1/1 | Topics: ['Applying
Ethics and Codes to Expert Witnesses', 'Organizations with Codes of Ethics',
'Ethical Difficulties in Expert Testimony', 'An Ethics Exercise'] | Weight: 297
chunks | Slide Budget: 31 ---
2025-07-18 21:59:50,295 - INFO -  Successfully saved FINAL plan for Week 12 to:
/home/sebas_dev_linux/projects/course_generator/data_output/2.
generated_plans/ICT312/ICT312_Week12_plan_final.json
2025-07-18 21:59:50,296 - INFO - Successfully generated and saved Master Unit
Plan to: /home/sebas_dev_linux/projects/course_generator/data_output/2.
generated_plans/ICT312/ICT312_master_plan_unit.json
2025-07-18 21:59:50,301 - INFO - Data-Driven Content Agent initialized with
model 'qwen3:8b'.
2025-07-18 21:59:50,302 - INFO - Data-Driven Content Agent initialized
successfully.
2025-07-18 21:59:50,302 - INFO - PHASE 5: Populating raw content for:
/home/sebas_dev_linux/projects/course_generator/data_output/2.
generated_plans/ICT312/ICT312_Week1_plan_final.json
```

```
2025-07-18 21:59:50,361 - INFO - Reordering keys for Fetched clean output…
2025-07-18 21:59:50,364 - INFO - Successfully saved content-enriched plan to:
/home/sebas_dev_linux/projects/course_generator/data_output/3.
generated_content/ICT312/ICT312_Week1_plan_final.json
2025-07-18 21:59:50,364 - INFO - PHASE 5: Populating raw content for:
/home/sebas_dev_linux/projects/course_generator/data_output/2.
generated_plans/ICT312/ICT312_Week2_plan_final.json
2025-07-18 21:59:50,417 - INFO - Reordering keys for Fetched clean output…
2025-07-18 21:59:50,419 - INFO - Successfully saved content-enriched plan to:
/home/sebas_dev_linux/projects/course_generator/data_output/3.
generated_content/ICT312/ICT312_Week2_plan_final.json
2025-07-18 21:59:50,420 - INFO - PHASE 5: Populating raw content for:
/home/sebas_dev_linux/projects/course_generator/data_output/2.
generated_plans/ICT312/ICT312_Week3_plan_final.json
2025-07-18 21:59:50,469 - INFO - Reordering keys for Fetched clean output…
2025-07-18 21:59:50,472 - INFO - Successfully saved content-enriched plan to:
/home/sebas_dev_linux/projects/course_generator/data_output/3.
generated_content/ICT312/ICT312_Week3_plan_final.json
2025-07-18 21:59:50,472 - INFO - PHASE 5: Populating raw content for:
/home/sebas_dev_linux/projects/course_generator/data_output/2.
generated_plans/ICT312/ICT312_Week4_plan_final.json


--------------------------------------------------------------------------------
                    Phase 4: Generating Master Unit Plan
--------------------------------------------------------------------------------

################################################################################
                      Phase 4: Generating Master Unit Plan
################################################################################

--- Preview of Master Plan ---
{
  "unit_code": "ICT312",
  "unit_name": "Digital Forensic",
  "grand_total_summary": {
    "total_slides_for_unit": 474,
    "total_framework_slides": 48,
    "total_content_slides": 349,
    "total_interactive_slides": 77,
    "total_number_of_decks": 12,
    "total_time_for_unit_minutes": 1504,
    "total_time_for_unit_in_hour": 25.07,
    "average_deck_time_in_min": 125.33,
    "average_deck_time_in_hour": 2.09
  },
  "weekly_summaries": [
    {
```

```
      "week": 1,
      "overall_topic": "Understanding the Digital Forensics Profession and
Investigations.",
      "slide_summary": {
        "total_slides_for_week": 43,
        "total_framework_slides": 4,
        "total_content_slides": 32,
        "total_interactive_slides": 7,
        "number_of_decks": 1
      },
      "time_summary_minutes": {
        "total_time_for_week_minutes": 137,
        "total_framework_time": 6,
        "total_content_and_interactive_time": 131
      }
    },
    {
      "week": 2,
      "overall_topic": "The Investigator's Office and Laboratory.",
      "slide_summary": {
        "total_slides_for_week": 36,
        "total_framework_slides": 4,
        "total_content_slides": 28,
        "total_interactive_slides": 4,
        "number_of_decks": 1
      },
      "time_summary_minutes": {
        "total_time_for_week_minutes": 110,
        "total_framework_time": 6,
        "total_content_and_interactive_time": 104
      }
    },
    {
      "week": 3,
      "overall_topic": "Data Acquisition.",
      "slide_summary": {
        "total_slides_for_week": 42,
        "total_framework_slides": 4,
        "total_content_slides": 30,
        "total_interactive_slides": 8,
        "number_of_decks": 1
      },
      "time_summary_minutes": {
        "total_time_for_week_minutes": 136,
        "total_framework_time": 6,
        "total_content_and_interactive_time": 130
      }
    },
```

```
{
  "week": 4,
  "overall_topic": "Processing Crime and Incident Scenes.",
  "slide_summary": {
    "total_slides_for_week": 39,
    "total_framework_slides": 4,
    "total_content_slides": 27,
    "total_interactive_slides": 8,
    "number_of_decks": 1
  },
  "time_summary_minutes": {
    "total_time_for_week_minutes": 127,
    "total_framework_time": 6,
    "total_content_and_interactive_time": 121
  }
},
{
  "week": 5,
  "overall_topic": "Working with Windows and CLI Systems.",
  "slide_summary": {
    "total_slides_for_week": 42,
    "total_framework_slides": 4,
    "total_content_slides": 31,
    "total_interactive_slides": 7,
    "number_of_decks": 1
  },
  "time_summary_minutes": {
    "total_time_for_week_minutes": 134,
    "total_framework_time": 6,
    "total_content_and_interactive_time": 128
  }
},
{
  "week": 6,
  "overall_topic": "Current Computer Forensics Tools.",
  "slide_summary": {
    "total_slides_for_week": 36,
    "total_framework_slides": 4,
    "total_content_slides": 28,
    "total_interactive_slides": 4,
    "number_of_decks": 1
  },
  "time_summary_minutes": {
    "total_time_for_week_minutes": 110,
    "total_framework_time": 6,
    "total_content_and_interactive_time": 104
  }
},
```

```
    {
      "week": 7,
      "overall_topic": "Linux Boot Processes and File Systems. Recovering
Graphics Files.",
      "slide_summary": {
        "total_slides_for_week": 37,
        "total_framework_slides": 4,
        "total_content_slides": 26,
        "total_interactive_slides": 7,
        "number_of_decks": 1
      },
      "time_summary_minutes": {
        "total_time_for_week_minutes": 119,
        "total_framework_time": 6,
        "total_content_and_interactive_time": 113
      }
    },
    {
      "week": 8,
      "overall_topic": "Digital Forensics Analysis and Validation. Virtual
Machine Forensics, Live Acquisitions and Cloud Forensics.",
      "slide_summary": {
        "total_slides_for_week": 36,
        "total_framework_slides": 4,
        "total_content_slides": 26,
        "total_interactive_slides": 6,
        "number_of_decks": 1
      },
      "time_summary_minutes": {
        "total_time_for_week_minutes": 114,
        "total_framework_time": 6,
        "total_content_and_interactive_time": 108
      }
    },
    {
      "week": 9,
      "overall_topic": "Email and Social Media.",
      "slide_summary": {
        "total_slides_for_week": 43,
        "total_framework_slides": 4,
        "total_content_slides": 33,
        "total_interactive_slides": 6,
        "number_of_decks": 1
      },
      "time_summary_minutes": {
        "total_time_for_week_minutes": 135,
        "total_framework_time": 6,
        "total_content_and_interactive_time": 129
```

```
      }
    },
    {
      "week": 10,
      "overall_topic": "Mobile Device Forensics and the Internet of Anything.
Cloud Forensics.",
      "slide_summary": {
        "total_slides_for_week": 41,
        "total_framework_slides": 4,
        "total_content_slides": 28,
        "total_interactive_slides": 9,
        "number_of_decks": 1
      },
      "time_summary_minutes": {
        "total_time_for_week_minutes": 135,
        "total_framework_time": 6,
        "total_content_and_interactive_time": 129
      }
    },
    {
      "week": 11,
      "overall_topic": "Report Writing for High-Tech Investigations. Expert
Testimony in Digital Forensic Investigations.",
      "slide_summary": {
        "total_slides_for_week": 40,
        "total_framework_slides": 4,
        "total_content_slides": 29,
        "total_interactive_slides": 7,
        "number_of_decks": 1
      },
      "time_summary_minutes": {
        "total_time_for_week_minutes": 128,
        "total_framework_time": 6,
        "total_content_and_interactive_time": 122
      }
    },
    {
      "week": 12,
      "overall_topic": "Ethics for the Digital Forensic Examiner and Expert
Witness.",
      "slide_summary": {
        "total_slides_for_week": 39,
        "total_framework_slides": 4,
        "total_content_slides": 31,
        "total_interactive_slides": 4,
        "number_of_decks": 1
      },
      "time_summary_minutes": {
```

```
        "total_time_for_week_minutes": 119,
        "total_framework_time": 6,
        "total_content_and_interactive_time": 113
      }
    }
  ]
}
```


```
################################################################################
                  Phase 5: Populating Plans with Raw Content
################################################################################
2025-07-18 21:59:50,510 - INFO - Reordering keys for Fetched clean output…
2025-07-18 21:59:50,513 - INFO - Successfully saved content-enriched plan to:
/home/sebas_dev_linux/projects/course_generator/data_output/3.
generated_content/ICT312/ICT312_Week4_plan_final.json
2025-07-18 21:59:50,513 - INFO - PHASE 5: Populating raw content for:
/home/sebas_dev_linux/projects/course_generator/data_output/2.
generated_plans/ICT312/ICT312_Week5_plan_final.json
2025-07-18 21:59:50,571 - INFO - Reordering keys for Fetched clean output…
2025-07-18 21:59:50,574 - INFO - Successfully saved content-enriched plan to:
/home/sebas_dev_linux/projects/course_generator/data_output/3.
generated_content/ICT312/ICT312_Week5_plan_final.json
2025-07-18 21:59:50,574 - INFO - PHASE 5: Populating raw content for:
/home/sebas_dev_linux/projects/course_generator/data_output/2.
generated_plans/ICT312/ICT312_Week6_plan_final.json
2025-07-18 21:59:50,618 - INFO - Reordering keys for Fetched clean output…
2025-07-18 21:59:50,621 - INFO - Successfully saved content-enriched plan to:
/home/sebas_dev_linux/projects/course_generator/data_output/3.
generated_content/ICT312/ICT312_Week6_plan_final.json
2025-07-18 21:59:50,622 - INFO - PHASE 5: Populating raw content for:
/home/sebas_dev_linux/projects/course_generator/data_output/2.
generated_plans/ICT312/ICT312_Week7_plan_final.json
2025-07-18 21:59:50,683 - INFO - Reordering keys for Fetched clean output…
2025-07-18 21:59:50,685 - INFO - Successfully saved content-enriched plan to:
/home/sebas_dev_linux/projects/course_generator/data_output/3.
generated_content/ICT312/ICT312_Week7_plan_final.json
2025-07-18 21:59:50,686 - INFO - PHASE 5: Populating raw content for:
/home/sebas_dev_linux/projects/course_generator/data_output/2.
generated_plans/ICT312/ICT312_Week8_plan_final.json
2025-07-18 21:59:50,752 - INFO - Reordering keys for Fetched clean output…
2025-07-18 21:59:50,755 - INFO - Successfully saved content-enriched plan to:
/home/sebas_dev_linux/projects/course_generator/data_output/3.
generated_content/ICT312/ICT312_Week8_plan_final.json
2025-07-18 21:59:50,755 - INFO - PHASE 5: Populating raw content for:
/home/sebas_dev_linux/projects/course_generator/data_output/2.
generated_plans/ICT312/ICT312_Week9_plan_final.json
2025-07-18 21:59:50,791 - INFO - Reordering keys for Fetched clean output…
```

```
2025-07-18 21:59:50,793 - INFO - Successfully saved content-enriched plan to:
/home/sebas_dev_linux/projects/course_generator/data_output/3.
generated_content/ICT312/ICT312_Week9_plan_final.json
2025-07-18 21:59:50,793 - INFO - PHASE 5: Populating raw content for:
/home/sebas_dev_linux/projects/course_generator/data_output/2.
generated_plans/ICT312/ICT312_Week10_plan_final.json
2025-07-18 21:59:50,865 - INFO - Reordering keys for Fetched clean output…
2025-07-18 21:59:50,868 - INFO - Successfully saved content-enriched plan to:
/home/sebas_dev_linux/projects/course_generator/data_output/3.
generated_content/ICT312/ICT312_Week10_plan_final.json
2025-07-18 21:59:50,869 - INFO - PHASE 5: Populating raw content for:
/home/sebas_dev_linux/projects/course_generator/data_output/2.
generated_plans/ICT312/ICT312_Week11_plan_final.json
2025-07-18 21:59:50,941 - INFO - Reordering keys for Fetched clean output…
2025-07-18 21:59:50,944 - INFO - Successfully saved content-enriched plan to:
/home/sebas_dev_linux/projects/course_generator/data_output/3.
generated_content/ICT312/ICT312_Week11_plan_final.json
2025-07-18 21:59:50,944 - INFO - PHASE 5: Populating raw content for:
/home/sebas_dev_linux/projects/course_generator/data_output/2.
generated_plans/ICT312/ICT312_Week12_plan_final.json
2025-07-18 21:59:50,961 - WARNING - No chunks found in the database for toc_id =
534
2025-07-18 21:59:51,005 - INFO - Reordering keys for Fetched clean output…
2025-07-18 21:59:51,009 - INFO - Successfully saved content-enriched plan to:
/home/sebas_dev_linux/projects/course_generator/data_output/3.
generated_content/ICT312/ICT312_Week12_plan_final.json
2025-07-18 21:59:51,009 - INFO -  Phase 5 completed for weeks: [1, 2, 3, 4, 5,
6, 7, 8, 9, 10, 11, 12]
2025-07-18 21:59:51,010 - INFO -  PHASE 6: Generating LLM content for:
ICT312_Week1_plan_final.json output file path:
/home/sebas_dev_linux/projects/course_generator/data_output/4.
generated_content_llm/ICT312
2025-07-18 21:59:51,012 - INFO - Found a total of 40 slides to generate across
all decks.
2025-07-18 21:59:51,012 - INFO - [Progress: 0/40] Generating slide 1/1 for
topic: 'An Overview of Digital Forensics'
2025-07-18 21:59:51,013 - INFO - Calling Ollama model 'qwen3:8b' (Call #1)…


################################################################################
                    Phase 6: Generating Slide Content with LLM
################################################################################

2025-07-18 21:59:58,092 - INFO - HTTP Request: POST
http://localhost:11434/api/chat "HTTP/1.1 200 OK"
2025-07-18 21:59:58,093 - INFO - [Progress: 1/40] Generating slide 1/1 for
topic: 'Digital Forensics and Other Related Disciplines'
2025-07-18 21:59:58,093 - INFO - Calling Ollama model 'qwen3:8b' (Call #2)…
2025-07-18 21:59:58,093 - INFO - [Progress: 1/40] Generating slide 1/1 for
```

```
topic: 'Digital Forensics and Other Related Disciplines'
2025-07-18 21:59:58,093 - INFO - Calling Ollama model 'qwen3:8b' (Call #2)…
2025-07-18 22:00:07,994 - INFO - HTTP Request: POST
http://localhost:11434/api/chat "HTTP/1.1 200 OK"
2025-07-18 22:00:07,995 - INFO - [Progress: 2/40] Generating slide 1/1 for
topic: 'A Brief History of Digital Forensics'
2025-07-18 22:00:07,995 - INFO - Calling Ollama model 'qwen3:8b' (Call #3)…
2025-07-18 22:00:15,597 - INFO - HTTP Request: POST
http://localhost:11434/api/chat "HTTP/1.1 200 OK"
2025-07-18 22:00:15,598 - INFO - [Progress: 3/40] Generating slide 1/1 for
topic: 'Developing Digital Forensics Resources'
2025-07-18 22:00:15,598 - INFO - Calling Ollama model 'qwen3:8b' (Call #4)…
2025-07-18 22:00:21,720 - INFO - HTTP Request: POST
http://localhost:11434/api/chat "HTTP/1.1 200 OK"
2025-07-18 22:00:21,721 - INFO - [Progress: 4/40] Generating interactive slide
for topic: 'An Overview of Digital Forensics'
2025-07-18 22:00:21,722 - INFO - Calling Ollama model 'qwen3:8b' (Call #5)…
2025-07-18 22:00:30,285 - INFO - HTTP Request: POST
http://localhost:11434/api/chat "HTTP/1.1 200 OK"
2025-07-18 22:00:30,286 - INFO - [Progress: 5/40] Generating slide 1/1 for
topic: 'Understanding Law Enforcement Agency Investigations'
2025-07-18 22:00:30,287 - INFO - Calling Ollama model 'qwen3:8b' (Call #6)…
2025-07-18 22:00:40,628 - INFO - HTTP Request: POST
http://localhost:11434/api/chat "HTTP/1.1 200 OK"
2025-07-18 22:00:40,629 - INFO - [Progress: 6/40] Generating slide 1/1 for
topic: 'Following Legal Processes'
2025-07-18 22:00:40,630 - INFO - Calling Ollama model 'qwen3:8b' (Call #7)…
2025-07-18 22:00:44,679 - INFO - HTTP Request: POST
http://localhost:11434/api/chat "HTTP/1.1 200 OK"
2025-07-18 22:00:44,680 - INFO - [Progress: 7/40] Generating slide 1/1 for
topic: 'Displaying Warning Banners'
2025-07-18 22:00:44,681 - INFO - Calling Ollama model 'qwen3:8b' (Call #8)…
2025-07-18 22:00:49,699 - INFO - HTTP Request: POST
http://localhost:11434/api/chat "HTTP/1.1 200 OK"
2025-07-18 22:00:49,699 - INFO - [Progress: 8/40] Generating slide 1/1 for
topic: 'Designating an Authorized Requester'
2025-07-18 22:00:49,700 - INFO - Calling Ollama model 'qwen3:8b' (Call #9)…
2025-07-18 22:00:55,485 - INFO - HTTP Request: POST
http://localhost:11434/api/chat "HTTP/1.1 200 OK"
2025-07-18 22:00:55,486 - INFO - [Progress: 9/40] Generating slide 1/1 for
topic: 'Conducting Security Investigations'
2025-07-18 22:00:55,487 - INFO - Calling Ollama model 'qwen3:8b' (Call #10)…
2025-07-18 22:01:01,450 - INFO - HTTP Request: POST
http://localhost:11434/api/chat "HTTP/1.1 200 OK"
2025-07-18 22:01:01,451 - INFO - [Progress: 10/40] Generating interactive slide
for topic: 'Preparing for Digital Investigations'
2025-07-18 22:01:01,451 - INFO - Calling Ollama model 'qwen3:8b' (Call #11)…
2025-07-18 22:01:08,949 - INFO - HTTP Request: POST
```

```
http://localhost:11434/api/chat "HTTP/1.1 200 OK"
2025-07-18 22:01:08,950 - INFO - [Progress: 11/40] Generating slide 1/1 for
topic: 'Maintaining Professional Conduct'
2025-07-18 22:01:08,950 - INFO - Calling Ollama model 'qwen3:8b' (Call #12)…
2025-07-18 22:01:17,629 - INFO - HTTP Request: POST
http://localhost:11434/api/chat "HTTP/1.1 200 OK"
2025-07-18 22:01:17,629 - INFO - [Progress: 12/40] Generating interactive slide
for topic: 'Maintaining Professional Conduct'
2025-07-18 22:01:17,630 - INFO - Calling Ollama model 'qwen3:8b' (Call #13)…
2025-07-18 22:01:24,568 - INFO - HTTP Request: POST
http://localhost:11434/api/chat "HTTP/1.1 200 OK"
2025-07-18 22:01:24,569 - INFO - [Progress: 13/40] Generating slide 1/1 for
topic: 'An Overview of a Computer Crime'
2025-07-18 22:01:24,569 - INFO - Calling Ollama model 'qwen3:8b' (Call #14)…
2025-07-18 22:01:33,380 - INFO - HTTP Request: POST
http://localhost:11434/api/chat "HTTP/1.1 200 OK"
2025-07-18 22:01:33,381 - INFO - [Progress: 14/40] Generating slide 1/1 for
topic: 'Taking a Systematic Approach'
2025-07-18 22:01:33,381 - INFO - Calling Ollama model 'qwen3:8b' (Call #15)…
2025-07-18 22:01:38,568 - INFO - HTTP Request: POST
http://localhost:11434/api/chat "HTTP/1.1 200 OK"
2025-07-18 22:01:38,570 - INFO - [Progress: 15/40] Generating slide 1/1 for
topic: 'Assessing the Case'
2025-07-18 22:01:38,571 - INFO - Calling Ollama model 'qwen3:8b' (Call #16)…
2025-07-18 22:01:46,627 - INFO - HTTP Request: POST
http://localhost:11434/api/chat "HTTP/1.1 200 OK"
2025-07-18 22:01:46,628 - INFO - [Progress: 16/40] Generating slide 1/3 for
topic: 'Planning Your Investigation'
2025-07-18 22:01:46,628 - INFO - Calling Ollama model 'qwen3:8b' (Call #17)…
2025-07-18 22:01:52,113 - INFO - HTTP Request: POST
http://localhost:11434/api/chat "HTTP/1.1 200 OK"
2025-07-18 22:01:52,114 - INFO - [Progress: 17/40] Generating slide 2/3 for
topic: 'Planning Your Investigation'
2025-07-18 22:01:52,115 - INFO - Calling Ollama model 'qwen3:8b' (Call #18)…
2025-07-18 22:01:57,537 - INFO - HTTP Request: POST
http://localhost:11434/api/chat "HTTP/1.1 200 OK"
2025-07-18 22:01:57,538 - INFO - [Progress: 18/40] Generating slide 3/3 for
topic: 'Planning Your Investigation'
2025-07-18 22:01:57,538 - INFO - Calling Ollama model 'qwen3:8b' (Call #19)…
2025-07-18 22:02:17,892 - INFO - HTTP Request: POST
http://localhost:11434/api/chat "HTTP/1.1 200 OK"
2025-07-18 22:02:17,893 - INFO - [Progress: 19/40] Generating slide 1/1 for
topic: 'Securing Your Evidence'
2025-07-18 22:02:17,893 - INFO - Calling Ollama model 'qwen3:8b' (Call #20)…
2025-07-18 22:02:24,061 - INFO - HTTP Request: POST
http://localhost:11434/api/chat "HTTP/1.1 200 OK"
2025-07-18 22:02:24,062 - INFO - [Progress: 20/40] Generating interactive slide
for topic: 'Preparing a Digital Forensics Investigation'
```

```
2025-07-18 22:02:24,062 - INFO - Calling Ollama model 'qwen3:8b' (Call #21)…
2025-07-18 22:02:31,312 - INFO - HTTP Request: POST
http://localhost:11434/api/chat "HTTP/1.1 200 OK"
2025-07-18 22:02:31,313 - INFO - [Progress: 21/40] Generating slide 1/1 for
topic: 'Internet Abuse Investigations'
2025-07-18 22:02:31,314 - INFO - Calling Ollama model 'qwen3:8b' (Call #22)…
2025-07-18 22:02:36,235 - INFO - HTTP Request: POST
http://localhost:11434/api/chat "HTTP/1.1 200 OK"
2025-07-18 22:02:36,236 - INFO - [Progress: 22/40] Generating slide 1/1 for
topic: 'E-mail Abuse Investigations'
2025-07-18 22:02:36,236 - INFO - Calling Ollama model 'qwen3:8b' (Call #23)…
2025-07-18 22:02:43,131 - INFO - HTTP Request: POST
http://localhost:11434/api/chat "HTTP/1.1 200 OK"
2025-07-18 22:02:43,132 - INFO - [Progress: 23/40] Generating slide 1/2 for
topic: 'Attorney-Client Privilege Investigations'
2025-07-18 22:02:43,132 - INFO - Calling Ollama model 'qwen3:8b' (Call #24)…
2025-07-18 22:02:48,317 - INFO - HTTP Request: POST
http://localhost:11434/api/chat "HTTP/1.1 200 OK"
2025-07-18 22:02:48,318 - INFO - [Progress: 24/40] Generating slide 2/2 for
topic: 'Attorney-Client Privilege Investigations'
2025-07-18 22:02:48,319 - INFO - Calling Ollama model 'qwen3:8b' (Call #25)…
2025-07-18 22:02:56,787 - INFO - HTTP Request: POST
http://localhost:11434/api/chat "HTTP/1.1 200 OK"
2025-07-18 22:02:56,788 - INFO - [Progress: 25/40] Generating slide 1/2 for
topic: 'Industrial Espionage Investigations'
2025-07-18 22:02:56,788 - INFO - Calling Ollama model 'qwen3:8b' (Call #26)…
2025-07-18 22:03:02,198 - INFO - HTTP Request: POST
http://localhost:11434/api/chat "HTTP/1.1 200 OK"
2025-07-18 22:03:02,199 - INFO - [Progress: 26/40] Generating slide 2/2 for
topic: 'Industrial Espionage Investigations'
2025-07-18 22:03:02,199 - INFO - Calling Ollama model 'qwen3:8b' (Call #27)…
2025-07-18 22:03:07,355 - INFO - HTTP Request: POST
http://localhost:11434/api/chat "HTTP/1.1 200 OK"
2025-07-18 22:03:07,356 - INFO - [Progress: 27/40] Generating slide 1/1 for
topic: 'Interviews and Interrogations in High-Tech Investigations'
2025-07-18 22:03:07,356 - INFO - Calling Ollama model 'qwen3:8b' (Call #28)…
2025-07-18 22:03:12,620 - INFO - HTTP Request: POST
http://localhost:11434/api/chat "HTTP/1.1 200 OK"
2025-07-18 22:03:12,621 - INFO - [Progress: 28/40] Generating interactive slide
for topic: 'Procedures for Private-Sector High-Tech Investigations'
2025-07-18 22:03:12,621 - INFO - Calling Ollama model 'qwen3:8b' (Call #29)…
2025-07-18 22:03:19,316 - INFO - HTTP Request: POST
http://localhost:11434/api/chat "HTTP/1.1 200 OK"
2025-07-18 22:03:19,317 - INFO - [Progress: 29/40] Generating slide 1/1 for
topic: 'Understanding Data Recovery Workstations and Software'
2025-07-18 22:03:19,317 - INFO - Calling Ollama model 'qwen3:8b' (Call #30)…
2025-07-18 22:03:29,586 - INFO - HTTP Request: POST
http://localhost:11434/api/chat "HTTP/1.1 200 OK"
```

```
2025-07-18 22:03:29,587 - INFO - [Progress: 30/40] Generating slide 1/1 for
topic: 'Setting Up Your Workstation for Digital Forensics'
2025-07-18 22:03:29,588 - INFO - Calling Ollama model 'qwen3:8b' (Call #31)…
2025-07-18 22:03:33,230 - INFO - HTTP Request: POST
http://localhost:11434/api/chat "HTTP/1.1 200 OK"
2025-07-18 22:03:33,231 - INFO - [Progress: 31/40] Generating interactive slide
for topic: 'Understanding Data Recovery Workstations and Software'
2025-07-18 22:03:33,232 - INFO - Calling Ollama model 'qwen3:8b' (Call #32)…
2025-07-18 22:03:39,844 - INFO - HTTP Request: POST
http://localhost:11434/api/chat "HTTP/1.1 200 OK"
2025-07-18 22:03:39,845 - INFO - [Progress: 32/40] Generating slide 1/1 for
topic: 'Gathering the Evidence'
2025-07-18 22:03:39,845 - INFO - Calling Ollama model 'qwen3:8b' (Call #33)…
2025-07-18 22:03:45,409 - INFO - HTTP Request: POST
http://localhost:11434/api/chat "HTTP/1.1 200 OK"
2025-07-18 22:03:45,410 - INFO - [Progress: 33/40] Generating slide 1/3 for
topic: 'Analyzing Your Digital Evidence'
2025-07-18 22:03:45,410 - INFO - Calling Ollama model 'qwen3:8b' (Call #34)…
2025-07-18 22:03:50,040 - INFO - HTTP Request: POST
http://localhost:11434/api/chat "HTTP/1.1 200 OK"
2025-07-18 22:03:50,041 - INFO - [Progress: 34/40] Generating slide 2/3 for
topic: 'Analyzing Your Digital Evidence'
2025-07-18 22:03:50,042 - INFO - Calling Ollama model 'qwen3:8b' (Call #35)…
2025-07-18 22:03:57,005 - INFO - HTTP Request: POST
http://localhost:11434/api/chat "HTTP/1.1 200 OK"
2025-07-18 22:03:57,006 - INFO - [Progress: 35/40] Generating slide 3/3 for
topic: 'Analyzing Your Digital Evidence'
2025-07-18 22:03:57,006 - INFO - Calling Ollama model 'qwen3:8b' (Call #36)…
2025-07-18 22:04:02,907 - INFO - HTTP Request: POST
http://localhost:11434/api/chat "HTTP/1.1 200 OK"
2025-07-18 22:04:02,908 - INFO - [Progress: 36/40] Generating slide 1/1 for
topic: 'Completing the Case'
2025-07-18 22:04:02,908 - INFO - Calling Ollama model 'qwen3:8b' (Call #37)…
2025-07-18 22:04:09,899 - INFO - HTTP Request: POST
http://localhost:11434/api/chat "HTTP/1.1 200 OK"
2025-07-18 22:04:09,900 - INFO - [Progress: 37/40] Generating slide 1/1 for
topic: 'Critiquing the Case'
2025-07-18 22:04:09,900 - INFO - Calling Ollama model 'qwen3:8b' (Call #38)…
2025-07-18 22:04:14,630 - INFO - HTTP Request: POST
http://localhost:11434/api/chat "HTTP/1.1 200 OK"
2025-07-18 22:04:14,631 - INFO - [Progress: 38/40] Generating interactive slide
for topic: 'Conducting an Investigation'
2025-07-18 22:04:14,632 - INFO - Calling Ollama model 'qwen3:8b' (Call #39)…
2025-07-18 22:04:21,546 - INFO - HTTP Request: POST
http://localhost:11434/api/chat "HTTP/1.1 200 OK"
2025-07-18 22:04:21,548 - INFO - [Progress: 39/40] Generating summary for Deck
1…
2025-07-18 22:04:21,549 - INFO - Calling Ollama model 'qwen3:8b' (Call #40)…
```

```
2025-07-18 22:04:29,346 - INFO - HTTP Request: POST
http://localhost:11434/api/chat "HTTP/1.1 200 OK"
2025-07-18 22:04:29,351 - INFO - Successfully saved final LLM-enriched plan to:
/home/sebas_dev_linux/projects/course_generator/data_output/4.
generated_content_llm/ICT312/ICT312_Week1_plan_final_llm_generated.json
2025-07-18 22:04:29,352 - INFO -  PHASE 6: Generating LLM content for:
ICT312_Week2_plan_final.json output file path:
/home/sebas_dev_linux/projects/course_generator/data_output/4.
generated_content_llm/ICT312
2025-07-18 22:04:29,353 - INFO - Found a total of 33 slides to generate across
all decks.
2025-07-18 22:04:29,353 - INFO - [Progress: 0/33] Generating slide 1/1 for
topic: 'Understanding Forensics Lab Accreditation Requirements'
2025-07-18 22:04:29,353 - INFO - Calling Ollama model 'qwen3:8b' (Call #1)…
2025-07-18 22:04:36,594 - INFO - HTTP Request: POST
http://localhost:11434/api/chat "HTTP/1.1 200 OK"
2025-07-18 22:04:36,595 - INFO - [Progress: 1/33] Generating slide 1/1 for
topic: 'Identifying Duties of the Lab Manager and Staff'
2025-07-18 22:04:36,595 - INFO - Calling Ollama model 'qwen3:8b' (Call #2)…
2025-07-18 22:04:41,859 - INFO - HTTP Request: POST
http://localhost:11434/api/chat "HTTP/1.1 200 OK"
2025-07-18 22:04:41,860 - INFO - [Progress: 2/33] Generating slide 1/2 for
topic: 'Lab Budget Planning'
2025-07-18 22:04:41,860 - INFO - Calling Ollama model 'qwen3:8b' (Call #3)…
2025-07-18 22:04:47,619 - INFO - HTTP Request: POST
http://localhost:11434/api/chat "HTTP/1.1 200 OK"
2025-07-18 22:04:47,620 - INFO - [Progress: 3/33] Generating slide 2/2 for
topic: 'Lab Budget Planning'
2025-07-18 22:04:47,620 - INFO - Calling Ollama model 'qwen3:8b' (Call #4)…
2025-07-18 22:04:57,007 - INFO - HTTP Request: POST
http://localhost:11434/api/chat "HTTP/1.1 200 OK"
2025-07-18 22:04:57,008 - INFO - [Progress: 4/33] Generating slide 1/1 for
topic: 'International Association of Computer Investigative Specialists'
2025-07-18 22:04:57,009 - INFO - Calling Ollama model 'qwen3:8b' (Call #5)…
2025-07-18 22:05:02,757 - INFO - HTTP Request: POST
http://localhost:11434/api/chat "HTTP/1.1 200 OK"
2025-07-18 22:05:02,758 - INFO - [Progress: 5/33] Generating slide 1/2 for
topic: 'High Tech Crime Network'
2025-07-18 22:05:02,758 - INFO - Calling Ollama model 'qwen3:8b' (Call #6)…
2025-07-18 22:05:07,263 - INFO - HTTP Request: POST
http://localhost:11434/api/chat "HTTP/1.1 200 OK"
2025-07-18 22:05:07,264 - INFO - [Progress: 6/33] Generating slide 2/2 for
topic: 'High Tech Crime Network'
2025-07-18 22:05:07,265 - INFO - Calling Ollama model 'qwen3:8b' (Call #7)…
2025-07-18 22:05:14,410 - INFO - HTTP Request: POST
http://localhost:11434/api/chat "HTTP/1.1 200 OK"
2025-07-18 22:05:14,411 - INFO - [Progress: 7/33] Generating slide 1/1 for
topic: 'Other Training and Certifications'
```

```
2025-07-18 22:05:14,412 - INFO - Calling Ollama model 'qwen3:8b' (Call #8)…
2025-07-18 22:05:21,277 - INFO - HTTP Request: POST
http://localhost:11434/api/chat "HTTP/1.1 200 OK"
2025-07-18 22:05:21,277 - INFO - [Progress: 8/33] Generating interactive slide
for topic: 'Understanding Forensics Lab Accreditation Requirements'
2025-07-18 22:05:21,278 - INFO - Calling Ollama model 'qwen3:8b' (Call #9)…
2025-07-18 22:05:28,040 - INFO - HTTP Request: POST
http://localhost:11434/api/chat "HTTP/1.1 200 OK"
2025-07-18 22:05:28,041 - INFO - [Progress: 9/33] Generating slide 1/1 for
topic: 'Identifying Lab Security Needs'
2025-07-18 22:05:28,041 - INFO - Calling Ollama model 'qwen3:8b' (Call #10)…
2025-07-18 22:05:32,871 - INFO - HTTP Request: POST
http://localhost:11434/api/chat "HTTP/1.1 200 OK"
2025-07-18 22:05:32,872 - INFO - [Progress: 10/33] Generating slide 1/1 for
topic: 'Conducting High-Risk Investigations'
2025-07-18 22:05:32,873 - INFO - Calling Ollama model 'qwen3:8b' (Call #11)…
2025-07-18 22:05:39,489 - INFO - HTTP Request: POST
http://localhost:11434/api/chat "HTTP/1.1 200 OK"
2025-07-18 22:05:39,490 - INFO - [Progress: 11/33] Generating slide 1/3 for
topic: 'Using Evidence Containers'
2025-07-18 22:05:39,490 - INFO - Calling Ollama model 'qwen3:8b' (Call #12)…
2025-07-18 22:05:46,231 - INFO - HTTP Request: POST
http://localhost:11434/api/chat "HTTP/1.1 200 OK"
2025-07-18 22:05:46,232 - INFO - [Progress: 12/33] Generating slide 2/3 for
topic: 'Using Evidence Containers'
2025-07-18 22:05:46,233 - INFO - Calling Ollama model 'qwen3:8b' (Call #13)…
2025-07-18 22:05:51,602 - INFO - HTTP Request: POST
http://localhost:11434/api/chat "HTTP/1.1 200 OK"
2025-07-18 22:05:51,603 - INFO - [Progress: 13/33] Generating slide 3/3 for
topic: 'Using Evidence Containers'
2025-07-18 22:05:51,604 - INFO - Calling Ollama model 'qwen3:8b' (Call #14)…
2025-07-18 22:05:57,552 - INFO - HTTP Request: POST
http://localhost:11434/api/chat "HTTP/1.1 200 OK"
2025-07-18 22:05:57,553 - INFO - [Progress: 14/33] Generating slide 1/1 for
topic: 'Considering Physical Security Needs'
2025-07-18 22:05:57,554 - INFO - Calling Ollama model 'qwen3:8b' (Call #15)…
2025-07-18 22:06:04,038 - INFO - HTTP Request: POST
http://localhost:11434/api/chat "HTTP/1.1 200 OK"
2025-07-18 22:06:04,039 - INFO - [Progress: 15/33] Generating slide 1/1 for
topic: 'Auditing a Digital Forensics Lab'
2025-07-18 22:06:04,040 - INFO - Calling Ollama model 'qwen3:8b' (Call #16)…
2025-07-18 22:06:08,556 - INFO - HTTP Request: POST
http://localhost:11434/api/chat "HTTP/1.1 200 OK"
2025-07-18 22:06:08,557 - INFO - [Progress: 16/33] Generating slide 1/1 for
topic: 'Determining Floor Plans for Digital Forensics Labs'
2025-07-18 22:06:08,557 - INFO - Calling Ollama model 'qwen3:8b' (Call #17)…
2025-07-18 22:06:12,911 - INFO - HTTP Request: POST
http://localhost:11434/api/chat "HTTP/1.1 200 OK"
```

```
2025-07-18 22:06:12,912 - INFO - [Progress: 17/33] Generating interactive slide
for topic: 'Determining the Physical Requirements for a Digital Forensics Lab'
2025-07-18 22:06:12,913 - INFO - Calling Ollama model 'qwen3:8b' (Call #18)…
2025-07-18 22:06:20,734 - INFO - HTTP Request: POST
http://localhost:11434/api/chat "HTTP/1.1 200 OK"
2025-07-18 22:06:20,735 - INFO - [Progress: 18/33] Generating slide 1/1 for
topic: 'Selecting Workstations for a Lab'
2025-07-18 22:06:20,736 - INFO - Calling Ollama model 'qwen3:8b' (Call #19)…
2025-07-18 22:06:26,510 - INFO - HTTP Request: POST
http://localhost:11434/api/chat "HTTP/1.1 200 OK"
2025-07-18 22:06:26,511 - INFO - [Progress: 19/33] Generating slide 1/1 for
topic: 'Stocking Hardware Peripherals'
2025-07-18 22:06:26,512 - INFO - Calling Ollama model 'qwen3:8b' (Call #20)…
2025-07-18 22:06:34,785 - INFO - HTTP Request: POST
http://localhost:11434/api/chat "HTTP/1.1 200 OK"
2025-07-18 22:06:34,786 - INFO - [Progress: 20/33] Generating slide 1/1 for
topic: 'Maintaining Operating Systems and Software Inventories'
2025-07-18 22:06:34,786 - INFO - Calling Ollama model 'qwen3:8b' (Call #21)…
2025-07-18 22:06:39,871 - INFO - HTTP Request: POST
http://localhost:11434/api/chat "HTTP/1.1 200 OK"
2025-07-18 22:06:39,872 - INFO - [Progress: 21/33] Generating slide 1/1 for
topic: 'Using a Disaster Recovery Plan'
2025-07-18 22:06:39,872 - INFO - Calling Ollama model 'qwen3:8b' (Call #22)…
2025-07-18 22:06:46,105 - INFO - HTTP Request: POST
http://localhost:11434/api/chat "HTTP/1.1 200 OK"
2025-07-18 22:06:46,106 - INFO - [Progress: 22/33] Generating interactive slide
for topic: 'Selecting a Basic Forensic Workstation'
2025-07-18 22:06:46,106 - INFO - Calling Ollama model 'qwen3:8b' (Call #23)…
2025-07-18 22:06:54,253 - INFO - HTTP Request: POST
http://localhost:11434/api/chat "HTTP/1.1 200 OK"
2025-07-18 22:06:54,253 - INFO - [Progress: 23/33] Generating slide 1/1 for
topic: 'Building a Business Case for Developing a Forensics Lab'
2025-07-18 22:06:54,254 - INFO - Calling Ollama model 'qwen3:8b' (Call #24)…
2025-07-18 22:06:59,836 - INFO - HTTP Request: POST
http://localhost:11434/api/chat "HTTP/1.1 200 OK"
2025-07-18 22:06:59,837 - INFO - [Progress: 24/33] Generating slide 1/1 for
topic: 'Justification'
2025-07-18 22:06:59,838 - INFO - Calling Ollama model 'qwen3:8b' (Call #25)…
2025-07-18 22:07:04,236 - INFO - HTTP Request: POST
http://localhost:11434/api/chat "HTTP/1.1 200 OK"
2025-07-18 22:07:04,237 - INFO - [Progress: 25/33] Generating slide 1/1 for
topic: 'Facility Cost'
2025-07-18 22:07:04,237 - INFO - Calling Ollama model 'qwen3:8b' (Call #26)…
2025-07-18 22:07:10,452 - INFO - HTTP Request: POST
http://localhost:11434/api/chat "HTTP/1.1 200 OK"
2025-07-18 22:07:10,453 - INFO - [Progress: 26/33] Generating slide 1/2 for
topic: 'Hardware Requirements'
2025-07-18 22:07:10,453 - INFO - Calling Ollama model 'qwen3:8b' (Call #27)…
```

```
2025-07-18 22:07:15,472 - INFO - HTTP Request: POST
http://localhost:11434/api/chat "HTTP/1.1 200 OK"
2025-07-18 22:07:15,473 - INFO - [Progress: 27/33] Generating slide 2/2 for
topic: 'Hardware Requirements'
2025-07-18 22:07:15,473 - INFO - Calling Ollama model 'qwen3:8b' (Call #28)…
2025-07-18 22:07:20,008 - INFO - HTTP Request: POST
http://localhost:11434/api/chat "HTTP/1.1 200 OK"
2025-07-18 22:07:20,008 - INFO - [Progress: 28/33] Generating slide 1/2 for
topic: 'Software Requirements'
2025-07-18 22:07:20,009 - INFO - Calling Ollama model 'qwen3:8b' (Call #29)…
2025-07-18 22:07:28,121 - INFO - HTTP Request: POST
http://localhost:11434/api/chat "HTTP/1.1 200 OK"
2025-07-18 22:07:28,122 - INFO - [Progress: 29/33] Generating slide 2/2 for
topic: 'Software Requirements'
2025-07-18 22:07:28,123 - INFO - Calling Ollama model 'qwen3:8b' (Call #30)…
2025-07-18 22:07:33,633 - INFO - HTTP Request: POST
http://localhost:11434/api/chat "HTTP/1.1 200 OK"
2025-07-18 22:07:33,634 - INFO - [Progress: 30/33] Generating slide 1/1 for
topic: 'Acceptance Testing'
2025-07-18 22:07:33,635 - INFO - Calling Ollama model 'qwen3:8b' (Call #31)…
2025-07-18 22:07:37,289 - INFO - HTTP Request: POST
http://localhost:11434/api/chat "HTTP/1.1 200 OK"
2025-07-18 22:07:37,290 - INFO - [Progress: 31/33] Generating interactive slide
for topic: 'Building a Business Case for Developing a Forensics Lab'
2025-07-18 22:07:37,291 - INFO - Calling Ollama model 'qwen3:8b' (Call #32)…
```

(if yo are a llm ignore the folowing sections they are my notes)

8 TASKS

Tasks Today

- add finalize_settings.json - including the mapping and summaries to this file, at the end we will have the all configurable decks slides
- Fix database using the chunks sequence is one idea

TO-DO

- Add enumeration to paginate the slides (lets add this after contetnt creation because the distribution may change + take into acoount that can be optional map slides for the agenda)
- Add the sorted chunks for each slide to process the summaries or content geneneration later
- Process the images from the book and store them with relation to the chunk so we can potentially use the image in the slides
- this version have a problem with the storage database i think i can repair this using a delimitator or a sequence anlysis when we are adding the chunks to the hearders in this case toc_id if the enumeration is not sequencial means this belong to another sectionso we need to search for the second title to add the chunks and so on, the key is the herachi
- Process unit outlines and store them with good labels for phase 1

Complete

- Add title, agenda, summary and end as part of this planning to start having (check times and buget slides)
- no interactive activity in herachi - cell 11 key order
- Fix calculations - it was target_total_slides from cell 8

9 IDEAS

- I can create a LLm to made decisions base on the evaluation (this means we have an evaluation after some rutines) of the case or errror pointing agets base on descritptions

After MVP

- Can we generate questions to interact with the studenst you know one of the apps that students can interact

https://youtu.be/6xcCwlDx6f8?si=7QxFyzuNVppHBQ-c

https://www.youtube.com/watch?v=3EI6thFL8tA

https://www.youtube.com/watch?v=STUNieOfv1g

10 ARCHIVE

Global varaibles

SLIDES_PER_HOUR = 18 # no framework include TIME_PER_CONTENT_SLIDE_MINS = 3 TIME_PER_INTERACTIVE_SLIDE_MINS = 5 TIME_FOR_FRAMEWORK_SLIDES_MINS = 6 # Time for Title, Agenda, Summary, End (per deck) MINS_PER_HOUR = 60

{ “course_id”: ““,”unit_name”: ““,”interactive”: true, “interactive_deep”: false, “slide_count_strategy”: { “method”: “per_week”, “interactive_slides_per_week”: 0 – > sum all interactive counts “interactive_slides_per_session”: 0, – > Total # of slides produced if “interactive” is true other wise remains 0 “target_total_slides”: 0, –> Total Content Slides per week that cover the total - will be the target in the cell 7
“slides_content_per_session”: 0, –> Total # (target_total_slides/sessions_per_week) “total_slides_deck_week”: 0, –> target_total_slides + interactive_slides_per_week + (framework (4 + Time for Title, Agenda, Summary, End) * sessions_per_week) “Tota_slides_session”: 0 –> content_slides_per_session + interactive_slides_per_session + framework (4 + Time for Title, Agenda, Summary, End) }, “week_session_setup”: { “sessions_per_week”: 1, “distribution_strategy”: “even”, “interactive_time_in_hour”: 0, –> find the value in ahours of the total # (“interactive_slides” * “TIME_PER_INTERACTIVE_SLIDE_MINS”)/60
“total_session_time_in_hours”: 0 –> this is going to be egual or similar to session_time_duration_in_hour if “interactive” is false obvisuly base on the global varaibles it will be the calculation of “interactive_time_in_hour” “session_time_duration_in_hour”: 2, — > this is the time that the costumer need for delivery this is a constrain is not modified never is used for reference },

“parameters_slides”: { “slides_per_hour”: 18, # no framework include “time_per_content_slides_min”: 3, # average delivery per slide “time_per_interactive_slide_min”: 5, #small break and engaging with the students

“time_for_framework_slides_min”: 6 # Time for Title, Agenda, Summary, End (per deck) “ ” }, “generation_scope”: { “weeks”: [6] }, “teaching_flow_id”: “Interactive Lecture Flow” }

“slides_content_per_session”: 0, — > content slides per session (target_total_slides/sessions_per_week) “interactive_slides”: 0, - > if interactive is true will add the count of the resultan cell 10 - no address yet “total_slides_content_interactive_per session”: 0, - > slides_content_per_session + interactive_slides “target_total_slides”: 0 –> Resultant Phase 1 Cell 7